Consolidated Lecture Notes for Module - 1

Session 1: The AI Landscape for Beginners

Learning Objectives

1. **Explain what Artificial Intelligence is** and how it differs from traditional rule-based software systems.
2. **Differentiate between AI, Machine Learning, Deep Learning, and Generative AI** using real-world examples and a clear hierarchy.
3. **Describe how Machine Learning systems learn from data** using the high-level concept of training and the formula y = f(x).
4. **Compare AI projects with traditional software development projects** in terms of workflow, uncertainty, and success metrics.
5. **Identify key milestones in the evolution of AI** and explain why 2022 marked a major turning point with Generative AI.

1. What is AI?

Core Question:
How can a machine replicate human intelligence?

Simple Analogy – Calculator:

Input: Numbers → Process: Follows rules → Output: Result

AI Components:

- Deep Learning
- Algorithms
- Machine Learning
- Neural Networks
- Classification
- Analysis

---

2. AI vs ML vs Deep Learning vs GenAI

Hierarchy (Each is a subset of the previous)

Artificial Intelligence (Outermost)

- Examples: Spam filters, Self-driving cars
- Techniques that enable computers to emulate human behavior

Machine Learning

- Examples: Netflix recommendations, Fraud detection
- Uses algorithms to detect patterns in large data sets
- Learns without explicit programming

Deep Learning

- Examples: Facial recognition, Speech-to-text
- Uses neural networks with multiple layers
- Extracts high-level features from raw data

Generative AI (The "New Baby")

- Example: ChatGPT
- Generates content such as text, images, or code
- Trained on large datasets using supervised and unsupervised learning

---

3. AI Projects vs Software Development Projects

Aspect	AI Projects	Software Projects
Nature of Work	Exploration, discovery, probabilistic	Deterministic, functional systems
Uncertainty	High (data quality dependent)	Low (requirements defined)
Iteration	Frequent data/model revisiting	Less exploratory (Agile)
Output	Insights, predictions, models	Functional applications
Success Metrics	Accuracy, precision, recall	Works per specifications (binary)
Data Dependency	Central (preprocessing required)	Secondary (input/output)

Example: Fruit Classification

Traditional Approach:

- Banana → Manual code → Identify "Banana"
- Apple → Manual code → Identify "Apple"
- Problem: Rules required for each item

AI / ML Approach:

- 100 photos (50 bananas + 50 apples) → ML Algorithm → Trained Model
- Formula: `y = f(x)`
  - y = output
  - f = algorithm
  - x = input
- 460 photos → Better accuracy
- Model learns patterns automatically

---

4. The Timeline of AI

Key Milestones:

- 1949 – Turing Test proposed
- 1955 – Term "AI" coined
- 1961–1966 – Early robots (ELIZA, SHAKEY)
- 1997–1998 – AI Winter & KISMET

- 1999–2002 – Consumer robots (AIBO, ROOMBA)
- 2011 – SIRI & WATSON
- 2014 – EUGENE & ALEXA
- 2016 – TAY
- 2017 – ALPHAGO defeats Go champion

What Changed in 2022?

Previous 10 Years (2012–2022):

- Focus on data collection and computational power
- Competitive AI demonstrations ("AI chess battles")
- Mostly confined to research labs

2022 Breakthrough (ChatGPT):

- AI became accessible to everyone
- Natural conversation interface
- Shift from theoretical to practical applications
- Democratization of AI

---

5. Why AI is Good / Why AI is Bad

What AI is GOOD at:

- Pattern Recognition – Identifies complex patterns
- Predictions – Forecasts based on historical data
- Classification – Automatically categorizes information
- Text/Image Understanding – Processes language and visuals

What AI is BAD at:

- Common Sense – Lacks real-world understanding
- Truth – Can generate incorrect or hallucinated outputs
- Ethics – No moral reasoning
- Responsibility – Cannot be held accountable
- General Intelligence – Powerful but task-specific

Note: Active research is ongoing to address these limitations.

---

6. Three Phases of Transformation

Phase 1: The Analyst

Theme: Foundations of Data
Symbols: Python | SQL
Goal: Master developer workflow and work with the fuel of AI (Data)

Key Skills:

- Python programming fundamentals
- SQL and database management
- Data cleaning and manipulation
- Working with data pipelines

Outcome: Data Analyst

Phase 2: The Scientist

Theme: Classical Machine Learning
Symbols: ML Algorithms | Visualization
Goal: Discover patterns and make predictions

Key Skills:

- Supervised and unsupervised ML algorithms
- Model training and evaluation
- MLOps (deployment and monitoring)
- Feature engineering

Outcome: ML Engineer / Data Scientist

Phase 3: The Architect

Theme: GenAI & Agents
Symbols: Brain | Chat
Goal: Build dynamic, interactive AI systems

Key Skills:

- Generative AI models and LLMs
- Prompt engineering
- AI agents and autonomous systems
- Interactive AI application development

Outcome: AI Architect

The Summit – Capstone

Goal: Integrate all skills into one comprehensive project
Deliverable: Production-ready AI application demonstrating full-stack AI capabilities

---

Progressive Learning Path

Analyst → Python + SQL, Data Foundation
⬇️
Scientist → Classical ML + MLOps, Pattern Discovery
⬇️
Architect → GenAI + AI Agents, Dynamic Systems
⬇️
Summit → Capstone Project, Full Integration
⬇️
AI Professional

Summary & Key Takeaways

- **AI Hierarchy:** AI ⊃ ML ⊃ DL ⊃ GenAI (GenAI is the newest “baby”)
- **AI vs Traditional Software:** AI learns from data; software follows explicit rules

- **2022 Turning Point:** ChatGPT made AI accessible to everyone
- **AI Strengths:** Patterns, predictions, classification, understanding
- **AI Limitations:** Common sense, truth, ethics, responsibility, general intelligence

Three Core Skill Sets:

- Data foundations (Python, SQL)
- Classical ML (algorithms, models)
- Modern AI (GenAI, agents)

End of Lecture

Session 2: Computing & Programming Foundations

What Is a Computer?

What you'll learn:

- The three fundamental operations of a computer
- How computers process information
- The basic workflow of computing

Detailed Explanation:

A computer only does **3 things**:

1. **Input** – You give something
2. **Process** – It follows instructions
3. **Output** – It shows result

The computer processes input to produce output, involving storage, processing, and display components.

Example:

Think of a simple calculator:

- **Input:** You press buttons for "1 + 2 ="
- **Process:** Calculator performs the addition
- **Output:** Screen displays "3"

This same Input → Process → Output model applies to all computing, from simple calculations to complex applications.

Key Takeaways:

- Every computer operation follows: Input → Process → Output
- Storage, processing, and display are involved
- The process step follows specific instructions
- This model applies to all types of computers

Hardware vs Software

What you'll learn:

- What hardware components are
- What software applications are

- The relationship between hardware and software

Detailed Explanation:

A computer has two main parts:

Hardware:

Physical components you can touch and see:

- **Keyboard**
- **Mouse**
- **Screen**
- **CPU** (Central Processing Unit)

CPU acts as the brain, handling general processing tasks.

GPU (Graphics Processing Unit) handles graphics-intensive tasks and is crucial for:

- **Machine learning**
- **Gaming**
- Video rendering

GPU cooling is important due to high power usage.

Software:

Programs and applications that run on hardware:

- **WhatsApp**
- **Chrome**
- **Calculator**
- **Music player**

Software instructs the hardware on what tasks to perform.

Key Takeaways:

- Hardware = Physical parts (keyboard, mouse, screen, CPU)
- Software = Programs (WhatsApp, Chrome, Calculator, Music player)
- CPU handles general processing
- GPU handles graphics and machine learning, needs cooling
- Software tells hardware what to do

What Is a Kernel?

What you'll learn:

- The kernel's role in the operating system
- How it manages computer resources
- Why it's essential for smooth operation

Detailed Explanation:

The kernel handles essential tasks to keep your computer humming along smoothly.

Core Function:

It manages **memory**, ensuring that programs get the resources they need.

What the Kernel Manages:

- **CPU** – Processing power allocation
- **Memory** – RAM distribution to programs
- **Devices** – Keyboard, mouse, storage, peripherals
- **Processes** – Running multiple programs simultaneously

How It Works:

The kernel acts as a bridge between:

- **Hardware** (physical components)
- **Software applications** (programs you use)

It coordinates communication and resource allocation between these layers.

Building Analogy:

Think of a company building:

- **Hardware** = Building infrastructure (electricity, plumbing, structure)
- **Kernel** = Building management system (controls and coordinates everything)
- **Software** = Employees working in the building

The building management ensures everything runs smoothly—power, elevators, security, climate control. Similarly, the kernel manages all computer resources.

Critical Importance:

If the kernel crashes or fails, the entire computer system stops working. It's that fundamental to operation.

Key Takeaways:

- Kernel handles essential tasks for smooth computer operation
- Manages memory, CPU, devices, and processes
- Acts as bridge between hardware and software
- Ensures programs get resources they need
- Kernel failure = Complete system failure

What Is Software?

What you'll learn:

- What software represents in a computer system
- The relationship between software and hardware

Detailed Explanation:

Think of a computer as the **body** and the software as its **brain**.

Software Definition:

Software is the set of instructions that tells the hardware what to do and how to do it.

Without software:

- Hardware is just physical components
- No tasks can be performed

- Computer cannot function

With software:

- Hardware receives instructions
- Tasks are executed
- Computer becomes useful

Examples of Software:

- Operating systems (Windows, macOS, Linux)
- Applications (Word, Excel, PowerPoint)
- Web browsers (Chrome, Firefox)
- Communication tools (Zoom, Slack)
- Entertainment (Music players, Games)

The Relationship:

Just as the brain controls the body by sending signals and instructions, software controls hardware by sending commands and directing operations.

Key Takeaways:

- Computer = Body
- Software = Brain
- Software instructs hardware on what to do
- Hardware without software is non-functional
- Software makes hardware useful

What Is Program?

What you'll learn:

- What a program is
- How programs differ from regular code
- The role of compilers and interpreters

Detailed Explanation:

A program is a **written instruction note** for a computer.

Instructions Written by Humans:

Programs contain instructions written by humans in programming languages like:

- Python
- C
- Java
- JavaScript

The Challenge:

Humans write in programming languages, but computers only understand **binary** (0s and 1s).

Example Question: "Where is Kakinada?"

- **Humans** understand this question in language
- **Computers** only understand: 01001011 01100001...

The Solution: Compiler & Interpreter

These are **translators** that convert human-written code into machine language.

Compiler:

- Translates **entire code** before execution
- Creates executable file
- Faster execution
- Shows errors after complete compilation

Interpreter:

- Translates **line-by-line** during execution
- No separate executable
- Slower execution
- Shows errors immediately

Analogy:

Think of it like language translation:

- You speak English
- Computer speaks Binary
- Compiler/Interpreter = Translator between you and computer

Key Takeaways:

- Program = Written instructions for computer
- Instructions written by humans in programming languages
- Computers only understand binary (0s and 1s)
- Compiler translates entire code at once
- Interpreter translates line-by-line
- Both convert human code to machine language

What Is Data?

What you'll learn:

- What constitutes data
- The hierarchy from data to wisdom
- How to extract value from information

Detailed Explanation:

Examples of Data:

- Marks
- Height
- Age
- Messages
- Photos

Everything you capture, observe, or record is data.

The Value Pyramid:

```
    Wisdom
      ↑
 Knowledge
      ↑
Information
      ↑
    Data
```

1. Data = Raw facts without context

- Just numbers, text, images
- No meaning by itself

2. Information = Processed data with context

- Data that has been organized
- Given meaning through relationships

3. Knowledge = Insights from information

- Understanding patterns
- Useful for decision-making

4. Wisdom = Strategic action

- Using knowledge to create value
- Making important decisions

Real Example: Amazon Patterns

Data:

- User visits at 8:00 AM
- Browses for 20 minutes
- Views electronics
- Adds items to cart
- Doesn't purchase

Information:

- Morning browser pattern
- Interested in electronics
- Price-sensitive behavior

Knowledge:

- User browses early, buys later
- Needs discount trigger
- High intent, low conversion

Wisdom:

- Send personalized offer at evening
- Time discount for maximum impact
- Result: Increased sales and revenue

Key Takeaways:

- Data = Raw facts (marks, height, age, messages, photos)
- Information = Data with context
- Knowledge = Insights for decisions
- Wisdom = Strategic action creating value
- Processing data creates business value

What Is Logic?

What you'll learn:

- What logic means in programming
- How decisions are made based on conditions
- What decision trees are and why they matter

Detailed Explanation:

Logic means **making a decision based on a condition**.

In very simple words:

1. **Something happens** (a condition exists)
2. **You decide what to do** (evaluate the condition)
3. **Based on a rule** (predetermined logic)

Decision Tree:

A decision tree is a visual representation of logic flow.

Simple Example:

```
    Traffic Signal
         ↓
    Is it Red?
      ↙    ↘
    YES     NO
     ↓       ↓
    STOP    GO
```

More Complex Example:

```
        Person
          ↓
    Age > 30?
      ↙      ↘
    YES       NO
     ↓         ↓
  Eats      Income
  Pizza?    Level?
   ↙  ↘
 YES  NO
  ↓    ↓
UNFIT FIT
```

Real Applications:

Decision trees are used in:

- **Programming** – If-else statements
- **Machine Learning** – Classification algorithms
- **Business Logic** – Workflow automation
- **AI Systems** – Decision-making processes

Why It Matters:

Understanding decision trees is fundamental because:

- All programming uses logic
- Machine learning heavily relies on decision trees
- Problem-solving requires logical thinking

Key Takeaways:

- Logic = Making decisions based on conditions
- Decision tree shows the flow of logic
- Something happens → You decide → Based on rule
- Decision trees are fundamental to programming
- Used extensively in machine learning

Traditional Software

What you'll learn:

- What traditional software is
- Its limitations compared to modern AI systems

Detailed Explanation:

Traditional software works only by **fixed rules written by humans**.

Three Key Characteristics:

1. **Does not learn** – Cannot acquire new knowledge from data
2. **Does not change** – Behavior remains constant
3. **Does not improve** – No automatic enhancement over time

How It Works:

- Programmers write explicit rules
- Software follows these rules exactly
- Same input → Same output (always)
- No adaptation to new situations

Examples:

- Calculator (always follows same math rules)
- Spell checker (fixed dictionary)
- Traditional banking software (predefined rules)

Limitations:

If a new scenario arises:

- Software cannot handle it automatically

- Human programmer must write new code
- System must be updated manually
- No self-improvement capability

Contrast with AI/ML (covered later in course):

- AI learns from data
- Adapts to new patterns
- Improves with experience
- Handles unforeseen situations

Key Takeaways:

- Traditional software = Fixed rules only
- Does not learn, change, or improve
- Humans must program every scenario
- Requires manual updates for new situations
- Foundation for understanding why AI is different

What Is IDE & Terminal (CLI)?

What you'll learn:

- What an IDE is and its purpose
- What a terminal is and how it works
- The difference between IDE and terminal

Detailed Explanation:

IDE (Integrated Development Environment)

IDE is a workplace for writing code.

An IDE provides a complete environment for:

- Writing code
- Editing code
- Debugging code
- Running programs

Examples:

- **VS Code** – Free, supports all languages
- **PyCharm** – Python-focused IDE

Why Use an IDE:

- Syntax highlighting (colors for different code parts)
- Autocomplete (suggests code as you type)
- Error detection (shows mistakes immediately)
- Integrated tools (all features in one place)

VS Code Special Features:

- Completely **free**
- **Open source** (code publicly available)
- Supports multiple languages (Python, JavaScript, Java, C++)

- Large extension library
- Used by millions of developers

Terminal (Command Line Interface - CLI)

Terminal is a text-based way to talk to computer.

Instead of clicking buttons and icons, you type commands directly.

What You Can Do:

- Run programs: `python script.py`
- Navigate folders: `cd folder_name`
- Create files: `touch file.py`
- Install software: `pip install package`

How It Works:

```
You type command → Terminal sends to OS → Kernel executes → Result shown
```

Examples of Terminals:

- Windows: Command Prompt, PowerShell
- Mac/Linux: Terminal, Bash

Difference: Terminal vs Kernel

Terminal:

- User interface
- Where you type commands
- Text-based interaction
- Like a reception desk

Kernel:

- Core OS component
- Manages hardware
- Executes commands
- Like building management

Terminal sends commands → **Kernel** executes them

IDE Integration

IDEs often integrate terminals:

- Write code in IDE editor
- Click "Run" button
- IDE uses terminal in background
- Output shown in IDE

Key Takeaways:

- IDE = Workplace for writing code (VS Code, PyCharm)
- Provides editor, debugger, and tools
- Terminal = Text-based way to talk to computer
- Terminal is interface; Kernel executes

- IDEs often include integrated terminals
- Both essential for development

What Is Files, File Types?

What you'll learn:

- What files are
- Common file types and extensions
- How folders organize files

Detailed Explanation:

File = Document

A file is a container for specific data.

Examples:

- **Photo file** – Contains image data
- **Song file** – Contains audio data
- **Video file** – Contains video data

File Types and Extensions:

Different extensions indicate different data formats:

Type	Extension	Example
Images	.png, .jpg	photo.png
Code	.py, .js	script.py
Documents	.pdf, .txt	report.pdf
Audio	.mp3	song.mp3
Video	.mp4	video.mp4

Folder = Bag of Files

Folders organize and group related files together.

Example Project Structure:

```
my_project/
├── main.py
├── data.csv
├── images/
│   ├── logo.png
│   └── photo.jpg
└── documents/
    └── readme.txt
```

Benefits of Organization:

- Easy to find files
- Logical grouping
- Better project management
- Clear structure

Open Source

Open Source means the code is publicly available.

VS Code Example:

- Microsoft created VS Code
- Made the code **public**
- Anyone can see the code
- Developers can **fork** (copy) it
- Make their own changes
- Create custom versions

Popular VS Code Forks:

- Cursor (AI-powered)
- Codium
- Other customized editors

Benefits:

- Free to use
- Community contributions
- Transparency
- Customization possible

Key Takeaways:

- File = Document containing data
- Extensions indicate type (.png, .py, .pdf, .mp3, .mp4)
- Folder = Bag organizing related files
- Open source = Publicly available code
- VS Code is open source, can be forked
- Proper organization improves workflow

How Code Runs

What you'll learn:

- The complete process from writing to execution
- How components work together

Detailed Explanation:

The complete process of code execution:

Step 1: You write code in a .py file

You create a Python file with your instructions:

```
# example.py
```

```python
print("Hello, World!")
```

Save it with `.py` extension (indicates Python file).

Step 2: Python reads it line by line

When you run the program:

- Python interpreter activates
- Opens your .py file
- Reads code line by line
- Processes each instruction sequentially

Step 3: Computer executes instructions

The interpreter:

- Translates Python code to machine code (binary)
- Sends to computer's CPU
- Kernel manages resources
- Hardware executes the operations

Step 4: Output appears on screen

Results are displayed:

```
Hello, World!
```

Complete Flow Diagram:

```
Write Code (.py file)
        ↓
Python Interpreter Reads
        ↓
Translates to Machine Code
        ↓
Computer Executes (CPU + Kernel)
        ↓
Output Shows on Screen
```

Behind the Scenes:

When you click "Run" in your IDE:

1. IDE saves your file
2. Calls Python interpreter via terminal
3. Interpreter processes your code
4. Kernel manages hardware resources
5. CPU executes binary instructions
6. Results display in IDE output panel

Session 3: Setting Up Your Lab

Learning Objectives:

- Install and configure VS Code for Python development
- Use Google Colab as the primary coding platform
- Understand and manage API keys securely
- Create a GitHub account for version control

Visual Studio Code Installation

What you'll learn:

- How to install VS Code on Windows and macOS
- Setting up Python extension
- Creating and running Python files

Detailed Explanation:

VS Code is a **free, open-source code editor** that supports almost all programming languages. It's widely used for writing, debugging, and building applications.

Computer Requirements:

- Minimum: 8GB RAM, 512GB storage
- GPU beneficial but not mandatory
- Alternative: Use Google Colab (free cloud resources)

Installation Process:

Windows:

1. Visit https://code.visualstudio.com
2. Download Windows installer
3. Run the installer
4. **CRITICAL STEP:** Enable "Add to PATH" during installation (allows terminal to recognize Python)
5. Complete installation

macOS:

1. Download appropriate version (Intel chip OR Apple Silicon M1/M2/M3/M4)
2. Double-click the downloaded file
3. Drag VS Code to Applications folder

Setting Up Python:

1. Open VS Code
2. Click Extensions icon (left sidebar)
3. Search for "Python"
4. Install the Python extension (over 120M downloads)

Creating Your First Python File:

1. File → Open Folder → Select/create project folder
2. Click "New File" icon
3. Name with `.py` extension (e.g., `test.py`)
4. Write code:

```
print("My name is Masai")
print(10 + 2)
```

5. Click ▶️ Run button
6. Output appears in terminal

Virtual Environment (Optional): If prompted, select interpreter → Create Virtual Environment → Choose venv → Select Python version

Key Takeaways:

- VS Code is free and supports all programming languages
- "Add to PATH" is essential on Windows
- Python extension required for Python development
- `.py` files execute completely when you click Run

Google Colab Platform

What you'll learn:

- Accessing free cloud computing resources
- Creating and running code notebooks
- Managing runtime and files

Detailed Explanation:

Google Colab (Colaboratory) is a **free cloud-based platform** that provides powerful computing resources for learning and experimentation.

Why Use Google Colab?

The problem: Not everyone has powerful laptops for coding, especially machine learning.

The solution: Google provides access to their powerful servers for **FREE**.

How It Works:

```
Your Laptop → Browser → Internet → Google Servers (Powerful Computer)
```

When connected, you're actually coding on Google's remote computer with:

- **12-16GB RAM** (free)
- **100+ GB storage** (free)
- **GPU access** (T4 GPU, free)

Getting Started:

1. Go to https://colab.research.google.com
2. Sign in with Google account
3. Click "New Notebook"
4. Click "Connect" (top right) to access resources

Understanding the Interface:

File Name: Click "Untitled0.ipynb" at top to rename

Cells:

- Click "+ Code" to add code cell
- Click "+ Text" to add documentation (Markdown)

Left Sidebar:

- 📁 Files: Upload files here
- 🔑 Secrets: Store API keys (covered next)

Executing Code:

Example:

```
print("My name is Masai")
print(10 + 2)
```

To run:

- Click ▶️ play button, OR
- Press Ctrl+Enter (Windows) or Cmd+Enter (Mac)
- Output appears directly below the cell

File Formats:

`.py` (Python Script):

- Entire file executes at once
- Used for production/deployment
- Runs in terminal

`.ipynb` (Interactive Notebook):

- Execute cell-by-cell
- See output immediately after each cell
- Perfect for learning and experimentation
- Google Colab uses this format

File Management:

Uploading Files:

1. Click 📁 folder icon (left sidebar)
2. Click upload button
3. Select file from computer

IMPORTANT: Uploaded files are **temporary** and deleted when runtime disconnects. Your notebook (`.ipynb`) is automatically saved to Google Drive.

Runtime Management:

Connecting:

- Click "Connect" to access Google's computer
- Green checkmark appears when connected
- Shows available RAM and disk space (e.g., 12.7 GB RAM)

Disconnecting (VERY IMPORTANT):

1. Go to Runtime → Disconnect and delete runtime

2. This frees Google's resources for other students
3. Your code is **NOT deleted** (saved to Google Drive)
4. Only temporary files and variables are cleared
5. **Always disconnect when finished working**

Key Takeaways:

- Google Colab provides free powerful computing resources
- No installation needed, works in browser
- `.ipynb` files allow cell-by-cell execution
- Notebooks auto-save to Google Drive
- Always disconnect runtime when done to free resources for others

---

Managing Secrets and API Keys

What you'll learn:

- What APIs and API keys are
- Why API keys must be protected
- How to use Google Colab Secrets Manager

Detailed Explanation:

What is an API?

API = Application Programming Interface

Think of a **restaurant analogy:**

```
You (Customer) → Waiter (API) → Kitchen (Server)
                      ↓
                 Food Returns
```

The waiter (API) carries your request to the kitchen and brings back your food (data).

Real Example - ChatGPT:

```
Your Browser → Question → Internet → OpenAI Server (ChatGPT)
                                            ↓
                            Answer ← AI processes question
```

The API is the bridge that connects your browser to OpenAI's ChatGPT server.

What are API Keys?

API keys are **unique security tokens** that:

- Authenticate who is making the request
- Track usage for billing purposes
- Prevent unauthorized access

Analogy: Like a house key - only people with the correct key can enter.

Creating API Keys:

Example with Google Gemini API:

1. Go to Google AI Studio
2. Sign in with Google account
3. Click "Create API Key"
4. Name it (e.g., "Gemini_API_Key")
5. Copy the generated key

Generated key looks like:

```
AIzaSyD1234567890abcdefghijklmnop
```

Billing:

- Most APIs charge based on usage
- Free tiers available for testing
- Dashboard shows usage (e.g., "Last 7 days: $1.11 USD")

⚠️ **CRITICAL SECURITY RULE:**

NEVER SHARE API KEYS WITH ANYONE

Why?

1. Anyone using your key → Charges to YOUR account
2. YOU pay for all usage, even unauthorized
3. No refunds for misuse

If key leaks:

1. Immediately delete the key
2. Generate a new key
3. Update your code

The Problem with Hardcoding:

BAD Practice:

```
# DON'T DO THIS - Key visible to everyone!
api_key = "AIzaSyD1234567890"
response = call_api(api_key)
```

If you share this code → Others copy your key → You get unexpected bills

The Solution: Google Colab Secrets Manager

Secrets Manager securely stores API keys so they remain hidden from your code.

How to Use:

Step 1: Store Your API Key

1. In Google Colab, click 🔑 key icon (left sidebar)
2. Click "Add new secret"
3. Name: `GEMINI_API_KEY`
4. Value: Paste your actual API key
5. Toggle "Notebook access" to ON
6. Click Save

Step 2: Access Secret in Code

```
from google.colab import userdata

# Get API key securely (actual value remains hidden)
api_key = userdata.get('GEMINI_API_KEY')

# Use in your code
# When others view notebook, they see variable name, NOT actual key
```

Benefits:

- API key never appears in notebook code
- Safe to share notebook publicly
- Secrets saved to your Google account
- Available across all your notebooks
- Survives runtime disconnections

Best Practices:

DO:

- Use Secrets Manager for all API keys
- Delete leaked keys immediately
- Use different keys for different projects

DON'T:

- Hardcode keys directly in code
- Share keys via email or chat
- Take screenshots showing visible keys
- Commit keys to GitHub

Key Takeaways:

- APIs connect your code to external services
- API keys authenticate requests and track billing
- NEVER share or hardcode API keys
- Use Google Colab Secrets Manager to store keys securely
- Access keys using `userdata.get('KEY_NAME')`

Creating GitHub Account

What you'll learn:

- What GitHub is and why it's important
- How to create your GitHub account
- Basic understanding of version control

Detailed Explanation:

What is GitHub?

GitHub is a web-based platform for storing, tracking, and managing code.

Why Do We Need GitHub?

Imagine a team of 10 developers building an application like Swiggy:

- Person 1 works on Orders module
- Person 2 works on Payment system
- Person 3 works on Login feature
- Person 4 works on Rider tracking

Problems:

- How to combine everyone's code?
- What if two people edit the same file?
- How to track who made which changes?
- How to revert if new code breaks the app?
- How to manage different versions (v1.0, v2.0, v3.0)?

Solution: GitHub handles all of this through version control.

Key Concepts:

Repository (Repo):

- A project folder containing all your code files
- Can be public (anyone can see) or private (only you and invited collaborators)

Version Control:

- Saves snapshots of your code at different points in time
- Can revert to previous versions if something breaks
- Tracks all changes made by all team members

Example timeline:

```
Version 1.0 → Version 1.1 → Version 2.0 → Version 3.0
```

If Version 3.0 has a critical bug, you can quickly revert to Version 2.0.

Creating Your GitHub Account:

Steps:

1. Go to https://github.com
2. Click "Sign Up"
3. Enter email, password, and username
4. Verify your email address
5. Complete the setup process

Tip: You can also sign up using your Google account for easier access.

What You Can Do with GitHub:

Store Code:

- Upload your projects to the cloud
- Keep code safe and accessible from anywhere

Collaborate:

- Work with team members on the same project
- Share code with others

- Contribute to open-source projects

Build Portfolio:

- Showcase your work to potential employers
- Demonstrate your coding skills

What's Next:

Detailed Git commands and repository management will be covered in future sessions.

For now:

- Create your GitHub account
- Explore the interface
- Familiarize yourself with the platform

Key Takeaways:

- GitHub stores and tracks all code changes over time
- Essential for team collaboration and version control
- Create account at github.com (can use Google account)
- Detailed usage and Git commands covered in future lectures

Summary

Lab Setup Complete

VS Code:

- Free, open-source code editor
- Enable "Add to PATH" during Windows installation
- Install Python extension
- Create `.py` files and run code

Google Colab:

- Free cloud platform with 12-16GB RAM
- No installation required
- Use `.ipynb` notebooks for cell-by-cell execution
- **Always disconnect runtime when finished**

API Keys & Secrets:

- API keys authenticate and track service usage
- **NEVER share or hardcode API keys**
- Use Google Colab Secrets Manager
- Access via `userdata.get('KEY_NAME')`

GitHub:

- Platform for code storage and version control
- Create account at github.com
- Essential for collaboration
- Detailed usage in future sessions

Best Practices

- Disconnect Google Colab runtime when done
- Always use Secrets Manager for API keys
- Never hardcode sensitive information
- Keep GitHub account ready for future use

End of Lecture

Session 4: The Developer Workflow

Introduction

A **developer workflow** is the step-by-step process professionals use to organize and save their work. Just like a recipe keeps cooking organized, a workflow keeps your files and changes organized.

What is the Command Line?

A **command line** (also called terminal or CLI) is a text-based way to control your computer by typing commands instead of clicking.

Why Learn This?

- **Faster** than clicking
- **More powerful** control
- **Required** for many tools
- **Industry standard**

Essential Commands

You need just 5 commands to start:

1. `pwd` **- Print Working Directory** Shows where you are:

```
pwd
```

Output: `/Users/yourname/Desktop`

2. `ls` **- List Files** Shows files in current location:

```
ls        # Mac/Linux
dir       # Windows
```

3. `cd` **- Change Directory** Move between folders:

```
cd Desktop     # Go to Desktop
cd ..          # Go up one level
cd ~           # Go to home
```

4. `mkdir` **- Make Directory** Create a folder:

```
mkdir my-project
```

5. `touch` - **Create File** Create an empty file:

```
touch notes.txt      # Mac/Linux
type nul > notes.txt # Windows
```

Practice Exercise

Try these commands in order:

```
cd Desktop
mkdir ai-learning
cd ai-learning
pwd
ls
```

Git: Version Control

What is Git?

Git tracks every change you make to your files. It's like having infinite undo with a complete history.

The Problem Git Solves

Without Git:

```
project_final.zip
project_final_v2.zip
project_REAL_final.zip
```

With Git: Every change is tracked automatically.

The Three Stages

Git has three stages you must understand:

```
Working Directory → Staging Area → Repository
    (editing)         (preparing)      (saved)
```

Stage 1: Working Directory
Files you're currently editing

Stage 2: Staging Area
Changes you've selected to save

Stage 3: Repository
Permanently saved history

Essential Git Commands

The Assignment Operator (git commands)

Just like `=` assigns values, Git commands move files through stages.

1. `git init` - Initialize Start Git in a folder (once per project):

```
git init
```

2. `git add` - Stage Changes Prepare files to save:

```
git add notes.txt    # Add one file
git add .            # Add all files
```

3. `git commit` - Save Changes Permanently save with a message:

```
git commit -m "Add meeting notes"
```

Good messages: ✅ "Add project notes"
✅ "Fix typo in README"

Bad messages: ❌ "stuff"
❌ "changes"

4. `git status` - Check Status See what changed:

```
git status
```

5. `git log` - View History See all saves:

```
git log --oneline
```

The Complete Workflow

Step-by-Step Process

Every work session follows this pattern:

```
1. cd my-project          (Navigate)
2. git status             (Check status)
3. [Edit files]           (Make changes)
4. git add .              (Stage)
5. git commit -m "..."    (Save)
6. git log --oneline      (View history)
```

Real Example

```
# Navigate
cd Desktop/my-notes

# Check what changed
git status

# Edit notes.txt (add content in text editor)

# Stage the change
git add notes.txt

# Save with message
git commit -m "Add learning notes"

# View history
git log --oneline
```

Practice: Complete Project

Follow these steps exactly:

Step 1: Create Project

```
cd Desktop
mkdir my-first-project
cd my-first-project
git init
```

Step 2: Create File

```
touch notes.txt
```

Step 3: Add Content Open `notes.txt` in any text editor and type: ``` My Learning Notes

Commands I learned:

- pwd (show location)
- cd (change folder)
- mkdir (create folder)
- git init (start tracking)
- git add (stage changes)
- git commit (save changes)

```
**Step 4: Save to Git**
```bash
```
```

```
git status
git add notes.txt
git commit -m "Create learning notes"
```

Step 5: Make Changes Add more to `notes.txt` :

```
Git is like save + history for my work!
```

Step 6: Save Again

```
git add notes.txt
git commit -m "Add Git description"
```

Step 7: View History

```
git log --oneline
```

You should see two commits!

Notebooks vs. Scripts

Two Ways to Work

Notebooks (`.ipynb`):

- Work cell by cell
- See results immediately
- Good for: Learning, exploring, experimenting

Scripts (`.py` , `.txt`):

- Complete files that run together
- Good for: Finished programs, repeated tasks

When to Use Each

Use Notebooks when:

- You're learning something new
- You want to try different things
- You need to see results as you go

Use Scripts when:

- You know what you're building
- The work will run multiple times
- You're sharing finished work

Best Practice: Start in notebook to explore → Move to script when ready

---

Common Mistakes

Mistake 1: Not Committing Often

❌ **Bad:** Work all day, commit once
✅ **Good:** Commit after each task

Mistake 2: Bad Messages

❌ **Bad:** `git commit -m "stuff"`
✅ **Good:** `git commit -m "Add contact list"`

Mistake 3: Forget to Check Location

✅ **Always run `pwd` first!**

Mistake 4: Not Using Git Status

✅ **Run `git status` before every commit**

Key Takeaways

Command Line:

- Use `pwd` to know where you are
- Use `cd` to move around
- Use `mkdir` to create folders
- Use `touch` to create files

Git:

- `git init` - Start tracking (once)
- `git add` - Select changes
- `git commit -m "..."` - Save with message
- `git status` - Check what changed
- `git log` - See history

Workflow:

1. Navigate to project
2. Check status
3. Make changes
4. Stage changes
5. Commit with good message
6. Repeat

Remember: This is the same workflow used at Google, Microsoft, and OpenAI!

What's Next?

You now know the professional developer workflow. The commands are simple, but they're powerful.

Practice daily - this becomes automatic with repetition!

Session 5: Python: The Language of AI

1. Variables

What is a Variable?

A **variable** is a named location in memory that stores a value.

Analogy: Like a labeled box - the label is the variable name, contents is the value.

Creating Variables

```
variable_name = value
```

Assignment Operator (=)

```
age = 25  # Read as "age is assigned 25"
```

- `=` is assignment, NOT mathematical equality
- Variable name on LEFT, value on RIGHT

Examples

```
age = 25
name = "Alex"
price = 19.99
is_active = True
```

Memory Address

```
age = 25
print(id(age))  # Shows memory address: 11655144
```

Variable Reassignment

```
health = 100
health = 75    # Old value discarded
health = 100
```

Multiple Variables

```
# Separate lines
player_name = "DragonSlayer"
health = 100
level = 5
```

```
# Same line
x, y, z = 10, 20, 30

# Same value
a = b = c = 0
```

Using Variables

```
price = 100
quantity = 3
total = price * quantity

score = 100
score = score + 50  # Update based on current value
```

Common Mistakes

Mistake 1: Using Before Creating

```
print(name)  # ERROR: NameError: name 'name' is not defined
```

Fix:

```
name = "Alice"
print(name)  # Works!
```

Mistake 2: Wrong Assignment Direction

```
25 = age  # ERROR: SyntaxError
```

Fix:

```
age = 25  # Variable on LEFT
```

---

2. Variable Naming Conventions

Naming Rules (MUST FOLLOW)

Rule 1: Start with Letter or Underscore

```
# Valid
age = 25
_value = 100
user_name = "Alex"
```

```
# Invalid
2age = 30      # ERROR: Can't start with number
@user = "test"  # ERROR: Can't use special characters
```

Rule 2: Only Letters, Numbers, Underscores

```
# Valid
student_name = "Alice"
score_2024 = 95
age2 = 30

# Invalid
student-name = "Bob"   # ERROR: No hyphens
total$ = 100           # ERROR: No special characters
```

Rule 3: Case Sensitive

```
age = 25
Age = 30
AGE = 35

print(age)  # 25
print(Age)  # 30
print(AGE)  # 35
# These are THREE different variables!
```

Rule 4: No Reserved Keywords

Cannot use Python keywords as variable names:

```
# Invalid
for = 10      # ERROR: 'for' is reserved
if = 20       # ERROR: 'if' is reserved
while = 30    # ERROR: 'while' is reserved
class = 40    # ERROR: 'class' is reserved
```

Common Reserved Keywords:

- for, while, if, else, elif
- def, class, return
- import, from, as
- True, False, None

Naming Conventions (BEST PRACTICE)

Use snake_case (Pythonic Style)

```
# Good (Python style)
student_name = "Alice"
total_score = 100
is_active = True

# Bad (Not Pythonic)
studentName = "Alice"   # camelCase - not Python style
StudentName = "Alice"   # PascalCase - not for variables
```

Be Descriptive

```
# Good (Clear and descriptive)
student_count = 30
average_score = 87.5
user_email = "test@example.com"

# Bad (Unclear abbreviations)
sc = 30
avg = 87.5
ue = "test@example.com"
```

Constants in UPPER_CASE

```
# Constants (values that don't change)
MAX_SIZE = 100
PI = 3.14159
TAX_RATE = 0.08
DEFAULT_TIMEOUT = 30
```

---

3. Data Types

Python Data Types Overview

Type	Example	Description
int	`x = 10`	Whole numbers
float	`x = 10.5`	Decimal numbers
str	`name = "Atharv"`	Text
bool	`is_ok = True`	True/False

Note: Python also has list, tuple, set, dict (covered in advanced topics)

Integer (int)

Definition: Whole numbers with no decimal point (positive, negative, or zero)

```
age = 25
year = 2024
temperature = -10
score = 0

print(type(age))  # <class 'int'>
```

Arithmetic Operations

```
a = 10
b = 3

print(a + b)   # 13  Addition
print(a - b)   # 7   Subtraction
print(a * b)   # 30  Multiplication
print(a ** b)  # 1000 Power (10³)

# Division
print(a / b)   # 3.333...  (always returns float)
print(a // b)  # 3         (floor division, returns int)

# Modulo (remainder)
print(a % b)   # 1

# Check if even
if a % 2 == 0:
    print("Even")
```

Important:

- No size limit in Python 3
- int + float = float

Float (float)

Definition: Numbers with decimal points

```
price = 19.99
height = 5.8
pi = 3.14159

print(type(price))  # <class 'float'>
```

Note: `5.0` is float, `5` is int

Arithmetic

```
a = 10.5
b = 2.5

print(a + b)  # 13.0
print(a * b)  # 26.25
print(10 / 2) # 5.0 (division always returns float)
```

Precision Limitation

```
result = 0.1 + 0.2
print(result)            # 0.30000000000000004 (not exactly 0.3!)
print(round(result, 2))  # 0.3 (use round() for display)
```

Why? Binary representation can't store some decimals exactly.

Common Uses: Prices, measurements, percentages, scientific values

---

String (str)

Definition: Text data in quotes

```
name = 'Alice'          # Single quotes
message = "Hello!"      # Double quotes (both work)
```

String Operations

Concatenation:

```
first = "Hello"
second = "World"
result = first + " " + second  # "Hello World"

# Can't mix string and numbers
age = 25
message = "Age: " + str(age)   # Convert first
```

Repetition:

```
print("ha" * 3)    # "hahaha"
print("=" * 20)    # "===================="
```

Length:

```
name = "Alice"
print(len(name))   # 5
```

String Methods

```
name = "Alice"
print(name.upper())             # "ALICE"
print(name.lower())             # "alice"

text = "  hello  "
print(text.strip())             # "hello"

text = "Hello World"
print(text.replace("World", "Python"))  # "Hello Python"

email = "user@example.com"
print(email.startswith("user"))  # True
print("@" in email)              # True
```

F-Strings (Modern Formatting)

```
name = "Alex"
age = 25
city = "Hyderabad"

# Old way
print("My name is " + name)

# New way (better)
print(f"My name is {name}")
print(f"My name is {name}, I am {age} years old and I live in {city}")

# Expressions
a = 10
b = 3
print(f"Sum is {a + b}")        # "Sum is 13"
print(f"Division is {a / b}")   # "Division is 3.33..."
```

Boolean (bool)

Definition: Only two values - `True` or `False` (must be capitalized)

```
is_active = True
is_logged_in = False

# From comparisons
age = 20
is_adult = age >= 18  # True
```

Naming Convention: Use question format

```
is_active = True      # "Is it active?"
has_data = False      # "Has data?"
```

```
can_proceed = True    # "Can proceed?"
```

---

4. Basic Math Operations

Arithmetic Operators

Basic Operations

Operator	Operation	Example	Result
+	Addition	`10 + 3`	`13`
-	Subtraction	`10 - 3`	`7`
*	Multiplication	`10 * 3`	`30`
/	Division (float)	`10 / 3`	`3.333...`
//	Floor Division	`10 // 3`	`3`
%	Modulus (remainder)	`10 % 3`	`1`
**	Power (exponent)	`2 ** 3`	`8`

Examples

```
a = 10
b = 3

print(a + b)   # 13
print(a - b)   # 7
print(a * b)   # 30
print(a / b)   # 3.3333333333333335
print(a // b)  # 3
print(a % b)   # 1
print(a ** b)  # 1000
```

Built-in Math Functions

Absolute Value

```
print(abs(-5))   # 5
print(abs(5))    # 5
```

Rounding

```
print(round(3.7))      # 4
```

```
print(round(3.14159, 2))  # 3.14
```

Min and Max

```
print(min(5, 10, 3))   # 3
print(max(5, 10, 3))   # 10
```

Sum

```
numbers = [1, 2, 3, 4, 5]
print(sum(numbers))  # 15
```

Math Library

Import math library:

```
import math
```

Square Root

```
import math
print(math.sqrt(16))  # 4.0
print(math.sqrt(2))   # 1.4142135623730951
```

Ceiling and Floor

```
import math

print(math.ceil(3.2))   # 4 (rounds up)
print(math.floor(3.8))  # 3 (rounds down)
```

Constants

```
import math

print(math.pi)  # 3.141592653589793
print(math.e)   # 2.718281828459045
```

Random Library

Import random library:

```
import random
```

Random Integer

```
import random

# Random number between 1 and 10 (inclusive)
number = random.randint(1, 10)
print(number)  # Could be 1, 2, 3, ..., 10
```

---

5. Print Function

Basic Usage

The `print()` function sends output to the screen (stdout).

Basic Syntax

```
print(value)
```

Examples

```
# Print a value
age = 25
print(age)  # 25

# Print text
print("Hello, World!")  # Hello, World!

# Print multiple values
name = "Bob"
age = 30
print(name, age)  # Bob 30
```

Print Parameters

sep (Separator)

Default separator is a space. You can change it:

```
print("apple", "banana", "cherry")  # apple banana cherry
print("apple", "banana", "cherry", sep=", ")  # apple, banana, cherry
print("apple", "banana", "cherry", sep="-")   # apple-banana-cherry
```

end (Ending Character)

Default ending is a newline. You can change it:

```
print("Hello")
print("World")
# Output:
# Hello
# World

print("Hello", end=" ")
print("World")
# Output: Hello World
```

Modern String Formatting

Old Way (Concatenation)

```
name = "Alex"
print("My name is " + name)  # Works but clunky
```

New Way (f-strings)

```
name = "Alex"
print(f"My name is {name}")  # Clean and modern
```

Complex Example

```
name = "Alex"
age = 25
city = "Hyderabad"

# Old way
print("My name is " + name + ", I am " + str(age) + " years old and I live in " +
city)

# New way (f-string)
print(f"My name is {name}, I am {age} years old and I live in {city}")
```

---

6. Input Function

Basic Usage

The `input()` function takes data from the user via keyboard.

Basic Syntax

```
variable = input("Prompt message: ")
```

Example

```
name = input("Enter your name: ")
print(f"Hello, {name}!")

# When you run:
# Enter your name: Alice
# Hello, Alice!
```

Important: Input Always Returns String

```
age = input("Enter age: ")
print(type(age))  # <class 'str'>

# Even if user types "25", it's stored as string "25"
```

Converting Input to Numbers

Two-Step Conversion

```
age_str = input("Enter your age: ")
age = int(age_str)
next_year = age + 1
print(f"Next year you'll be {next_year}")
```

One-Step Conversion (Inline)

```
age = int(input("Enter your age: "))
next_year = age + 1
print(f"Next year you'll be {next_year}")
```

Float Input

```
price = float(input("Enter price: "))
tax = price * 0.08
total = price + tax
print(f"Total with tax: {total}")
```

Multiple Inputs

```
first = input("First name: ")
last = input("Last name: ")
age = int(input("Age: "))
```

```
print(f"{first} {last}, age {age}")
```

Error Handling

What happens if user enters invalid data:

```
age = int(input("Enter your age: "))
# If user types "Alex", you get:
# ValueError: invalid literal for int() with base 10: 'Alex'
```

Always validate input in real programs!

---

7. Type Checking & Conversion

Checking Types

The type() Function

```
age = 25
print(type(age))  # <class 'int'>

name = "Alice"
print(type(name))  # <class 'str'>

price = 19.99
print(type(price))  # <class 'float'>

is_active = True
print(type(is_active))  # <class 'bool'>
```

Type Conversion

Converting to Integer - int()

```
# String to int
age = int("25")
print(age)  # 25

# Float to int (truncates decimal)
score = int(25.9)
print(score)  # 25

# Boolean to int
print(int(True))   # 1
print(int(False))  # 0
```

What causes errors:

```
# Can't convert decimal string
age = int("25.5")  # ERROR: ValueError

# Can't convert text
name = int("Alex")  # ERROR: ValueError
```

Converting to Float - float()

```
# String to float
price = float("3.14")
print(price)  # 3.14

# Int to float
count = float(5)
print(count)  # 5.0

# String (integer) to float
value = float("5")
print(value)  # 5.0

# Boolean to float
print(float(True))   # 1.0
print(float(False))  # 0.0
```

Converting to String - str()

```
# Int to string
age = str(25)
print(age)  # "25"

# Float to string
price = str(3.14)
print(price)  # "3.14"

# Boolean to string
active = str(True)
print(active)  # "True"
```

Converting to Boolean - bool()

```
# Non-zero numbers are True
print(bool(1))      # True
print(bool(100))    # True
print(bool(-5))     # True
print(bool(0))      # False
```

```
# Non-empty strings are True
print(bool("text"))  # True
print(bool(""))      # False

# Non-empty lists are True
print(bool([1, 2]))  # True
print(bool([]))      # False
```

Common Use Cases

User Input (Always Needs Conversion)

```
# Bad - trying to do math with string
age_str = input("Enter age: ")  # Returns string "25"
next_year = age_str + 1  # ERROR!

# Good - convert first
age = int(input("Enter age: "))
next_year = age + 1  # Works!
```

String Concatenation

```
# Bad - can't concatenate string and number
score = 95
message = "Score: " + score  # ERROR!

# Good - convert number to string
score = 95
message = "Score: " + str(score)  # Works!
print(message)  # "Score: 95"

# Better - use f-string
score = 95
message = f"Score: {score}"  # Even better!
print(message)  # "Score: 95"
```

Calculation Display

```
result = 10 / 3
print(f"Result: {result}")  # f-string handles conversion automatically
```

Quick Reference Summary

Variables

```
age = 25                    # Create variable
age = 30                    # Reassign
x, y, z = 1, 2, 3           # Multiple assignment
print(id(age))              # Memory address
```

Naming

```
student_name = "Alice"      # snake_case (Good)
MAX_SIZE = 100              # UPPER_CASE for constants
# 2name = 10                # Invalid: starts with number
# student-name = "Bob"      # Invalid: hyphen not allowed
```

Data Types

```
x = 10                      # int
x = 10.5                    # float
x = "text"                  # str
x = True                    # bool
```

Math Operations

```
10 + 3    # 13   Addition
10 - 3    # 7    Subtraction
10 * 3    # 30   Multiplication
10 / 3    # 3.33 Division (float)
10 // 3   # 3    Floor division
10 % 3    # 1    Modulus
10 ** 3   # 1000 Power
```

Built-in Functions

```
abs(-5)             # 5
round(3.7)          # 4
min(1, 2, 3)        # 1
max(1, 2, 3)        # 3
sum([1, 2, 3])      # 6
len("hello")        # 5
```

Math Library

```
import math
math.sqrt(16)       # 4.0
math.ceil(3.2)      # 4
```

```
math.floor(3.8)      # 3
math.pi              # 3.14159...
```

Input/Output

```
print("Hello")                    # Output
name = input("Name: ")            # Input (string)
age = int(input("Age: "))         # Input (convert to int)
print(f"Hi {name}, age {age}")    # F-string
```

Type Checking & Conversion

```
type(25)             # <class 'int'>
int("25")            # 25
float("3.14")        # 3.14
str(100)             # "100"
bool(1)              # True
```

Common Errors & Solutions

NameError

```
# Error
print(name)  # NameError: name 'name' is not defined

# Solution
name = "Alice"
print(name)  # Works
```

TypeError

```
# Error
age = "25"
next_year = age + 1  # TypeError: can't concatenate str and int

# Solution
age = int("25")
next_year = age + 1  # Works
```

ValueError

```
# Error
age = int("Alex")  # ValueError: invalid literal for int()

# Solution
age = int("25")  # Use valid number string
```

SyntaxError

```
# Error
2age = 25  # SyntaxError: invalid syntax

# Solution
age2 = 25  # Variable names can't start with numbers
```

End of Lecture Notes

Session 6: Python: Decision Making

What You Will Learn

- **Comparison Operators** - Compare values and make decisions (`==` , `!=` , `>` , `<` , `>=` , `<=`)
- **Logical Operators** - Combine multiple conditions (`and` , `or` , `not`)
- **If Statements** - Execute code based on conditions
- **Elif and Else** - Handle multiple scenarios and defaults
- **Nested Conditionals** - Create complex decision-making logic

---

Comparison Operators

Comparison operators compare two values and return a **boolean** result: `True` or `False` .

The Six Comparison Operators

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 3`	`True`
`>`	Greater than	`5 > 3`	`True`
`<`	Less than	`3 < 5`	`True`
`>=`	Greater than or equal	`5 >= 5`	`True`
`<=`	Less than or equal	`3 <= 5`	`True`

Examples

Equal To (`==`)

```
age = 18
print(age == 18)  # True
print(age == 20)  # False

name = "Alice"
print(name == "Alice")  # True
print(name == "Bob")    # False
```

Common Mistake: Using `=` instead of `==`

```
# Wrong - This assigns 18 to age
if age = 18:  # SyntaxError!

# Correct - This checks if age equals 18
if age == 18:  # Correct!
```

Not Equal To (`!=`)

```
status = "pending"
print(status != "completed")  # True
print(status != "pending")    # False
```

Greater Than (`>`)

```
score = 85
print(score > 75)   # True
print(score > 90)   # False
print(score > 85)   # False (not greater, equal!)
```

Less Than (`<`)

```
temperature = 25
print(temperature < 30)  # True
print(temperature < 20)  # False
print(temperature < 25)  # False (not less, equal!)
```

Greater Than or Equal To (`>=`)

```
age = 18
print(age >= 18)  # True (equal counts!)
print(age >= 16)  # True
print(age >= 21)  # False
```

Less Than or Equal To (`<=`)

```
attempts = 3
print(attempts <= 3)  # True (equal counts!)
print(attempts <= 5)  # True
print(attempts <= 2)  # False
```

Storing Comparison Results

```
# Comparisons return boolean values
age = 20
is_adult = age >= 18
print(is_adult)        # True
print(type(is_adult))  # <class 'bool'>

# Can use in conditions
temperature = 35
is_hot = temperature > 30
print(f"Is it hot? {is_hot}")  # Is it hot? True
```

Common Mistakes

1. Using `=` Instead of `==`

```
# Wrong
x = 5
if x = 5:  # SyntaxError
    print("Five")

# Correct
if x == 5:
    print("Five")
```

2. Forgetting Type Matters

```
user_input = input("Enter age: ")  # Returns string!
# user_input = "18" (string)

# Wrong comparison
if user_input >= 18:  # TypeError!

# Correct - convert first
age = int(user_input)
if age >= 18:
    print("Adult")
```

3. Confusing `>` and `>=`

```
score = 60
# Be clear about boundaries
print(score > 60)   # False
print(score >= 60)  # True
```

Practice Exercise

Try to predict the output:

```
a = 10
b = 20
c = 10

print(a == c)    # ?
print(a != b)    # ?
print(b > a)     # ?
print(a >= c)    # ?
print(b <= 15)   # ?
```

▶ Answer

Logical Operators

Logical operators combine or modify boolean values. They're essential for creating complex conditions.

The Three Logical Operators

Operator	Description	Returns True When
`and`	Both conditions must be True	Both are True
`or`	At least one condition must be True	One or both are True
`not`	Reverses the boolean value	The value is False

The `and` Operator

Returns `True` only if **both** conditions are True.

Truth Table

A	B	A and B
True	True	**True**
True	False	False
False	True	False

False	False	False

Examples

```
# Simple example
age = 22
has_id = True
can_drive = age >= 18 and has_id
print(can_drive)  # True

# Multiple conditions
temperature = 25
is_sunny = True
is_weekend = True
perfect_day = temperature > 20 and is_sunny and is_weekend
print(perfect_day)  # True

# When one is False
score = 85
passed_exam = score >= 60
submitted_assignment = False
can_pass_course = passed_exam and submitted_assignment
print(can_pass_course)  # False (one is False!)
```

Real-World: Loan Qualification

```
age = 25
income = 50000
has_good_credit = True

qualifies = age >= 21 and income >= 30000 and has_good_credit
print(f"Qualifies for loan: {qualifies}")  # True
```

The `or` Operator

Returns `True` if **at least one** condition is True.

Truth Table

A	B	A or B
True	True	**True**
True	False	**True**
False	True	**True**
False	False	False

Examples

```
# Simple example
is_weekend = True
is_holiday = False
can_sleep_in = is_weekend or is_holiday
print(can_sleep_in)  # True (one is True!)

# Both are True
has_cash = True
has_card = True
can_pay = has_cash or has_card
print(can_pay)  # True

# Both are False
is_raining = False
is_snowing = False
is_bad_weather = is_raining or is_snowing
print(is_bad_weather)  # False (both are False)
```

Real-World: Access Control

```
is_admin = False
is_owner = True
has_access = is_admin or is_owner
print(f"Has access: {has_access}")  # True
```

The `not` Operator

Reverses the boolean value.

Truth Table

A	not A
True	False
False	True

Examples

```
# Simple reversal
is_raining = False
is_sunny = not is_raining
print(is_sunny)  # True

# With comparisons
age = 16
is_adult = not (age < 18)  # Same as age >= 18
```

```
print(is_adult)  # False

# Double negative
is_logged_in = True
is_not_logged_in = not is_logged_in
is_logged_in_again = not is_not_logged_in
print(is_logged_in_again)  # True (double negative!)
```

Real-World: Form Validation

```
form_submitted = False
can_edit = not form_submitted
print(f"Can edit: {can_edit}")  # True
```

Combining Logical Operators

You can combine multiple logical operators to create complex conditions.

Example 1: Age Range Check

```
age = 25
is_adult = age >= 18 and age < 65
print(f"Working age adult: {is_adult}")  # True
```

Example 2: Access Control

```
is_admin = False
is_moderator = True
is_owner = False

has_special_access = is_admin or is_moderator or is_owner
print(f"Has special access: {has_special_access}")  # True
```

Example 3: Complex Validation

```
age = 17
has_parent_consent = True
has_id = True

# Can register if (18+) OR (under 18 but has parent consent)
can_register = (age >= 18) or (age < 18 and has_parent_consent and has_id)
print(f"Can register: {can_register}")  # True
```

Common Mistakes

1. Using `&` or `|` Instead of `and` / `or`

```
# Wrong (these are bitwise operators)
age = 50
if age != 50 & age != 30:  # Doesn't work as expected!
    print("I AM INSIDE THIS")

# Correct
if age != 50 and age != 30:
    print("I AM INSIDE THIS")
```

2. Forgetting Parentheses

```
# Confusing
age = 20
has_license = True
can_drive = age >= 18 and has_license or not has_license

# Clear
can_drive = (age >= 18 and has_license) or (not has_license)
```

3. Redundant Comparisons

```
# Redundant
is_adult = (age >= 18) == True

# Better
is_adult = age >= 18
```

Practice Exercise

Predict the output:

```
a = True
b = False
c = True

print(a and b)          # ?
print(a or b)           # ?
print(not b)            # ?
print(a and b or c)     # ?
print(a and (b or c))   # ?
print(not (a and b))    # ?
```

▶ Answer

If Statements

An `if` statement allows your program to execute code **only when a condition is True**.

Basic Syntax

```
if condition:
    # Code block (indented)
    # Runs only if condition is True
```

Key Components:

1. The `if` keyword
2. A condition (boolean expression)
3. A colon `:`
4. Indented code block

Simple Examples

Example 1: Basic If Statement

```
age = 20

if age >= 18:
    print("You are an adult")

# Output: You are an adult
```

Example 2: Condition is False

```
age = 15

if age >= 18:
    print("You are an adult")

# No output (condition is False, code doesn't run)
```

Example 3: Multiple Statements

```
score = 95

if score >= 90:
    print("Excellent work!")
    print("You got an A")
    print("Keep it up!")

# All three lines print because condition is True
```

Indentation Rules

Python uses **indentation** (spaces or tabs) to group code blocks. This is **required**, not optional!

Correct Indentation

```
if age >= 18:
    print("You can vote")      # Indented - part of if block
    print("You are an adult")  # Indented - part of if block
print("Program continues")     # Not indented - always runs
```

Indentation Error

```
if age >= 18:
print("You can vote")  # IndentationError: expected an indented block
```

Inconsistent Indentation

```
if age >= 18:
    print("Line 1")  # 4 spaces
      print("Line 2")  # 6 spaces - IndentationError!
```

Best Practice: Use **4 spaces** for each indentation level (standard Python convention)

Using Conditions

With Comparison Operators

```
temperature = 30

if temperature > 25:
    print("It's hot outside")

score = 85
if score >= 60:
    print("You passed")

password = "secret"
if password == "secret":
    print("Access granted")
```

With Logical Operators

```
age = 20
has_license = True

if age >= 18 and has_license:
    print("You can drive")

is_weekend = True
```

```
is_holiday = False

if is_weekend or is_holiday:
    print("You can sleep in")

form_submitted = False
if not form_submitted:
    print("You can still edit")
```

With Variables

```
is_logged_in = True

if is_logged_in:
    print("Welcome back!")

is_admin = False
if is_admin:
    print("Admin panel")  # Doesn't print (False)
```

Real-World Examples

Example 1: Age Verification

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You can create an account")
    print("Welcome!")

if age >= 21:
    print("You can access premium features")
```

Example 2: Login System

```
username = input("Enter username: ")
password = input("Enter password: ")

correct_username = "admin"
correct_password = "secret123"

if username == correct_username and password == correct_password:
    print("Login successful!")
    print("Welcome, admin!")

if username != correct_username:
    print("Invalid username")
```

```
if password != correct_password:
    print("Invalid password")
```

Common Mistakes

1. Forgetting the Colon

```
# Wrong
if age >= 18
    print("Adult")  # SyntaxError: invalid syntax

# Correct
if age >= 18:
    print("Adult")
```

2. No Indentation

```
# Wrong
if age >= 18:
print("Adult")  # IndentationError

# Correct
if age >= 18:
    print("Adult")
```

3. Using `=` Instead of `==`

```
# Wrong - This assigns 18 to age!
if age = 18:  # SyntaxError
    print("18 years old")

# Correct - This compares
if age == 18:
    print("18 years old")
```

4. Forgetting Indentation for Multiple Lines

```
# Wrong - Only first line is in if block
if age >= 18:
    print("You are an adult")
print("You can vote")  # Always runs!

# Correct - Both lines in if block
if age >= 18:
    print("You are an adult")
    print("You can vote")  # Both indented
```

5. Empty If Block

```
# Wrong - If block can't be empty
if age >= 18:

# Correct - Use pass as placeholder
if age >= 18:
    pass  # Do nothing (placeholder)
```

Practice Exercise

Predict what will be printed:

```
x = 10

if x > 5:
    print("A")
    print("B")

if x > 15:
    print("C")

print("D")
```

▶ Answer

Elif Statements

The `elif` (else if) statement allows you to check multiple conditions where **only one block of code will execute**.

Basic Syntax

```
if condition1:
    # Runs if condition1 is True
elif condition2:
    # Runs if condition1 is False and condition2 is True
elif condition3:
    # Runs if condition1 and condition2 are False and condition3 is True
```

Key Characteristics:

1. `elif` must come after an `if` statement
2. You can have multiple `elif` statements
3. Only the **first True condition** executes
4. Once a match is found, remaining conditions are skipped

Why Use Elif?

Problem with Multiple If Statements

```
score = 85

if score >= 90:
    grade = "A"

if score >= 80:
    grade = "B"  # This runs!

if score >= 70:
    grade = "C"  # This also runs!

print(grade)  # C (last assignment wins!)
```

Issues:

- All conditions are checked (inefficient)
- Multiple blocks might execute
- Results can be unexpected

Solution with Elif

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"  # This runs and STOPS
elif score >= 70:
    grade = "C"  # Never checked

print(grade)  # B (correct!)
```

Benefits:

- Only one block executes
- More efficient (stops after first match)
- Clearer intent

Examples

Example 1: Grade Calculator

```
score = 75

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
```

```
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")

# Output: Grade: C
```

Example 2: Age Categories

```
age = 15

if age >= 65:
    print("Senior citizen")
elif age >= 18:
    print("Adult")
elif age >= 13:
    print("Teenager")  # This runs
elif age >= 3:
    print("Child")

# Output: Teenager
```

Order Matters!

The order of conditions is crucial because Python checks from top to bottom:

Incorrect Order (Session Example)

```
marks = 90

if marks >= 50:
    print("Grade C")  # This runs for 90!
elif marks >= 75:
    print("Grade B")  # Never checked
elif marks >= 90:
    print("Grade A")  # Never checked
else:
    print("Fail")

# Output: Grade C (Wrong! Should be Grade A)
```

Problem: The first condition (`marks >= 50`) is True for 90, so it executes and stops. The more specific conditions never get checked.

Correct Order

```
marks = 90
```

```python
if marks >= 90:
    print("Grade A")  # Checks this first!
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")

# Output: Grade A (Correct!)
```

Rule: Put more **specific** conditions before more **general** ones.

Real-World Applications

Weather Response

```python
weather = input("Enter weather: ")

if weather == "rain":
    print("Take an umbrella")
elif weather == "sunny":
    print("Wear sunglasses")
elif weather == "cold":
    print("Wear a jacket")
else:
    print("Check the weather forecast")
```

Day of Week

```python
day = "Sunday"

if day == "Monday":
    print("Start of the work week")
elif day == "Saturday":
    print("Weekend begins")
elif day == "Sunday":
    print("Relax, it's a holiday!")
else:
    print("Regular day")

# Output: Relax, it's a holiday!
```

Else Statements

The `else` statement provides a **default action** when all previous conditions in an if-elif chain are False.

Basic Syntax

```
if condition:
    # Runs if condition is True
else:
    # Runs if condition is False
```

With elif:

```
if condition1:
    # Code
elif condition2:
    # Code
else:
    # Runs if ALL above conditions are False
```

Key Points:

1. No condition needed (no `else score > 50:`)
2. Always comes last
3. Optional - not required
4. Catches all remaining cases

Simple Examples

Example 1: Binary Choice

```
age = 16

if age >= 18:
    print("Can vote")
else:
    print("Cannot vote yet")

# Output: Cannot vote yet
```

Example 2: Complete Grade System

```
score = 75

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
```

```
print(f"Grade: {grade}")  # Grade: C
```

Example 3: Number Classification

```
number = -5

if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")

# Output: Negative
```

When to Use Else

Use Else When:

- You want a default action for unmatched cases
- You need to handle "everything else"
- Two possible paths (if-else)

Skip Else When:

- Not all cases need handling
- You only care about specific conditions

```
# With else
age = 15
if age >= 18:
    print("Adult")
else:
    print("Minor")

# Without else (also valid)
age = 15
if age >= 18:
    print("Adult")
# Nothing prints if age < 18
```

Common Mistakes

1. Condition in Else

```
# Wrong
if score >= 60:
    print("Pass")
```

```
else score < 60:  # SyntaxError
    print("Fail")

# Correct
if score >= 60:
    print("Pass")
else:
    print("Fail")
```

2. Else Before Elif

```
# Wrong
if score >= 90:
    print("A")
else:
    print("Not A")
elif score >= 80:  # SyntaxError
    print("B")

# Correct
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("Below B")
```

Nested Conditionals

A nested conditional is an if statement inside another if statement.

Basic Syntax

```
if condition1:
    if condition2:
        # Runs if both condition1 AND condition2 are True
    else:
        # Runs if condition1 is True but condition2 is False
else:
    # Runs if condition1 is False
```

Why Use Nested Conditionals?

When you need to check a second condition **only if** the first one is True.

Example 1: Age and License Check

```
age = 20
has_license = True

if age >= 18:
    if has_license:
        print("You can drive")
    else:
        print("You need a license")
else:
    print("Too young to drive")
```

Example 2: Login System

```
username = "admin"
password = "secret"

if username == "admin":
    if password == "secret":
        print("Login successful")
    else:
        print("Wrong password")
else:
    print("User not found")
```

Multiple Levels of Nesting

```
score = 85
submitted = True
on_time = True

if score >= 60:
    if submitted:
        if on_time:
            print("Full credit")
        else:
            print("Late penalty")
    else:
        print("Missing assignment")
else:
    print("Failed")
```

Complex Nested Example (Session Example)

```
age = int(input("Enter your age: "))
has_id = input("Do you have an ID? (yes/no): ")
is_student = input("Are you a student? (yes/no): ")
```

```
if age >= 18:
    print("You are an adult")

    if has_id == "yes":
        print("ID verified")

        if is_student == "yes":
            print("Student discount applied")
        else:
            print("No student discount")

    else:
        print("ID required for entry")

elif age >= 13:
    print("You are a teenager")

    if has_id == "yes" or is_student == "yes":
        print("Limited access granted")
    else:
        print("Access denied")

else:
    print("You are a child")

    if not has_id:
        print("Entry not allowed without guardian")

print("Program completed")
```

Nested vs Logical Operators

These are equivalent:

```
# Using nested if
if age >= 18:
    if has_license:
        print("Can drive")

# Using and operator
if age >= 18 and has_license:
    print("Can drive")
```

When to use nested: When you need different actions at each level

When to use and: When you just need to check multiple conditions

Common Mistakes

1. Too Much Nesting

```python
# Avoid - too deep
if a:
    if b:
        if c:
            if d:
                if e:  # Hard to read!
                    print("Something")

# Better - use logical operators
if a and b and c and d and e:
    print("Something")
```

2. Forgetting Indentation

```python
# Wrong
if age >= 18:
if has_license:  # IndentationError
    print("Can drive")

# Correct
if age >= 18:
    if has_license:
        print("Can drive")
```

Truthy and Falsy Values

Python evaluates certain values as `False` in boolean contexts, even if they're not the literal boolean `False`.

Falsy Values

Values that Python treats as `False`:

- `False` (boolean)
- `0`, `0.0` (zero in any numeric type)
- `""`, `''` (empty strings)
- `None`
- `[]`, `{}`, `()` (empty collections)

Truthy Values

Everything else is truthy, including:

- `True`
- Non-zero numbers: `1`, `-1`, `3.14`
- Non-empty strings: `"hello"`, `" "` (even just a space!)
- Non-empty collections: `[1]`, `{'a': 1}`

Examples

Empty String is Falsy

```
user_input = ""
if user_input:
    print("Got input")  # Won't print
else:
    print("No input")  # Prints

# Output: No input
```

Non-Empty String (Even Space) is Truthy

```
user_input = " "  # Just a space!
if user_input:
    print("Got input")  # Prints!
else:
    print("No input")

# Output: Got input
```

Zero is Falsy

```
count = 0
if count:
    print("Has items")  # Won't print
else:
    print("Empty")  # Prints

# Output: Empty
```

Any Other Number is Truthy

```
count = -5
if count:
    print("Has items")  # Prints!

# Output: Has items
```

Session Example

```
has_id = ""  # Empty string

if has_id:
    print("I am inside if")  # Won't print
# No output - empty string is falsy
```

Common Pattern: Checking Non-Empty

```
# Cleaner way to check if string is not empty
username = input("Enter username: ")

if username:  # Instead of: if username != ""
    print(f"Welcome, {username}")
else:
    print("Username cannot be empty")
```

Summary: Common Mistakes

1. Using `=` Instead of `==`

```
# Wrong
if x = 5:  # SyntaxError

# Correct
if x == 5:
```

2. No Indentation

```
# Wrong
if age >= 18:
print("Adult")  # IndentationError

# Correct
if age >= 18:
    print("Adult")
```

3. Inconsistent Indentation

```
# Wrong
if age >= 18:
    print("Adult")
      print("Can vote")  # IndentationError

# Correct
if age >= 18:
    print("Adult")
    print("Can vote")
```

4. Using `&` Instead of `and`

```python
# Wrong - bitwise operator
if age >= 18 & has_id:

# Correct - logical operator
if age >= 18 and has_id:
```

5. Wrong Order in Elif

```python
# Wrong - general condition first
if marks >= 50:
    print("Pass")
elif marks >= 90:  # Never reached!
    print("Excellent")

# Correct - specific condition first
if marks >= 90:
    print("Excellent")
elif marks >= 50:
    print("Pass")
```

6. Condition in Else

```python
# Wrong
else score < 60:  # SyntaxError

# Correct
else:
```

What's Next?

Now that you've mastered decision making in Python, you're ready to explore how to repeat actions and process collections of data efficiently:

- **For Loops** - Execute code a specific number of times or iterate through sequences
- **While Loops** - Repeat code as long as a condition remains True
- **Iterating Over Lists** - Process each item in a collection automatically
- **Loop Control** - Use `break` to exit loops early and `continue` to skip iterations
- **Nested Loops** - Combine loops to work with multi-dimensional data and create complex patterns

These concepts will enable you to write more powerful and efficient programs that can handle repetitive tasks and large datasets!

Session 7: Python: Automation & Loops

1. **While Loops** - How to repeat code based on conditions, including input validation, menu systems, and avoiding infinite loops

2. **Loop Control Statements** - Using break to exit loops early and continue to skip iterations, with practical applications in search, validation, and data filtering
3. **For Loops and Range** - Iterating over sequences (strings, lists) and generating number sequences with range() for counting, processing collections, and creating patterns
4. **Nested Loops** - Combining loops within loops to create patterns, process multi-dimensional data, and generate combinations

---

While Loops

A while loop repeats code **as long as** a condition is True.

Basic Syntax

```
while condition:
    # Code to repeat
    # Must eventually make condition False!
```

Simple Examples

Example 1: Count to 5

```
count = 1

while count <= 5:
    print(count)
    count += 1

# Output: 1 2 3 4 5
```

Example 2: User Input Validation

```
password = ""

while len(password) < 8:
    password = input("Enter password (min 8 chars): ")

print("Password accepted!")
```

Example 3: Menu System

```
choice = ""

while choice != "quit":
    print("\n1. New Game")
    print("2. Load Game")
    print("3. Quit")
    choice = input("Enter choice: ")
```

```
    if choice == "1":
        print("Starting new game...")
    elif choice == "2":
        print("Loading game...")
    elif choice == "quit":
        print("Goodbye!")
```

Infinite Loops

Warning: If condition never becomes False, the loop runs forever!

```
# Infinite loop - BAD!
count = 1
while count <= 5:
    print(count)
    # Forgot to increment count!

# Correct
count = 1
while count <= 5:
    print(count)
    count += 1  # Makes condition eventually False
```

While with Break

```
while True:  # Infinite loop
    password = input("Enter password: ")
    if password == "secret":
        break  # Exit loop
    print("Wrong password!")
```

Real-World Applications

ATM Withdrawal

```
balance = 1000

while balance > 0:
    print(f"Balance: ${balance}")
    amount = int(input("Withdraw amount (0 to quit): "))

    if amount == 0:
        break
    elif amount > balance:
        print("Insufficient funds")
    else:
        balance -= amount
```

```
        print(f"Withdrew ${amount}")

print("Thank you!")
```

Number Guessing Game

```
secret = 42
guess = 0

while guess != secret:
    guess = int(input("Guess the number: "))
    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")

print("Correct!")
```

The Break Statement

`break` immediately exits the current loop, regardless of the loop condition.

Basic Syntax

```
while condition:
    # Code
    if some_condition:
        break  # Exit loop immediately
    # More code
```

Examples

Example 1: Search and Stop

```
numbers = [3, 7, 2, 9, 5]
target = 9
index = 0

while index < len(numbers):
    if numbers[index] == target:
        print(f"Found {target} at index {index}")
        break  # Stop searching
    index += 1
```

Example 2: Exit on Command

```
while True:
    command = input("Enter command (or 'quit'): ")
    if command == "quit":
        break
    print(f"Executing: {command}")

print("Program ended")
```

Example 3: Limit Attempts

```
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    password = input("Enter password: ")
    if password == "secret":
        print("Access granted!")
        break
    attempts += 1
    print(f"Wrong! {max_attempts - attempts} attempts left")
else:
    print("Account locked")
```

Break vs Condition

```
# Using break
while True:
    x = int(input("Enter number (0 to stop): "))
    if x == 0:
        break
    print(f"You entered: {x}")

# Using condition
x = -1
while x != 0:
    x = int(input("Enter number (0 to stop): "))
    if x != 0:
        print(f"You entered: {x}")
```

The Continue Statement

`continue` skips the rest of the current iteration and goes to the next one.

Basic Syntax

```
while condition:
    # Code
```

```
    if some_condition:
        continue  # Skip to next iteration
    # This code is skipped if continue runs
```

Examples

Example 1: Skip Even Numbers

```
count = 0

while count < 10:
    count += 1
    if count % 2 == 0:
        continue  # Skip even numbers
    print(count)  # Only prints odd numbers

# Output: 1 3 5 7 9
```

Example 2: Input Validation

```
attempts = 0

while attempts < 5:
    age = int(input("Enter age: "))
    attempts += 1

    if age < 0:
        print("Invalid age!")
        continue  # Skip to next attempt

    print(f"Valid age: {age}")
    break
```

Example 3: Filter Data

```
count = 0
total = 0
num_count = 0

while count < 10:
    num = int(input("Enter number (negative to skip): "))
    count += 1

    if num < 0:
        continue  # Skip negative numbers

    total += num
    num_count += 1
```

```
average = total / num_count if num_count > 0 else 0
print(f"Average: {average}")
```

Continue vs Break

```
# continue - skips iteration
while count < 10:
    count += 1
    if count == 5:
        continue  # Skips printing 5
    print(count)

# break - exits loop
while count < 10:
    count += 1
    if count == 5:
        break  # Stops at 5
    print(count)
```

For Loops

A for loop iterates over a sequence (list, string, range, etc.).

Basic Syntax

```
for item in sequence:
    # Code using item
```

Examples

Example 1: Iterate String

```
name = "Alice"

for letter in name:
    print(letter)

# Output: A l i c e (each on new line)
```

Example 2: Iterate List

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

```
# Output:
# apple
# banana
# cherry
```

Example 3: Count Using Range

```
for i in range(5):
    print(i)

# Output: 0 1 2 3 4
```

Example 4: Sum Numbers

```
numbers = [10, 20, 30, 40]
total = 0

for num in numbers:
    total += num

print(f"Total: {total}")  # 100
```

For Loop vs While Loop

```
# For loop - when you know iterations
for i in range(5):
    print(i)

# While loop - when condition-based
count = 0
while count < 5:
    print(count)
    count += 1
```

Real-World Applications

Validate All Inputs

```
scores = [85, 92, 78, 95, 88]
all_passing = True

for score in scores:
    if score < 60:
        all_passing = False
        break
```

```
if all_passing:
    print("Everyone passed!")
```

Process Each Item

```
prices = [19.99, 29.99, 39.99]
tax_rate = 0.08

for price in prices:
    final_price = price * (1 + tax_rate)
    print(f"${price} -> ${final_price:.2f}")
```

The Range Function

`range()` generates a sequence of numbers, commonly used with for loops.

Three Forms

```
range(stop)              # 0 to stop-1
range(start, stop)       # start to stop-1
range(start, stop, step) # start to stop-1, by step
```

Examples

Form 1: range(stop)

```
for i in range(5):
    print(i)

# Output: 0 1 2 3 4 (stops before 5!)
```

Form 2: range(start, stop)

```
for i in range(2, 7):
    print(i)

# Output: 2 3 4 5 6 (stops before 7!)
```

Form 3: range(start, stop, step)

```
for i in range(0, 10, 2):
    print(i)
```

```
# Output: 0 2 4 6 8 (even numbers)
```

Counting Down

```
for i in range(5, 0, -1):
    print(i)

# Output: 5 4 3 2 1 (countdown)
```

Real-World Applications

Multiplication Table

```
num = 5

for i in range(1, 11):
    print(f"{num} x {i} = {num * i}")
```

Year Range

```
for year in range(2020, 2025):
    print(f"Year: {year}")

# Output: 2020 2021 2022 2023 2024
```

Skip Every Third

```
for i in range(0, 20, 3):
    print(i)

# Output: 0 3 6 9 12 15 18
```

Range with Lists

```
fruits = ["apple", "banana", "cherry"]

# Access by index
for i in range(len(fruits)):
    print(f"{i}: {fruits[i]}")

# Output:
# 0: apple
# 1: banana
# 2: cherry
```

Break and Continue in For Loops

Both `break` and `continue` work in for loops just like while loops.

Break in For Loop

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

for num in numbers:
    if num == 6:
        break  # Stop at 6
    print(num)

# Output: 1 2 3 4 5
```

Continue in For Loop

```
for num in range(1, 11):
    if num % 2 == 0:
        continue  # Skip even numbers
    print(num)

# Output: 1 3 5 7 9
```

Examples

Find First Match

```
names = ["Alice", "Bob", "Charlie", "David"]
target = "Charlie"

for i in range(len(names)):
    if names[i] == target:
        print(f"Found {target} at index {i}")
        break
```

Filter While Iterating

```
scores = [95, 45, 87, 32, 91, 68]

for score in scores:
    if score < 60:
        continue  # Skip failing scores
    print(f"Passing score: {score}")

# Output: Passing score: 95, 87, 91, 68
```

Early Exit on Error

```
data = [10, 20, 0, 40, 50]

for num in data:
    if num == 0:
        print("Error: Zero found!")
        break
    result = 100 / num
    print(f"100 / {num} = {result}")
```

Nested Loops with Break

```
# Break only exits innermost loop
for i in range(3):
    for j in range(3):
        if j == 2:
            break  # Exits inner loop only
        print(f"i={i}, j={j}")
```

Nested Loops

A nested loop is a loop inside another loop.

Basic Syntax

```
for outer_var in outer_sequence:
    for inner_var in inner_sequence:
        # Inner loop body
        # Runs completely for each outer iteration
```

Simple Example

```
for i in range(3):
    for j in range(2):
        print(f"i={i}, j={j}")

# Output:
# i=0, j=0
# i=0, j=1
# i=1, j=0
# i=1, j=1
# i=2, j=0
# i=2, j=1
```

Patterns

Rectangle of Stars

```
for row in range(3):
    for col in range(5):
        print("*", end="")
    print()  # New line after each row

# Output:
# *****
# *****
# *****
```

Multiplication Table

```
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i}x{j}={i*j}", end="\t")
    print()

# Output: 5x5 multiplication table
```

Right Triangle

```
for i in range(1, 6):
    for j in range(i):
        print("*", end="")
    print()

# Output:
# *
# **
# ***
# ****
# *****
```

Nested Loops with Lists

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for row in matrix:
    for num in row:
```

```
        print(num, end=" ")
    print()

# Output:
# 1 2 3
# 4 5 6
# 7 8 9
```

Real-World: All Combinations

```
sizes = ["S", "M", "L"]
colors = ["Red", "Blue", "Green"]

for size in sizes:
    for color in colors:
        print(f"{size}-{color}")

# Output: All 9 combinations
```

Key Takeaways

- While vs For - Use while for condition-based loops, for for iterating sequences or known counts
- Avoid Infinite Loops - Always update your loop variable or use break to exit
- Break vs Continue - break exits the loop, continue skips to next iteration
- Range Function - range(start, stop, step) generates numbers; stops BEFORE stop value
- Nested Loops - Inner loop completes fully for each outer iteration; great for patterns and combinations

---

Session 8: Python: Organizing with Functions

- **Keywords**: Function, Parameter, Argument, Return, Scope, Modular
- **Prerequisites**: Basic understanding of Python variables and data types.

Learning Objectives

By the end of this lecture, you will be able to:

1. **Define** and execute custom functions to encapsulate specific tasks.
2. **Implement** flexible function interfaces using positional, keyword, default, and variable arguments.
3. **Utilize** return statements to pass processed data back to the calling scope.
4. **Refactor** repetitive script logic into modular, reusable components.

Defining Functions

Why We Need Functions

Imagine you are writing a program to calculate the area of a rectangle. It's simple enough: `length * breadth`. But what if you need to do this for ten different rectangles? In a procedural script, you would

likely copy and paste the same multiplication logic ten times. If you later decide to add a validation step (like ensuring lengths aren't negative), you would have to hunt down and update every single copy.

Functions allow you to define a task once and reuse it multiple times. Compare the procedural approach with the functional approach:

Before: Procedural Code (Repetitive)

```
# Rectangle 1
l1, b1 = 5, 3
area1 = l1 * b1
print(area1)

# Rectangle 2
l2, b2 = 6, 4
area2 = l2 * b2
print(area2)

# Rectangle 3
l3, b3 = 7, 2
area3 = l3 * b3
print(area3)
```

After: Functional Approach (Modular)

```
def calculate_area(length, breadth):
    return length * breadth

# Reusing the logic reduces errors and improves readability
print(calculate_area(5, 3))
print(calculate_area(6, 4))
print(calculate_area(7, 2))
```

Syntax and Structure

In Python, you create a function using the `def` keyword followed by the function name and parentheses `()`. This tells the interpreter that the indented block of code following the header belongs to this specific function.

Here is the anatomy of a basic function:

```
def calculate_area(length, breadth):
    # Indentation is mandatory for the function body
    area = length * breadth
    return area
```

Defining a function is distinct from **calling** it. Writing the code above builds the tool, but it does not run it. To execute the code, you must "call" the function by its name followed by parentheses containing the required inputs.

```
# Calling the function
area = calculate_area(5, 3)
print(area)  # Output: 15
```

> 💡 ***Pro Tip**: You must define a function before you call it. While the execution call must occur after the definition, functions within a script can reference each other regardless of their relative order, as long as they are defined before the program actually runs that line of code.*

Synthesis Point

Functions act as named containers for logic; defining them builds the tool, while calling them puts that tool to work. Python also includes many built-in functions, such as `print()`, which follow these same calling conventions.

Parameters and Arguments

The Bridge Between Scopes

Functions would be limited if they could only perform the exact same action every time. To make them dynamic, we pass data into them.

- **Parameters**: These are placeholders defined in the function definition.
- **Arguments**: These are the actual values passed when calling the function.

```
# 'name' and 'age' are parameters
def greet(name, age):
    print(f"My name is {name} and I am {age} years old.")

# "Alice" and 25 are arguments
greet("Alice", 25)
```

Positional vs. Keyword Arguments

By default, Python matches arguments to parameters based on their order (position). In the example above, `"Alice"` is assigned to `name` because it is first.

However, you can also use **keyword arguments** to be explicit. By naming the parameters during the call, you gain order independence.

```
def order_pizza(size, topping1, topping2="cheese"):
    print(f"Ordering a {size} pizza with {topping1} and {topping2}")

# Order doesn't matter here because we use keywords
order_pizza(topping1="pepperoni", size="large")
```

Default Parameters

Functions can have default parameter values that are used if no argument is passed for those parameters. If the caller provides an argument, the default is overridden.

Constraint: Parameters with default values must always be defined *after* non-default parameters.

```python
def greet(name="Guest"):
    print(f"Hello, {name}")

greet()          # Uses default: Prints "Hello, Guest"
greet("Balaji")  # Overrides default: Prints "Hello, Balaji"
```

Variable Number of Arguments (`*args`)

There are cases where you won't know how many arguments a user might pass. Python handles this using `*args`. Inside the function, `args` is treated like a tuple containing all the passed values.

```python
def print_numbers(*numbers):
    # 'numbers' is a tuple of all arguments passed
    for number in numbers:
        print(number)

print_numbers(1, 2, 3, 4) # Works with any number of inputs
```

Keyword Variable Arguments (`**kwargs`)

While `*args` collects positional arguments into a tuple, `**kwargs` collects extra keyword arguments into a dictionary. This is useful for passing arbitrary named data, such as configuration settings or user profiles.

```python
def build_profile(first, last, **user_info):
    # user_info is a dictionary
    profile = {'first_name': first, 'last_name': last}
    for key, value in user_info.items():
        profile[key] = value
    return profile

user_profile = build_profile('Albert', 'Einstein',
                             location='Princeton',
                             field='Physics')

print(user_profile)
# Output: {'first_name': 'Albert', 'last_name': 'Einstein', 'location': 'Princeton',
'field': 'Physics'}
```

Synthesis Point

Parameters define the interface of your function. You can make this interface rigid (positional), flexible (keywords/defaults), or dynamic (`*args`) depending on how the function needs to receive data.

Return Values

Exiting the Function

The `return` statement sends a result back to the caller and immediately exits the function. While `print()` displays information, it doesn't allow the program to use that data for further calculations.

Without a `return` statement, functions return `None` by default.

```
def log_message(msg):
    print(f"Log: {msg}")
    # No return statement implies 'return None'

result = log_message("System ready")
print(result)  # Output: None
```

Returning Multiple Values

Python allows functions to return multiple values simultaneously. This is achieved by returning a tuple, which can then be unpacked into separate variables by the caller.

```
def calc(a, b):
    # Returns a tuple: (sum, difference, product)
    return a + b, a - b, a * b

# Unpacking the results into three variables
sum_, diff, product = calc(10, 5)

print(sum_)     # 15
print(diff)     # 5
print(product)  # 50
```

Synthesis Point

Use `print` when you want to perform an action like displaying text; use `return` when you want the function to provide an output for use in the next step of the program.

Code Modularity

Composing Functions

The true power of functions appears when you use them to modularize code and build larger programs. Instead of writing one massive block of code, you can create small, specialized functions that call each other.

Consider a simple calculator program. You can define separate functions for each operation and a display function to organize the output:

```
def add(a, b):
    return a + b

def divide(a, b):
    # Validation ensures robustness
    if b == 0:
        return "Error: Division by zero"
    return a / b
```

```python
def display_result(operation, result):
    print(f"The result of {operation} is: {result}")
```

Here is the flow of control when these functions work together:

```
sequenceDiagram
    participant Main as Main Script
    participant Add as add()
    participant Display as display_result()

    Main->>Add: Call add(10, 5)
    Add-->>Main: Return 15
    Main->>Display: Call display_result("Addition", 15)
    Display-->>Main: Returns None
```

Robustness and Validation

Modularity makes it easier to include validation, such as a "divide by zero" check. By encapsulating this logic within a function, you ensure the check happens consistently, making your program more robust and easier to organize.

Synthesis Point

Modularity transforms complex problems into manageable pieces. A well-structured program is a composition of small, reliable functions working together for better organization.

Key Takeaways

1. **Define Once, Reuse Multiple Times**: Functions encapsulate logic to avoid repetitive code, making programs easier to read and maintain.
2. **Definition vs. Calling**: Use the `def` keyword to define the function; it must be called by name with parentheses `()` to execute.
3. **Order and Keywords**: Arguments can be passed by position (order matters) or by naming the parameters (keyword arguments), which allows for order independence.
4. **Defaults for Flexibility**: Default parameters provide standard values but must be defined after non-default parameters.
5. **Dynamic Inputs**: Use `*args` to handle an arbitrary number of positional arguments as a tuple.
6. **Data Flow with Return**: The `return` statement sends results back to the caller and exits the function. Functions can return multiple values as tuples.
7. **Modular Composition**: Build complex systems by composing smaller functions, such as a calculator with separate logic for math and display tasks.
8. **Next Steps**: Understanding the order of arguments and return values prepares us for upcoming topics like local and global variables.

Constraints:

- Use default arguments for the unit.
- Ensure the conversion logic is separated from the printing logic.

Session 9: Python: Data Collections

Lists

A list is an ordered, mutable collection of items.

Creating Lists

```
# Empty list
my_list = []

# List with items
numbers = [1, 2, 3, 4, 5]
names = ["Alice", "Bob", "Charlie"]
mixed = [1, "hello", 3.14, True]
```

List Characteristics

1. **Ordered**: Items maintain their position
2. **Mutable**: Can be changed after creation
3. **Allow duplicates**: Same value can appear multiple times
4. **Mixed types**: Can contain different data types

Examples

Shopping List

```
shopping = ["milk", "eggs", "bread", "butter"]
print(shopping)  # ['milk', 'eggs', 'bread', 'butter']
```

Number List

```
scores = [85, 92, 78, 95, 88]
print(f"First score: {scores[0]}")  # 85
print(f"Number of scores: {len(scores)}")  # 5
```

Creating from Range

```
numbers = list(range(5))
print(numbers)  # [0, 1, 2, 3, 4]
```

Key Takeaways

1. **Square brackets**: [] to create lists
2. **Comma-separated**: Items separated by commas
3. **Any type**: Can store any Python objects

4. **Ordered**: Position matters
5. **Mutable**: Can be modified

Accessing list elements using indexing

Lecture Notes: Access List Elements with Indexing

List Indexing

Access individual list elements using their index (position).

Basic Indexing

```
fruits = ["apple", "banana", "cherry", "date"]

print(fruits[0])  # "apple" (first item)
print(fruits[1])  # "banana" (second item)
print(fruits[3])  # "date" (fourth item)
```

Remember: Python uses 0-based indexing!

Negative Indexing

```
fruits = ["apple", "banana", "cherry", "date"]

print(fruits[-1])  # "date" (last item)
print(fruits[-2])  # "cherry" (second to last)
print(fruits[-4])  # "apple" (first item)
```

Index Errors

```
fruits = ["apple", "banana", "cherry"]

print(fruits[10])  # IndexError: list index out of range
```

Modifying Elements

```
numbers = [10, 20, 30, 40]
numbers[0] = 15  # Change first item
print(numbers)  # [15, 20, 30, 40]
```

Real-World Examples

Access Student Data

```
students = ["Alice", "Bob", "Charlie", "David"]
print(f"First student: {students[0]}")
print(f"Last student: {students[-1]}")
```

Update Temperature

```
temps = [72, 75, 78, 80]
temps[0] = 70  # Correct first reading
print(temps)
```

Key Takeaways

1. **0-based**: First item is index 0
2. **Negative indexing**: -1 is last item
3. **Mutable**: Can change values
4. **Index errors**: Check bounds!
5. **Use len()**: Get list size

Extracting list portions using slicing

Lecture Notes: Extract List Portions with Slicing

List Slicing

Extract a portion (sub-list) from a list using slicing.

Basic Syntax

```
list[start:stop]  # From start to stop-1
list[start:stop:step]  # With step
```

Examples

Basic Slicing

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[2:5])  # [2, 3, 4] (indices 2, 3, 4)
print(numbers[0:3])  # [0, 1, 2] (first 3)
print(numbers[5:])   # [5, 6, 7, 8, 9] (from index 5 to end)
print(numbers[:4])   # [0, 1, 2, 3] (first 4)
```

With Step

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[::2])    # [0, 2, 4, 6, 8] (every 2nd)
print(numbers[1::2])   # [1, 3, 5, 7, 9] (odd indices)
print(numbers[::-1])   # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] (reversed)
```

Practical Examples

```
# Get first 3
fruits = ["apple", "banana", "cherry", "date", "elderberry"]
print(fruits[:3])  # ['apple', 'banana', 'cherry']

# Get last 2
print(fruits[-2:])  # ['date', 'elderberry']

# Get middle items
print(fruits[1:4])  # ['banana', 'cherry', 'date']
```

Slicing Creates New List

```
original = [1, 2, 3, 4, 5]
subset = original[1:4]
subset[0] = 100

print(original)  # [1, 2, 3, 4, 5] (unchanged!)
print(subset)    # [100, 3, 4]
```

Key Takeaways

1. **[start:stop]**: Extract from start to stop-1
2. **Omit start**: Defaults to 0
3. **Omit stop**: Goes to end
4. **Negative indices**: Count from end
5. **[::-1]**: Reverse a list

Modifying lists using built-in methods

Lecture Notes: Modify Lists with Methods

Modify Lists with Methods

This lesson covers list methods (append, remove, etc.).

Key Concepts

1. **Definition**: Understanding list methods (append, remove, etc.)
2. **Syntax**: How to use it correctly
3. **Examples**: Practical demonstrations
4. **Applications**: Real-world use cases

Basic Syntax

```
# Syntax examples for list methods (append, remove, etc.)
```

Examples

Example 1: Basic Usage

```
# Demonstrate list methods (append, remove, etc.)
```

Example 2: Practical Application

```
# Real-world example
```

Common Patterns

Standard ways to use list methods (append, remove, etc.) effectively.

Best Practices

1. Write clear code
2. Handle edge cases
3. Use appropriate methods
4. Follow Python conventions

Common Mistakes

1. **Mistake 1**: Common error
2. **Mistake 2**: Another pitfall

Key Takeaways

1. Main concept 1
2. Main concept 2
3. Main concept 3
4. Best practices
5. When to use

Iterating through lists using loops

Lecture Notes: Iterating Through Lists Using Loops

Iterating Through Lists Using Loops

Master the art of processing list data efficiently using Python's iteration tools.

Key Concepts

1. **for Loop**: Python's primary iteration tool
2. **enumerate()**: Access index and value simultaneously
3. **range()**: Iterate with indices
4. **zip()**: Combine multiple lists
5. **Loop Control**: break, continue statements
6. **Common Patterns**: Accumulation, filtering, transformation, search

Basic Syntax

```
# Basic for loop
for item in list_name:
    # Process each item

# With enumerate (index + value)
for index, item in enumerate(list_name):
    # Process with index

# With range (indices only)
for i in range(len(list_name)):
    # Access list_name[i]

# With zip (multiple lists)
for item1, item2 in zip(list1, list2):
    # Process pairs
```

Examples

Example 1: Basic Iteration

```
# Print all items
fruits = ["apple", "banana", "cherry", "date"]

print("Available fruits:")
for fruit in fruits:
    print(f"- {fruit}")

# Output:
# Available fruits:
# - apple
# - banana
```

```
# - cherry
# - date
```

Example 2: Calculate Statistics

```
# Calculate sum and average
test_scores = [85, 92, 78, 95, 88, 76, 90, 84]

total = 0
count = 0

for score in test_scores:
    total += score
    count += 1

average = total / count

print(f"Total students: {count}")
print(f"Total points: {total}")
print(f"Average score: {average:.2f}")
print(f"Highest score: {max(test_scores)}")
print(f"Lowest score: {min(test_scores)}")

# Output:
# Total students: 8
# Total points: 688
# Average score: 86.00
# Highest score: 95
# Lowest score: 76
```

Example 3: Using enumerate()

```
# Numbered list with positions
tasks = ["Buy groceries", "Finish homework", "Call dentist", "Go for run"]

print("Today's Tasks:")
for i, task in enumerate(tasks, start=1):
    print(f"{i}. {task}")

# Output:
# Today's Tasks:
# 1. Buy groceries
# 2. Finish homework
# 3. Call dentist
# 4. Go for run
```

```
# Find position of specific item
students = ["Alice", "Bob", "Charlie", "David", "Eve"]
search_name = "Charlie"
```

```
for i, student in enumerate(students):
    if student == search_name:
        print(f"{search_name} is at position {i} (or #{i+1} in the class)")
        break
# Output: Charlie is at position 2 (or #3 in the class)
```

Example 4: Using range() for Modification

```
# Modify list elements
prices = [10.00, 20.00, 15.00, 30.00, 25.00]

print("Original prices:", prices)

# Apply 10% discount
for i in range(len(prices)):
    prices[i] = prices[i] * 0.9  # 10% off

print("After 10% discount:", prices)

# Output:
# Original prices: [10.0, 20.0, 15.0, 30.0, 25.0]
# After 10% discount: [9.0, 18.0, 13.5, 27.0, 22.5]
```

Example 5: Using zip() with Multiple Lists

```
# Combine related data
students = ["Alice", "Bob", "Charlie", "David"]
math_scores = [85, 92, 78, 95]
english_scores = [88, 84, 90, 87]

print("Student Report Cards:")
print("-" * 40)

for student, math, english in zip(students, math_scores, english_scores):
    average = (math + english) / 2
    print(f"{student:10} | Math: {math:3} | English: {english:3} | Avg:
{average:.1f}")

# Output:
# Student Report Cards:
# ----------------------------------------
# Alice      | Math:  85 | English:  88 | Avg: 86.5
# Bob        | Math:  92 | English:  84 | Avg: 88.0
# Charlie    | Math:  78 | English:  90 | Avg: 84.0
# David      | Math:  95 | English:  87 | Avg: 91.0
```

Example 6: Filtering Data

```python
# Filter based on conditions
ages = [15, 22, 17, 30, 19, 14, 25, 18, 16, 21]

minors = []
adults = []

for age in ages:
    if age < 18:
        minors.append(age)
    else:
        adults.append(age)

print(f"Minors (< 18): {minors}")
print(f"Adults (>= 18): {adults}")
print(f"Total minors: {len(minors)}")
print(f"Total adults: {len(adults)}")

# Output:
# Minors (< 18): [15, 17, 19, 14, 16]
# Adults (>= 18): [22, 30, 25, 18, 21]
# Total minors: 5
# Total adults: 5
```

Example 7: Building New Lists (Transformation)

```python
# Transform data
temperatures_celsius = [0, 10, 20, 30, 40]
temperatures_fahrenheit = []

for temp_c in temperatures_celsius:
    temp_f = (temp_c * 9/5) + 32
    temperatures_fahrenheit.append(temp_f)

print("Celsius to Fahrenheit Conversion:")
for c, f in zip(temperatures_celsius, temperatures_fahrenheit):
    print(f"{c}°C = {f}°F")

# Output:
# Celsius to Fahrenheit Conversion:
# 0°C = 32.0°F
# 10°C = 50.0°F
# 20°C = 68.0°F
# 30°C = 86.0°F
# 40°C = 104.0°F
```

Example 8: Search with break

```python
# Find first match
products = ["Laptop", "Mouse", "Keyboard", "Monitor", "Headphones"]
```

```
prices = [999.99, 25.50, 79.99, 299.99, 89.99]

budget = 100.00

print(f"Products under ${budget}:")
found = False

for i, product in enumerate(products):
    if prices[i] < budget:
        print(f"- {product}: ${prices[i]}")
        found = True

if not found:
    print("No products found within budget")

# Output:
# Products under $100.0:
# - Mouse: $25.5
# - Keyboard: $79.99
# - Headphones: $89.99
```

Example 9: Skip with continue

```
# Process only valid data
sensor_readings = [23.5, -999, 24.1, 23.8, -999, 25.0, 24.3]
# -999 represents sensor error

valid_readings = []

for reading in sensor_readings:
    if reading == -999:
        continue  # Skip error readings
    valid_readings.append(reading)

if valid_readings:
    average = sum(valid_readings) / len(valid_readings)
    print(f"Valid readings: {valid_readings}")
    print(f"Average temperature: {average:.2f}°C")
else:
    print("No valid readings")

# Output:
# Valid readings: [23.5, 24.1, 23.8, 25.0, 24.3]
# Average temperature: 24.14°C
```

Example 10: Nested Lists (2D Data)

```
# Process 2D data (list of lists)
gradebook = [
```

```
    ["Alice", 85, 90, 88],
    ["Bob", 92, 88, 95],
    ["Charlie", 78, 85, 80]
]

print("Student Averages:")
print("-" * 30)

for student_data in gradebook:
    name = student_data[0]
    scores = student_data[1:]  # Get all scores except name

    average = sum(scores) / len(scores)
    print(f"{name:10} | Average: {average:.2f}")

# Output:
# Student Averages:
# ------------------------------
# Alice      | Average: 87.67
# Bob        | Average: 91.67
# Charlie    | Average: 81.00
```

Common Patterns

Pattern 1: Accumulation (Sum, Product, Count)

```
# Sum pattern
numbers = [10, 20, 30, 40, 50]
total = 0
for num in numbers:
    total += num
print(f"Sum: {total}")  # Sum: 150

# Product pattern
numbers = [2, 3, 4, 5]
product = 1
for num in numbers:
    product *= num
print(f"Product: {product}")  # Product: 120

# Count pattern
words = ["apple", "banana", "apricot", "cherry", "avocado"]
a_count = 0
for word in words:
    if word.startswith('a'):
        a_count += 1
print(f"Words starting with 'a': {a_count}")  # 3
```

Pattern 2: Find Maximum/Minimum

```python
# Find maximum with position
scores = [78, 92, 85, 95, 88]

max_score = scores[0]
max_index = 0

for i, score in enumerate(scores):
    if score > max_score:
        max_score = score
        max_index = i

print(f"Highest score: {max_score} at position {max_index}")
# Output: Highest score: 95 at position 3
```

Pattern 3: Filter and Collect

```python
# Collect items matching criteria
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = []
odds = []

for num in numbers:
    if num % 2 == 0:
        evens.append(num)
    else:
        odds.append(num)

print(f"Evens: {evens}")  # [2, 4, 6, 8, 10]
print(f"Odds: {odds}")    # [1, 3, 5, 7, 9]
```

Pattern 4: Transform and Map

```python
# Transform each element
names = ["alice", "bob", "charlie"]
capitalized = []

for name in names:
    capitalized.append(name.title())

print(capitalized)  # ['Alice', 'Bob', 'Charlie']
```

Pattern 5: Parallel Processing

```python
# Process multiple related lists
item_names = ["Apple", "Banana", "Cherry"]
quantities = [5, 3, 8]
prices = [1.20, 0.50, 2.00]
```

```
print("Shopping Receipt:")
print("-" * 35)

total_cost = 0

for name, qty, price in zip(item_names, quantities, prices):
    cost = qty * price
    total_cost += cost
    print(f"{name:10} x{qty} @ ${price:.2f} = ${cost:.2f}")

print("-" * 35)
print(f"{'Total:':10}              ${total_cost:.2f}")

# Output:
# Shopping Receipt:
# -----------------------------------
# Apple      x5 @ $1.20 = $6.00
# Banana     x3 @ $0.50 = $1.50
# Cherry     x8 @ $2.00 = $16.00
# -----------------------------------
# Total:                  $23.50
```

Best Practices

1. **Choose the right loop type**
   - Use `for item in list` when you only need values
   - Use `enumerate()` when you need both index and value
   - Use `range(len(list))` when you need to modify elements
   - Use `zip()` for parallel lists
2. **Use descriptive variable names**

```
# Good
for student in students:
    print(student)

# Avoid
for x in students:
    print(x)
```

3. **Avoid modifying list size during iteration**

```
# Bad - can cause issues
for item in items:
    if condition:
        items.remove(item)  # Dangerous!

# Good - create new list
filtered = []
```

```
for item in items:
    if condition:
        filtered.append(item)
```

4. **Use break for early exit**

```
# Stop when found
for item in items:
    if item == target:
        print("Found!")
        break
```

5. **Use continue to skip**

```
# Skip invalid data
for value in values:
    if value < 0:
        continue
    process(value)
```

Common Mistakes

1. **Mistake 1: Modifying list while iterating**

```
# Wrong
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    numbers.remove(num)  # Causes issues!

# Correct
numbers = [1, 2, 3, 4, 5]
numbers_copy = numbers.copy()
for num in numbers_copy:
    numbers.remove(num)
```

2. **Mistake 2: Forgetting to initialize accumulator**

```
# Wrong
for num in numbers:
    total += num  # Error: total not defined

# Correct
total = 0
for num in numbers:
    total += num
```

3. **Mistake 3: Using wrong index with enumerate**

```
# Wrong
for i, item in enumerate(items):
    print(items[i])  # Works but unnecessary

# Correct
for i, item in enumerate(items):
    print(item)  # Use item directly
```

4. **Mistake 4: Assuming lists have same length with zip**

```
# Be aware: zip stops at shortest list
list1 = [1, 2, 3, 4, 5]
list2 = [10, 20, 30]

for a, b in zip(list1, list2):
    print(a, b)
# Only prints 3 pairs, not 5!
```

Performance Tips

1. **Avoid repeated len() calls**

```
# Less efficient
for i in range(len(items)):
    if i < len(items) - 1:  # len() called every iteration
        ...

# Better
n = len(items)
for i in range(n):
    if i < n - 1:
        ...
```

2. **Use built-in functions when possible**

```
# Slower
total = 0
for num in numbers:
    total += num

# Faster
total = sum(numbers)
```

3. **List comprehensions (preview)**

```
# Traditional loop
squares = []
```

```
for num in numbers:
    squares.append(num ** 2)

# List comprehension (faster, more Pythonic)
squares = [num ** 2 for num in numbers]
```

Key Takeaways

1. **for loop** is Python's primary iteration tool for lists
2. **enumerate()** provides index and value together
3. **range()** useful for index-based access and modification
4. **zip()** combines multiple lists elegantly
5. **break** exits loop early, **continue** skips to next iteration
6. Common patterns: accumulation, filtering, transformation, search
7. Always initialize accumulators before loops
8. Avoid modifying list structure during iteration
9. Choose appropriate loop style for your needs
10. Use descriptive variable names for readability

Building lists efficiently with comprehensions

Lecture Notes: Building Lists Efficiently with Comprehensions

Building Lists Efficiently with Comprehensions

Transform your Python code with elegant, concise list comprehensions - the Pythonic way to build lists.

Key Concepts

1. **List Comprehension Syntax**: Concise list creation in one line
2. **Filtering**: Adding conditions to select items
3. **Transformation**: Applying expressions to modify items
4. **Conditional Expressions**: Using if-else within comprehensions
5. **Nested Comprehensions**: Working with 2D data
6. **Best Practices**: When to use comprehensions vs loops

Basic Syntax

```
# Basic comprehension
new_list = [expression for item in iterable]

# With filtering
new_list = [expression for item in iterable if condition]

# With if-else
```

```
new_list = [expr_if if condition else expr_else for item in iterable]

# Nested
new_list = [expression for item1 in iterable1 for item2 in iterable2]
```

Examples

Example 1: Basic List Comprehension

Traditional loop:

```
squares = []
for num in range(1, 6):
    squares.append(num ** 2)
print(squares)  # [1, 4, 9, 16, 25]
```

List comprehension:

```
squares = [num ** 2 for num in range(1, 6)]
print(squares)  # [1, 4, 9, 16, 25]
```

Benefits:

- More concise (1 line vs 3 lines)
- More readable once you understand the syntax
- Faster execution
- More Pythonic

Example 2: Filtering with Conditions

```
# Get even numbers from 1-20
evens = [num for num in range(1, 21) if num % 2 == 0]
print(evens)
# [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

# Filter words longer than 5 characters
words = ["apple", "pie", "banana", "cat", "strawberry", "dog"]
long_words = [word for word in words if len(word) > 5]
print(long_words)
# ['banana', 'strawberry']

# Get positive numbers only
numbers = [-5, 3, -2, 8, -1, 10, -7, 4]
positives = [num for num in numbers if num > 0]
print(positives)
# [3, 8, 10, 4]
```

Example 3: Transform and Filter Together

```python
# Square only even numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_squares = [num ** 2 for num in numbers if num % 2 == 0]
print(even_squares)
# [4, 16, 36, 64, 100]

# Uppercase names starting with 'A'
names = ["Alice", "Bob", "Andrew", "Charlie", "Anna", "David"]
a_names_upper = [name.upper() for name in names if name.startswith('A')]
print(a_names_upper)
# ['ALICE', 'ANDREW', 'ANNA']
```

Example 4: Using if-else (Conditional Expression)

```python
# Label numbers as even or odd
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
labels = ["even" if num % 2 == 0 else "odd" for num in numbers]
print(labels)
# ['odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even']

# Classify scores as Pass or Fail
scores = [45, 78, 92, 65, 88, 53, 71]
results = ["Pass" if score >= 70 else "Fail" for score in scores]
print(results)
# ['Fail', 'Pass', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass']
```

Common Patterns

Pattern 1: Map (Transform Each Element)

```python
# Convert to uppercase
words = ["hello", "world", "python"]
upper = [word.upper() for word in words]

# Calculate lengths
lengths = [len(word) for word in words]
```

Pattern 2: Filter (Select Elements)

```python
# Get even numbers
evens = [n for n in range(20) if n % 2 == 0]

# Get strings longer than 5 chars
long = [s for s in strings if len(s) > 5]
```

Pattern 3: Map and Filter Combined

```python
# Square even numbers
even_squares = [n**2 for n in range(10) if n % 2 == 0]

# Uppercase long words
long_upper = [w.upper() for w in words if len(w) > 5]
```

Best Practices

1. **Keep It Simple**

```python
# Good - clear and readable
squares = [x ** 2 for x in range(10)]

# Bad - too complex
result = [x**2 if x%2==0 else x**3 if x%3==0 else x for x in range(20) if
x>5]
```

2. **Use Meaningful Variable Names**

```python
# Good
adult_names = [person.name for person in people if person.age >= 18]

# Avoid
result = [p.n for p in people if p.a >= 18]
```

3. **Don't Nest Too Deeply**

```python
# Good - flat comprehension
flat = [num for row in matrix for num in row]

# Avoid - too complex
result = [[x*y for x in row1] for row1 in [[a+b for a in r] for r in matrix]]
```

Common Mistakes

1. **Forgetting Brackets**

```python
# Wrong - generator, not list
squares = (x ** 2 for x in range(10))

# Correct - list comprehension
squares = [x ** 2 for x in range(10)]
```

2. **Wrong if-else Placement**

```python
# Wrong - syntax error
result = [x for x in range(10) if x % 2 == 0 else x * 2]

# Correct - if-else before for
result = [x if x % 2 == 0 else x * 2 for x in range(10)]
```

3. **Using When Loop Is Better**

```python
# Don't use comprehension for side effects
# Bad:
[print(x) for x in numbers]  # Creates unnecessary list

# Good:
for x in numbers:
    print(x)
```

Key Takeaways

1. **List comprehensions** create lists concisely in one line
2. **Syntax**: `[expression for item in iterable]`
3. **Add filtering** with `if` at the end
4. **Use if-else** before `for` for conditional expressions
5. **More Pythonic** and often faster than loops
6. **Keep simple** - use loops for complex logic
7. **Creates new list** - doesn't modify original
8. **Readability first** - if it's hard to read, use a loop

Creating and initializing tuples (including single-element tuples)

Lecture Notes: Creating and Initializing Tuples (Including Single-Element Tuples)

Creating and Initializing Tuples

Master Python's immutable sequence type for storing fixed collections of data.

Key Concepts

1. **Tuple Definition**: Ordered, immutable collections
2. **Creation Methods**: Parentheses, packing, constructor
3. **Single-Element Syntax**: Trailing comma requirement
4. **Immutability**: Cannot change after creation
5. **Tuple vs List**: When to use each

6. **Packing/Unpacking**: Efficient data handling

Basic Syntax

```
# Creating tuples
empty_tuple = ()
numbers = (1, 2, 3, 4, 5)
coordinates = (10, 20)
single = (42,)  # Note the trailing comma!

# Tuple packing (parentheses optional)
point = 10, 20, 30

# Tuple unpacking
x, y, z = point

# Constructor
from_list = tuple([1, 2, 3])
from_string = tuple("abc")
```

Examples

Example 1: Creating Tuples

```
# Empty tuple
empty = ()
print(type(empty))  # <class 'tuple'>

# Tuple with items
fruits = ("apple", "banana", "cherry")
numbers = (1, 2, 3, 4, 5)
mixed = (1, "hello", 3.14, True)

# Without parentheses (tuple packing)
coordinates = 10, 20
print(coordinates)  # (10, 20)
print(type(coordinates))  # <class 'tuple'>
```

Example 2: Single-Element Tuples (Critical!)

```
# WRONG - This is NOT a tuple!
not_tuple = (5)
print(type(not_tuple))  # <class 'int'>
print(not_tuple)  # 5

# CORRECT - Need trailing comma
single_tuple = (5,)
print(type(single_tuple))  # <class 'tuple'>
```

```
print(single_tuple)  # (5,)

# Also correct without parentheses
another_single = 42,
print(type(another_single))  # <class 'tuple'>
```

Example 3: Tuple Constructor

```
# From list
my_list = [1, 2, 3, 4, 5]
my_tuple = tuple(my_list)
print(my_tuple)  # (1, 2, 3, 4, 5)

# From string
letters = tuple("Python")
print(letters)  # ('P', 'y', 't', 'h', 'o', 'n')

# From range
numbers = tuple(range(5))
print(numbers)  # (0, 1, 2, 3, 4)
```

Example 4: Tuple Immutability

```
coordinates = (10, 20, 30)

# These will cause errors:
# coordinates[0] = 15  # TypeError
# coordinates.append(40)  # AttributeError
# del coordinates[1]  # TypeError

# Can create new tuple
new_coords = coordinates + (40, 50)
print(new_coords)  # (10, 20, 30, 40, 50)
print(coordinates)  # (10, 20, 30) - unchanged!
```

Example 5: Tuple Packing and Unpacking

```
# Packing
person = "Alice", 25, "Engineer"
print(person)  # ('Alice', 25, 'Engineer')

# Unpacking
name, age, job = person
print(f"{name} is {age} years old and works as {job}")

# Swapping values
a = 5
b = 10
```

```
a, b = b, a
print(a, b)  # 10 5

# Ignoring values
x, _, z = (1, 2, 3)  # Ignore middle value
print(x, z)  # 1 3
```

Example 6: Real-World Use Cases

```
# RGB colors
red = (255, 0, 0)
green = (0, 255, 0)
blue = (0, 0, 255)

def create_color(r, g, b):
    return (r, g, b)

orange = create_color(255, 165, 0)
print(orange)  # (255, 165, 0)

# Coordinates
point_2d = (10, 20)
point_3d = (10, 20, 30)

# Database records
student = ("Alice", 25, "CS", 3.8)
name, age, major, gpa = student

# Function returning multiple values
def get_stats(numbers):
    return min(numbers), max(numbers), sum(numbers) / len(numbers)

minimum, maximum, average = get_stats([5, 10, 15, 20])
print(f"Min: {minimum}, Max: {maximum}, Avg: {average}")
```

Common Patterns

Pattern 1: Fixed Data Representation

```
# Configuration that shouldn't change
DATABASE_CONFIG = ("localhost", 5432, "mydb")
RGB_RED = (255, 0, 0)
SCREEN_SIZE = (1920, 1080)
```

Pattern 2: Function Return Values

```
def divide_with_remainder(a, b):
    return a // b, a % b
```

```
quotient, remainder = divide_with_remainder(17, 5)
print(f"{quotient} remainder {remainder}")  # 3 remainder 2
```

Pattern 3: Multiple Assignment

```
# Swap values
x, y = y, x

# Parallel assignment
first, second, third = 1, 2, 3
```

Best Practices

1. **Use tuples for fixed data**

   ```
   # Good - data that won't change
   DAYS_IN_MONTH = (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)

   # Use list for data that changes
   shopping_cart = ["apple", "banana"]  # Will add/remove items
   ```

2. **Don't forget comma for single-element**

   ```
   # Wrong
   single = (5)  # This is int 5

   # Correct
   single = (5,)  # This is tuple (5,)
   ```

3. **Use for function returns**

   ```
   def get_user():
       return "Alice", 25, "alice@email.com"

   name, age, email = get_user()
   ```

4. **Parentheses for clarity**

   ```
   # Less clear
   point = 10, 20

   # More clear
   point = (10, 20)
   ```

Common Mistakes

1. **Forgetting trailing comma**

```
# Wrong
single = (42)  # int, not tuple

# Correct
single = (42,)  # tuple
```

2. **Trying to modify tuples**

```
# Wrong
t = (1, 2, 3)
t[0] = 10  # TypeError

# Correct - create new tuple
t = (10, 2, 3)
```

3. **Confusing with lists**

```
# Tuple (immutable)
coords = (10, 20)

# List (mutable)
coords = [10, 20]
```

Key Takeaways

1. **Tuples are immutable** - cannot change after creation
2. **Use parentheses** - `(1, 2, 3)` for clarity
3. **Single-element needs comma** - `(5,)` not `(5)`
4. **Faster than lists** - use for fixed data
5. **Perfect for returns** - return multiple values
6. **Packing and unpacking** - convenient syntax
7. **Limited methods** - only count() and index()
8. **Hashable** - can be dict keys (if elements hashable)

Accessing tuple elements using indexing and slicing

Lecture Notes: Accessing Tuple Elements Using Indexing and Slicing

Accessing Tuple Elements

Master tuple indexing and slicing to efficiently extract and work with tuple data.

Key Concepts

1. **Positive Indexing**: Zero-based access from the start
2. **Negative Indexing**: Access from the end
3. **Slicing**: Extract subsets of tuples
4. **Step Parameter**: Control element selection
5. **Nested Access**: Navigate multilevel structures
6. **Immutability**: Read-only element access

Basic Syntax

Indexing

```
# Access single element
element = my_tuple[index]

# Positive index (from start)
first = my_tuple[0]

# Negative index (from end)
last = my_tuple[-1]
```

Slicing

```
# Basic slicing
subset = my_tuple[start:stop]

# With step
subset = my_tuple[start:stop:step]

# Omitting parameters
all_elements = my_tuple[:]
reversed_tuple = my_tuple[::-1]
```

Examples

Example 1: Positive Indexing

```
colors = ("red", "green", "blue", "yellow", "purple")

# Index:   0       1        2        3         4

first = colors[0]
print(first)  # red

second = colors[1]
```

```
print(second)  # green

last_index = colors[4]
print(last_index)  # purple

# IndexError if index out of range
# invalid = colors[10]  # IndexError: tuple index out of range
```

Key Points:

- Indices start at 0
- Last valid index is `len(tuple) - 1`
- Out of range causes IndexError

Example 2: Negative Indexing

```
fruits = ("apple", "banana", "cherry", "date")

# Index:  -4       -3        -2        -1

last = fruits[-1]
print(last)  # date

second_last = fruits[-2]
print(second_last)  # cherry

first_negative = fruits[-4]
print(first_negative)  # apple (same as index 0)
```

Why use negative indexing:

- Easy access to last elements
- No need to calculate `len(tuple) - 1`
- More readable code

Example 3: Basic Slicing

```
numbers = (10, 20, 30, 40, 50, 60, 70)
#          0   1   2   3   4   5   6

# Get elements from index 1 to 4 (4 is exclusive)
subset = numbers[1:4]
print(subset)  # (20, 30, 40)

# Get elements from index 0 to 3
first_three = numbers[0:3]
print(first_three)  # (10, 20, 30)

# Get elements from index 3 to 6
last_part = numbers[3:6]
print(last_part)  # (40, 50, 60)
```

Remember: Stop index is **exclusive** (not included)

Example 4: Slicing with Omitted Parameters

```
letters = ('a', 'b', 'c', 'd', 'e', 'f', 'g')

# From beginning to index 3
print(letters[:3])    # ('a', 'b', 'c')

# From index 3 to end
print(letters[3:])    # ('d', 'e', 'f', 'g')

# Entire tuple (creates a copy)
print(letters[:])     # ('a', 'b', 'c', 'd', 'e', 'f', 'g')

# Last three elements
print(letters[-3:])   # ('e', 'f', 'g')

# All except last two
print(letters[:-2])   # ('a', 'b', 'c', 'd', 'e')
```

Example 5: Using Step Parameter

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

# Every second element
even_indices = numbers[::2]
print(even_indices)  # (0, 2, 4, 6, 8)

# Every third element
every_third = numbers[::3]
print(every_third)  # (0, 3, 6, 9)

# Start at index 1, every second element
odd_indices = numbers[1::2]
print(odd_indices)  # (1, 3, 5, 7, 9)

# From index 2 to 8, step by 2
custom = numbers[2:8:2]
print(custom)  # (2, 4, 6)
```

Example 6: Reversing Tuples

```
original = (1, 2, 3, 4, 5)

# Reverse using negative step
reversed_tuple = original[::-1]
print(reversed_tuple)  # (5, 4, 3, 2, 1)
```

```
# Original unchanged (immutability)
print(original)  # (1, 2, 3, 4, 5)

# Reverse from index 1 to 4
partial_reverse = original[4:1:-1]
print(partial_reverse)  # (5, 4, 3)
```

Example 7: Nested Tuple Access

```
# 2D matrix as tuple of tuples
matrix = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 9)
)

# Access first row
row1 = matrix[0]
print(row1)  # (1, 2, 3)

# Access element at row 0, column 1
element = matrix[0][1]
print(element)  # 2

# Access last row
last_row = matrix[-1]
print(last_row)  # (7, 8, 9)

# Access last element of last row
bottom_right = matrix[-1][-1]
print(bottom_right)  # 9

# Access middle element
middle = matrix[1][1]
print(middle)  # 5
```

Example 8: Real-World - RGB Color

```
color = (255, 128, 64)

# Extract components
red = color[0]
green = color[1]
blue = color[2]

print(f"RGB({red}, {green}, {blue})")
# RGB(255, 128, 64)

# Darken color by halving each component
```

```
darker = tuple(c // 2 for c in color)
print(f"Darker: RGB{darker}")
# Darker: RGB(127, 64, 32)
```

Example 9: Real-World - Function Returns

```
def get_stats(numbers):
    return min(numbers), max(numbers), sum(numbers) / len(numbers)

# Function returns a tuple
stats = get_stats([15, 25, 35, 45, 55])

# Access using indexing
print(f"Min: {stats[0]}")    # Min: 15
print(f"Max: {stats[1]}")    # Max: 55
print(f"Avg: {stats[2]}")    # Avg: 35.0

# Or unpack
minimum, maximum, average = stats
```

Example 10: Real-World - Coordinates

```
# GPS coordinate (latitude, longitude, altitude)
location = (40.7128, -74.0060, 10)

# Extract components
latitude = location[0]
longitude = location[1]
altitude = location[2]

print(f"Location: {latitude}°N, {longitude}°E at {altitude}m")
# Location: 40.7128°N, -74.006°E at 10m

# Get only lat/long (exclude altitude)
lat_long = location[:2]
print(lat_long)  # (40.7128, -74.006)
```

Common Patterns

Pattern 1: Get First and Last

```
data = (10, 20, 30, 40, 50)

first = data[0]
last = data[-1]
```

```
print(f"First: {first}, Last: {last}")
# First: 10, Last: 50
```

Pattern 2: Get All Except First

```
items = ("header", "item1", "item2", "item3")

# Skip the header
content = items[1:]
print(content)  # ('item1', 'item2', 'item3')
```

Pattern 3: Get All Except Last

```
path = ("home", "user", "documents", "file.txt")

# Get directory path without filename
directory = path[:-1]
print(directory)  # ('home', 'user', 'documents')
```

Pattern 4: Get Middle Elements

```
sequence = (1, 2, 3, 4, 5, 6, 7)

# Skip first and last
middle = sequence[1:-1]
print(middle)  # (2, 3, 4, 5, 6)
```

Pattern 5: Split at Index

```
values = (10, 20, 30, 40, 50)

split_index = 3
before = values[:split_index]
after = values[split_index:]

print(f"Before: {before}")  # Before: (10, 20, 30)
print(f"After: {after}")    # After: (40, 50)
```

Best Practices

1. **Use negative indexing for clarity**

```
# Less clear
last = my_tuple[len(my_tuple) - 1]
```

```
# More clear
last = my_tuple[-1]
```

2. **Remember stop is exclusive**

```
# To get indices 1, 2, 3, use stop=4
subset = my_tuple[1:4]  # Gets indices 1, 2, 3
```

3. **Use slicing for safety**

```
# This might fail if tuple is empty
first = my_tuple[0]  # IndexError if empty

# This is safe, returns empty tuple
first_three = my_tuple[:3]  # Always works
```

4. **Immutability means read-only**

```
# Can read
value = my_tuple[0]  # OK

# Cannot modify
# my_tuple[0] = 10  # TypeError
```

Common Mistakes

1. **Forgetting stop is exclusive**

```
numbers = (0, 1, 2, 3, 4)

# Wrong: Think this gets 4 elements
subset = numbers[0:3]  # Actually gets 3: (0, 1, 2)

# Correct
subset = numbers[0:4]  # Gets 4: (0, 1, 2, 3)
```

2. **Confusing indexing with slicing**

```
data = (10, 20, 30)

# Indexing returns element
x = data[1]      # 20 (integer)

# Slicing returns tuple
y = data[1:2]    # (20,) (tuple)
```

3. **Trying to modify tuple elements**

```
# Wrong
coords = (10, 20)
coords[0] = 15  # TypeError

# Correct - create new tuple
coords = (15, 20)
```

4. **Index out of range**

```
small = (1, 2, 3)

# Wrong
value = small[5]  # IndexError

# Correct - check length first
if len(small) > 5:
    value = small[5]
```

Comparison: Indexing vs Slicing

Operation	Indexing	Slicing
Syntax	`tuple[i]`	`tuple[i:j]`
Returns	Single element	New tuple
Out of range	IndexError	Empty tuple or partial
Type	Element type	Always tuple
Example	`(1,2,3)[1]` → `2`	`(1,2,3)[1:2]` → `(2,)`

```
data = (10, 20, 30, 40, 50)

# Indexing
element = data[2]
print(type(element))  # <class 'int'>
print(element)        # 30

# Slicing
subset = data[2:3]
print(type(subset))   # <class 'tuple'>
print(subset)         # (30,)
```

Key Takeaways

1. **Zero-based indexing**: First element is at index 0
2. **Negative indices**: Count from end (-1 is last)

3. **Slicing syntax**: `tuple[start:stop:step]`
4. **Stop is exclusive**: Not included in result
5. **Slicing never fails**: Returns empty or partial tuple
6. **Read-only**: Can access but cannot modify
7. **Nested access**: Use multiple brackets `[i][j]`
8. **Step = -1**: Reverses the tuple
9. **Omit parameters**: Use defaults for convenience
10. **Slicing creates new tuple**: Original unchanged

Implementing tuple packing and unpacking techniques

Implement tuple packing and unpacking techniques

This is a template for Lecture Notes

Verifying tuple immutability and handling modification errors

Verify tuple immutability and handle modification errors

This is a template for Lecture Notes

Using tuple methods count() and index()

Use tuple methods count() and index()

This is a template for Lecture Notes

Creating and initializing sets for unique values

Lecture Notes: Create and Use Sets

Create and Use Sets

This lesson covers sets and uniqueness.

Key Concepts

1. **Definition**: Understanding sets and uniqueness
2. **Syntax**: How to use it correctly
3. **Examples**: Practical demonstrations

4. **Applications**: Real-world use cases

Basic Syntax

```
# Syntax examples for sets and uniqueness
```

Examples

Example 1: Basic Usage

```
# Demonstrate sets and uniqueness
```

Example 2: Practical Application

```
# Real-world example
```

Common Patterns

Standard ways to use sets and uniqueness effectively.

Best Practices

1. Write clear code
2. Handle edge cases
3. Use appropriate methods
4. Follow Python conventions

Common Mistakes

1. **Mistake 1**: Common error
2. **Mistake 2**: Another pitfall

Key Takeaways

1. Main concept 1
2. Main concept 2
3. Main concept 3
4. Best practices
5. When to use

Adding elements to sets using add() and update()

Add elements to sets using add() and update()

This is a template for Lecture Notes

Performing set union and intersection operations

Perform set union and intersection operations

This is a template for Lecture Notes

Performing set difference and symmetric difference operations

Perform set difference and symmetric difference operations

This is a template for Lecture Notes

Applying set membership testing for efficiency

Apply set membership testing for efficiency

This is a template for Lecture Notes

Creating and initializing dictionaries for key-value pairs

Lecture Notes: Create and Use Dictionaries

Create and Use Dictionaries

This lesson covers dictionaries and key-value pairs.

Key Concepts

1. **Definition**: Understanding dictionaries and key-value pairs
2. **Syntax**: How to use it correctly
3. **Examples**: Practical demonstrations
4. **Applications**: Real-world use cases

Basic Syntax

```
# Syntax examples for dictionaries and key-value pairs
```

Examples

Example 1: Basic Usage

```
# Demonstrate dictionaries and key-value pairs
```

Example 2: Practical Application

```
# Real-world example
```

Common Patterns

Standard ways to use dictionaries and key-value pairs effectively.

Best Practices

1. Write clear code
2. Handle edge cases
3. Use appropriate methods
4. Follow Python conventions

Common Mistakes

1. **Mistake 1**: Common error
2. **Mistake 2**: Another pitfall

Key Takeaways

1. Main concept 1
2. Main concept 2
3. Main concept 3
4. Best practices
5. When to use

Accessing dictionary values using keys and get() method

Access dictionary values using keys and get() method

This is a template for Lecture Notes

Adding and modifying dictionary entries

Add and modify dictionary entries

This is a template for Lecture Notes

Iterating through dictionary keys, values, and items

Lecture Notes: Iterate Through Dictionaries

Iterate Through Dictionaries

This lesson covers dict iteration methods.

Key Concepts

1. **Definition**: Understanding dict iteration methods
2. **Syntax**: How to use it correctly
3. **Examples**: Practical demonstrations
4. **Applications**: Real-world use cases

Basic Syntax

```
# Syntax examples for dict iteration methods
```

Examples

Example 1: Basic Usage

```
# Demonstrate dict iteration methods
```

Example 2: Practical Application

```
# Real-world example
```

Common Patterns

Standard ways to use dict iteration methods effectively.

Best Practices

1. Write clear code
2. Handle edge cases
3. Use appropriate methods
4. Follow Python conventions

Common Mistakes

1. **Mistake 1**: Common error
2. **Mistake 2**: Another pitfall

Key Takeaways

1. Main concept 1
2. Main concept 2
3. Main concept 3
4. Best practices
5. When to use

Merging dictionaries using the update() method

Merge dictionaries using the update() method

This is a template for Lecture Notes

Session 10: Python: Data Collections - II

What You'll Learn

In these notes, you'll learn to:

- **Create and manipulate lists** to store and organize multiple values in a single variable
- **Apply essential list operations** like adding, removing, sorting, and accessing elements using indexes
- **Build and work with dictionaries** to store data as key-value pairs (like a real-world contact list)
- **Loop through lists and dictionaries** to process data efficiently and solve real programming problems

---

Part 1: Lists in Python

What Is a List?

Think of a list like a shopping cart. You can add items, remove items, rearrange them, and check what's inside—all in one organized container.

A **list** is an ordered collection that stores multiple items in a single variable. Each item has a position (called an **index**), and you can change what's inside anytime.

Here's a simple list:

```
fruits = ["apple", "banana", "cherry"]
```

This connects to variables you already know. Instead of storing one value, a list stores many values together.

Why Lists Matter

Lists solve a crucial problem: **storing and managing multiple related values**.

Without lists, you'd need separate variables for everything:

```
fruit1 = "apple"
fruit2 = "banana"
fruit3 = "cherry"
```

This gets messy fast! You'll need lists when:

- Building a to-do app (storing multiple tasks)
- Managing student grades (storing scores)
- Tracking shopping items (storing products)
- Processing survey responses (storing answers)

Real-world use case: Netflix uses lists to store your watchlist, viewing history, and recommended shows.

Detailed Walkthrough: Working with Lists

Creating a List

Lists use square brackets `[]` with items separated by commas:

```
numbers = [10, 20, 30, 40, 50]
mixed = [1, "hello", 3.14, True]  # Can mix types!
empty = []  # Empty list
```

Lists are **mutable**, meaning you can change them after creation.

Accessing Elements with Indexing

Each item has a position number starting from **0** (not 1!).

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])  # Output: apple
print(fruits[2])  # Output: cherry
```

Negative indexing counts from the end, starting at **-1**:

```
print(fruits[-1])  # Output: cherry (last item)
print(fruits[-2])  # Output: banana (second to last)
```

Common mistake: Using `fruits[3]` when there are only 3 items causes an error. Remember: indexing starts at 0!

Adding Elements to a List

1. append() - Adds one item to the end:

```
fruits = ["apple", "banana"]
fruits.append("cherry")
print(fruits)  # Output: ['apple', 'banana', 'cherry']
```

2. insert() - Adds an item at a specific position:

```
fruits.insert(1, "orange")  # Insert at index 1
print(fruits)  # Output: ['apple', 'orange', 'banana', 'cherry']
```

3. extend() - Adds multiple items from another list:

```
fruits.extend(["mango", "grape"])
print(fruits)  # Output: ['apple', 'orange', 'banana', 'cherry', 'mango', 'grape']
```

Key difference: `append()` adds the whole list as one item, while `extend()` adds each element separately.

Removing Elements

1. remove() - Removes the first matching value:

```
fruits = ["apple", "banana", "cherry", "banana"]
fruits.remove("banana")  # Removes first "banana"
print(fruits)  # Output: ['apple', 'cherry', 'banana']
```

Caution: If the value doesn't exist, you get an error!

2. pop() - Removes and returns an item by index:

```
fruits = ["apple", "banana", "cherry"]
last = fruits.pop()  # Removes last item
print(last)  # Output: cherry
print(fruits)  # Output: ['apple', 'banana']
```

3. del - Deletes by index without returning:

```
del fruits[0]  # Removes first item
```

4. clear() - Empties the entire list:

```
fruits.clear()
print(fruits)  # Output: []
```

Finding and Counting Elements

Check if something exists:

```
fruits = ["apple", "banana", "cherry"]
print("banana" in fruits)  # Output: True
print("grape" not in fruits)  # Output: True
```

Find the position:

```
position = fruits.index("cherry")
print(position)  # Output: 2
```

Count occurrences:

```
numbers = [1, 2, 3, 2, 2, 5]
count = numbers.count(2)
print(count)  # Output: 3
```

Modifying Values

Change any element by assigning a new value to its index:

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "orange"  # Change banana to orange
print(fruits)  # Output: ['apple', 'orange', 'cherry']
```

Sorting and Reversing

sort() - Arranges in order (modifies original):

```
numbers = [5, 2, 8, 1]
numbers.sort()
print(numbers)  # Output: [1, 2, 5, 8]

numbers.sort(reverse=True)  # Descending order
print(numbers)  # Output: [8, 5, 2, 1]
```

sorted() - Returns a new sorted list (keeps original):

```
numbers = [5, 2, 8, 1]
sorted_nums = sorted(numbers)
print(sorted_nums)  # Output: [1, 2, 5, 8]
print(numbers)  # Output: [5, 2, 8, 1] (unchanged)
```

reverse() - Flips the order:

```
fruits = ["apple", "banana", "cherry"]
fruits.reverse()
print(fruits)  # Output: ['cherry', 'banana', 'apple']
```

Looping Through Lists

Method 1: Direct iteration (most common):

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Method 2: Using indexes:

```
for i in range(len(fruits)):
    print(f"Index {i}: {fruits[i]}")
```

Method 3: Getting both index and value:

```
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
```

Use case: You'd use Method 3 when building a numbered menu in a restaurant app.

List Slicing

Slicing extracts a portion of a list using `[start:end:step]`.

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[2:5])    # Output: [2, 3, 4] (end is exclusive)
print(numbers[:4])     # Output: [0, 1, 2, 3] (start from beginning)
print(numbers[5:])     # Output: [5, 6, 7, 8, 9] (go to end)
print(numbers[::2])    # Output: [0, 2, 4, 6, 8] (every 2nd item)
print(numbers[::-1])   # Output: [9, 8, 7...0] (reverse)
```

Common mistake: Forgetting that the end index is **not included** in the slice.

Nested Lists

Lists can contain other lists, creating a grid-like structure:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matrix[0])      # Output: [1, 2, 3] (first row)
print(matrix[1][2])   # Output: 6 (row 1, column 2)
```

Real-world example: Think of a spreadsheet where each row is a list inside a bigger list.

Part 2: Dictionaries in Python

What Is a Dictionary?

Think of a real dictionary. You look up a **word** (the key) to find its **definition** (the value).

A **dictionary** stores data as **key-value pairs**. Instead of using numbers to access data, you use meaningful labels (keys).

Here's a simple dictionary:

```
student = {"name": "Alice", "age": 20, "grade": 85}
```

This connects to lists, but instead of positions, you use descriptive keys.

Why Dictionaries Matter

Dictionaries solve the problem of **organizing related information** with meaningful labels.

Without dictionaries:

```
student_name = "Alice"
student_age = 20
student_grade = 85
```

You'd need separate variables and have to remember what each represents.

With dictionaries:

```
student = {"name": "Alice", "age": 20, "grade": 85}
print(student["name"])  # Clear and organized!
```

You'll need dictionaries when:

- Storing user profiles (username, email, password)
- Managing product details (name, price, stock)
- Configuring app settings (theme, language, notifications)
- Working with JSON data from websites

Real-world use case: Instagram stores your profile as a dictionary with keys like username, bio, followers, and profile_picture.

Detailed Walkthrough: Working with Dictionaries

Creating a Dictionary

Dictionaries use curly braces `{}` with `key: value` pairs:

```
person = {"name": "Bob", "age": 25, "city": "New York"}
empty_dict = {}
```

Important rule: Keys must be unique. If you repeat a key, only the last value is kept:

```
test = {"age": 20, "age": 25}
print(test)  # Output: {'age': 25}
```

Accessing Values

Access values using keys in square brackets:

```
person = {"name": "Bob", "age": 25}
print(person["name"])  # Output: Bob
```

Common mistake: Trying to access a key that doesn't exist causes an error:

```
print(person["city"])  # ERROR! Key doesn't exist
```

Solution: Use `.get()` method for safe access:

```
city = person.get("city")
print(city)  # Output: None (no error)

city = person.get("city", "Unknown")
print(city)  # Output: Unknown (custom default)
```

Adding and Modifying Items

Add a new key-value pair:

```
person = {"name": "Bob", "age": 25}
person["city"] = "New York"  # Add new key
print(person)  # Output: {'name': 'Bob', 'age': 25, 'city': 'New York'}
```

Modify an existing value:

```
person["age"] = 26  # Update age
print(person)  # Output: {'name': 'Bob', 'age': 26, 'city': 'New York'}
```

It's that simple! If the key exists, you update it. If not, you add it.

Removing Items

1. pop() - Removes a key and returns its value:

```
person = {"name": "Bob", "age": 25, "city": "New York"}
age = person.pop("age")
print(age)  # Output: 25
print(person)  # Output: {'name': 'Bob', 'city': 'New York'}
```

2. del - Deletes a key-value pair:

```
del person["city"]
print(person)  # Output: {'name': 'Bob'}
```

3. clear() - Empties the dictionary:

```
person.clear()
print(person)  # Output: {}
```

Checking for Keys

Use `in` to check if a key exists:

```
person = {"name": "Bob", "age": 25}

if "name" in person:
    print("Name found!")

if "email" not in person:
    print("Email missing!")
```

Use case: Check if a user exists before showing their profile.

Working with Keys, Values, and Items

Get all keys:

```
person = {"name": "Bob", "age": 25, "city": "New York"}
keys = person.keys()
print(keys)  # Output: dict_keys(['name', 'age', 'city'])
```

Get all values:

```
values = person.values()
print(values)  # Output: dict_values(['Bob', 25, 'New York'])
```

Get key-value pairs:

```
items = person.items()
print(items)  # Output: dict_items([('name', 'Bob'), ('age', 25), ('city', 'New
York')])
```

Looping Through Dictionaries

Loop through keys (default):

```
person = {"name": "Bob", "age": 25, "city": "New York"}

for key in person:
```

```
    print(key)
# Output: name, age, city
```

Loop through values:

```
for value in person.values():
    print(value)
# Output: Bob, 25, New York
```

Loop through both keys and values (most useful):

```
for key, value in person.items():
    print(f"{key}: {value}")
# Output:
# name: Bob
# age: 25
# city: New York
```

Real-world example: Display all settings in an app's preferences screen.

Nested Dictionaries

Dictionaries can contain other dictionaries for hierarchical data:

```
students = {
    "student1": {"name": "Alice", "grade": 85},
    "student2": {"name": "Bob", "grade": 92}
}

print(students["student1"]["name"])  # Output: Alice
print(students["student2"]["grade"]) # Output: 92
```

Use case: Storing multiple user profiles where each user has their own details.

Key Type Rules

Keys must be immutable types:

- ✅ Strings: `{"name": "Alice"}`
- ✅ Numbers: `{1: "one"}`
- ✅ Tuples: `{(1, 2): "coordinates"}`
- ❌ Lists: `{[1, 2]: "error"}` (causes error!)

Values can be anything:

```
data = {
    "numbers": [1, 2, 3],
    "nested": {"key": "value"},
    "count": 42
}
```

Key Takeaways

Lists:

- **Lists store ordered collections** using square brackets with numeric indexes starting at 0
- **Use append(), insert(), extend()** to add items; **remove(), pop(), del, clear()** to remove them
- **Loop with for item in list** (direct), **for i in range(len(list))** (by index), or **enumerate()** (both)
- **Slice with [start:end:step]** to extract portions; remember the end is exclusive
- **Lists are mutable** - you can change them anytime, making them perfect for dynamic data

Dictionaries:

- **Dictionaries store key-value pairs** using curly braces where keys act as labels for values
- **Access with dict[key]** or **dict.get(key, default)** to avoid errors
- **Add or update with dict[key] = value**; remove with **pop(), del, or clear()**
- **Loop with .items()** to get both keys and values: `for key, value in dict.items()`
- **Keys must be immutable** (strings, numbers, tuples), but **values can be anything**

Mental Model:

- Think of **lists as numbered shelves** where position matters
- Think of **dictionaries as labeled drawers** where names matter

What's Next: These structures are the building blocks of Python. You'll use them in **tuples** (immutable lists), **sets** (unique collections), and especially **JSON** (data format for web apps). Master these now, and everything else becomes easier!

Practice Tip: The best way to learn is by doing. Try creating a list of your favorite movies and a dictionary of your contact info. Experiment with adding, removing, and looping through them. Check the official Python documentation when you get stuck!

Session 11: Python: Data Collections & File Handling

What You'll Learn

In these notes, you'll learn to:

- **Understand all file modes** in Python (r, w, a, r+, w+, a+, x, and binary modes)
- **Open and close files** safely using the `open()` function and `with` blocks
- **Read data from text files** using different methods based on your needs
- **Write and append data** to files for logging, saving user data, or creating outputs

---

Detailed Explanation

What Is File Handling?

Think of your computer's hard drive like a giant filing cabinet. Each file is a folder with information inside. When you write a program, sometimes you need to:

- **Read** information from those files (like opening a folder to see what's inside)
- **Write** new information to files (like adding a note to a folder)
- **Update** existing files (like editing a document)

- **Save** data permanently (so it's still there even after your program stops running)

File handling is how your Python program interacts with files on your computer—opening them, reading from them, writing to them, and closing them properly.

Here's a simple definition: **File handling is the process of reading from and writing to files stored on your computer's disk.**

If you've already worked with variables and data types in Python, you know how to store information temporarily (in memory). But when your program ends, that data disappears. File handling lets you **save data permanently** so you can use it later or share it with other programs.

Why It Matters

File handling is everywhere in real-world programming. Here's why you need to learn it:

1. Save User Data
When someone fills out a form in your app, you need to save that information somewhere. Files let you store it permanently.

2. Read Configuration Settings
Most apps read settings from files (like `config.txt` or `settings.json`) so users can customize how the app works.

3. Create Logs
You'll need this when tracking errors, user activity, or system events. Logs help you debug problems and understand what happened in your app.

4. Process Existing Data
You might need to read customer lists, product inventories, or analysis results from text files created by other programs.

5. Work with Different File Types
From text files to images to Excel files, understanding file modes helps you handle any file type correctly.

Real-world example: When you build a to-do list app, you'll use file handling to save tasks so they're still there when the user reopens the app tomorrow.

Detailed Walkthrough

Part 1: Understanding File Modes - The Complete Guide

Before you can work with files, you need to understand **file modes**. Think of file modes as different "permissions" you give your program when accessing a file—like telling a librarian whether you want to just read a book, write in it, or both.

The Complete List of File Modes:

Mode	Name	What It Does	Creates File?	Overwrites?
`"r"`	Read	Read only (default)	❌ No (error if missing)	N/A
`"w"`	Write	Write only (creates new)	✅ Yes	✅ Yes (erases content)

`"a"`	Append	Add to end only	✅ Yes	❌ No (preserves content)
`"r+"`	Read + Write	Read and write	❌ No (error if missing)	❌ No (can modify content)
`"w+"`	Write + Read	Write and read	✅ Yes	✅ Yes (erases content)
`"a+"`	Append + Read	Append and read	✅ Yes	❌ No (preserves content)
`"x"`	Exclusive Create	Create only (fails if exists)	✅ Yes	N/A (won't overwrite)
`"rb"`	Read Binary	Read binary files	❌ No	N/A
`"wb"`	Write Binary	Write binary files	✅ Yes	✅ Yes
`"ab"`	Append Binary	Append binary files	✅ Yes	❌ No

Don't worry if this looks overwhelming! We'll go through each one with examples.

Part 2: Text Mode File Operations

Let's explore each text mode in detail with practical examples.

Mode 1: `"r"` - Read Only (Most Common)

When to use: When you just want to **look at** the contents of a file without changing anything.

```
# Reading a file that already exists
with open("shopping_list.txt", "r") as file:
    content = file.read()
    print(content)
```

Key Points:

- ✅ File must already exist (or you'll get `FileNotFoundError`)
- ✅ You can only read, not write
- ✅ Safest mode—you can't accidentally damage the file
- ✅ Default mode (if you don't specify, Python uses `"r"`)

Example Use Case: Reading a configuration file to get app settings.

```
# Read database credentials
with open("config.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Common Mistake: Trying to write in read mode

```
with open("data.txt", "r") as file:
    file.write("Hello")  # ❌ ERROR! Can't write in read mode
```

Mode 2: `"w"` - Write Only (Destructive!)

When to use: When you want to create a **brand new file** or **completely replace** an existing one.

```
# Create a new file and write to it
with open("output.txt", "w") as file:
    file.write("This is line 1\n")
    file.write("This is line 2\n")
```

Key Points:

- ⚠️ **DANGER:** If the file exists, it **erases everything** and starts fresh
- ✅ Creates the file if it doesn't exist
- ✅ You can only write, not read
- ✅ Perfect for generating reports or exports from scratch

Example Use Case: Generating a daily report

```
# Generate today's sales report (replaces yesterday's)
with open("daily_report.txt", "w") as file:
    file.write("Sales Report - 2024-02-13\n")
    file.write("Total Sales: $5,430\n")
    file.write("Top Product: Laptop\n")
```

⚠️ **Warning Example:** The danger of `"w"` mode

```
# Oops! This deletes all your old data
with open("important_data.txt", "w") as file:
    file.write("Just one line\n")
# Everything that was in important_data.txt before is now GONE!
```

Think of `"w"` like this: Using a pencil with an eraser attached. Before you write anything new, the eraser automatically wipes the entire page clean!

Mode 3: `"a"` - Append Only (Safe Addition)

When to use: When you want to **add** new content to the end of a file without deleting what's already there.

```
# Add a new entry to an existing log
with open("activity.log", "a") as file:
    file.write("2024-02-13 10:30 - User logged in\n")
```

Key Points:

- ✅ Adds to the **end** of the file (preserves existing content)
- ✅ Creates the file if it doesn't exist
- ✅ You can only write, not read
- ✅ Perfect for logs, where you keep adding entries

Example Use Case: Keeping a running log

```python
# Log user actions over time
with open("user_log.txt", "a") as file:
    file.write("Session 1: User browsed products\n")

# Later in the program...
with open("user_log.txt", "a") as file:
    file.write("Session 1: User added item to cart\n")

# Later again...
with open("user_log.txt", "a") as file:
    file.write("Session 1: User completed checkout\n")
```

After running this, `user_log.txt` contains all three lines—nothing was deleted!

Think of `"a"` like this: Using a notebook where you always write on the next blank page. You never erase what you wrote before.

Mode 4: `"r+"` - Read AND Write (Most Flexible)

When to use: When you need to both **read existing content** and **modify** it.

```python
# Read file, then add to it
with open("counter.txt", "r+") as file:
    # First, read the current count
    content = file.read()
    count = int(content)

    # Increment it
    count += 1

    # Move cursor back to beginning
    file.seek(0)

    # Write new count (overwrites old)
    file.write(str(count))

    # Remove any leftover characters
    file.truncate()
```

Key Points:

- ✅ Can both read and write
- ❌ File must exist (error if missing)
- ⚠️ Doesn't auto-erase content (but you can overwrite)
- ⚠️ Cursor starts at the **beginning** of the file
- 🔧 You control where to read/write using `seek()`

Example Use Case: Updating a visitor counter

```python
# counter.txt contains: "42"

with open("counter.txt", "r+") as file:
```

```
    # Read current value
    current = int(file.read())
    print(f"Previous visitors: {current}")

    # Update it
    new_count = current + 1

    # Go back to start and overwrite
    file.seek(0)
    file.write(str(new_count))
    file.truncate()  # Remove any extra characters

print(f"New count: {new_count}")
```

Important Methods for `"r+"` mode:

- `file.seek(0)` — Move cursor to beginning of file
- `file.seek(10)` — Move cursor to position 10
- `file.truncate()` — Delete everything after cursor position

Common Mistake: Forgetting to seek

```
with open("data.txt", "r+") as file:
    content = file.read()  # Cursor is now at END of file
    file.write("New data")  # This appends, doesn't replace!
    # You probably wanted to seek(0) first!
```

Mode 5: `"w+"` - Write AND Read (Fresh Start)

When to use: When you want to create a **new file**, write to it, then read back what you wrote—all in one operation.

```
# Create file, write, then read it back
with open("temp.txt", "w+") as file:
    # Write some data
    file.write("Line 1\n")
    file.write("Line 2\n")
    file.write("Line 3\n")

    # Go back to beginning
    file.seek(0)

    # Read what we just wrote
    content = file.read()
    print(content)
```

Key Points:

- ⚠️ **Erases** the file if it exists (like `"w"`)
- ✅ Creates the file if it doesn't exist
- ✅ Can both write and read

- ⚠️ Cursor starts at the **beginning**

Example Use Case: Creating a temporary file for processing

```
# Generate data, process it, then clean up
with open("temp_calculation.txt", "w+") as file:
    # Write calculations
    file.write("10 + 20 = 30\n")
    file.write("15 * 3 = 45\n")

    # Read them back to verify
    file.seek(0)
    for line in file:
        print(f"Verified: {line.strip()}")
```

Difference between `"r+"` and `"w+"`:

Feature	`"r+"`	`"w+"`
File must exist?	✅ Yes	❌ No (creates it)
Erases content?	❌ No	✅ Yes
Use case	Modify existing file	Create new file

Think of it like this:
`"r+"` = Edit an existing document
`"w+"` = Create a brand new document

Mode 6: `"a+"` - Append AND Read (Safe Addition + Reading)

When to use: When you want to **add to the end** of a file and also **read** its contents.

```
# Add to file, then read everything
with open("notes.txt", "a+") as file:
    # Add new note
    file.write("Don't forget to buy milk\n")

    # Go back to beginning to read all notes
    file.seek(0)
    all_notes = file.read()
    print("All notes:")
    print(all_notes)
```

Key Points:

- ✅ Preserves existing content (like `"a"`)
- ✅ Creates file if it doesn't exist
- ✅ Can both append and read
- ⚠️ Cursor starts at the **end** (for appending)
- 🔧 Must use `seek(0)` to read from beginning

Example Use Case: Adding to a log, then displaying it

```python
with open("guest_book.txt", "a+") as file:
    # Add new guest
    file.write("Alice visited on 2024-02-13\n")

    # Show all guests
    file.seek(0)
    print("=== Guest Book ===")
    for guest in file:
        print(guest.strip())
```

Common Mistake: Trying to read without seeking

```python
with open("log.txt", "a+") as file:
    file.write("New entry\n")
    content = file.read()  # Returns empty! Cursor is at end

# Correct way:
with open("log.txt", "a+") as file:
    file.write("New entry\n")
    file.seek(0)  # Go back to start
    content = file.read()  # Now this works!
```

Mode 7: `"x"` - Exclusive Creation (Safety First!)

When to use: When you want to create a file **only if it doesn't exist**. Prevents accidental overwrites!

```python
# Create file only if it doesn't exist
try:
    with open("unique_log.txt", "x") as file:
        file.write("This file was just created!\n")
    print("File created successfully!")
except FileExistsError:
    print("Oops! File already exists. Not overwriting.")
```

Key Points:

- ✅ Creates file if it doesn't exist
- ❌ **Raises error** if file already exists
- ✅ Prevents accidental data loss
- ✅ Write-only (can't read in same operation)

Example Use Case: Ensuring unique filenames

```python
import datetime

# Try to create today's unique log
today = datetime.date.today()
filename = f"log_{today}.txt"

try:
    with open(filename, "x") as file:
```

```
        file.write(f"Log started on {today}\n")
    print(f"Created new log: {filename}")
except FileExistsError:
    print(f"Log for {today} already exists!")
    # Maybe append to it instead
    with open(filename, "a") as file:
        file.write("Additional entry\n")
```

Think of `"x"` like this: It's like a "Create New" button that's grayed out if the file already exists. It protects you from accidentally replacing important data.

When to use each mode - Quick Decision Tree:

```
Do you need to READ existing content?
├─ YES → Does file exist?
│  ├─ YES → Do you need to WRITE too?
│  │  ├─ YES → Use "r+" (read + modify)
│  │  └─ NO → Use "r" (read only)
│  └─ NO → Can't read non-existent file!
│
└─ NO → Do you need to WRITE?
   ├─ YES → Should it PRESERVE existing content?
   │  ├─ YES → Use "a" (append)
   │  └─ NO → Use "w" (overwrite)
   └─ NO → Why are you opening the file? 🤔
```

Part 3: Binary Mode - Working with Non-Text Files

So far we've worked with **text files** (`.txt` , `.csv` , etc.). But what about images, PDFs, videos, or Excel files? These are **binary files**, and they need special handling.

What Are Binary Files?

Text files store information as readable characters (letters, numbers, symbols).
Binary files store information as raw bytes (not directly readable by humans).

Examples of binary files:

- Images (`.jpg` , `.png` , `.gif`)
- Videos (`.mp4` , `.avi`)
- Audio (`.mp3` , `.wav`)
- Documents (`.pdf` , `.docx`)
- Excel files (`.xlsx`)
- Executable programs (`.exe`)

Binary Mode Basics

Add `"b"` to any file mode to work with binary files:

Text Mode	Binary Mode	Use For
`"r"`	`"rb"`	Read binary file
`"w"`	`"wb"`	Write binary file

`"a"`	`"ab"`	Append to binary file
`"r+"`	`"rb+"`	Read + write binary
`"w+"`	`"wb+"`	Write + read binary
`"a+"`	`"ab+"`	Append + read binary

`"rb"` - Read Binary

When to use: When you need to read an image, PDF, or other binary file.

```
# Read an image file
with open("photo.jpg", "rb") as file:
    image_data = file.read()
    print(f"Image size: {len(image_data)} bytes")
```

Example: Copy an image

```
# Copy image from one file to another
with open("original.jpg", "rb") as source:
    image_data = source.read()

with open("copy.jpg", "wb") as destination:
    destination.write(image_data)

print("Image copied successfully!")
```

`"wb"` - Write Binary

When to use: When you need to create or overwrite a binary file.

```
# Download an image and save it (simplified example)
image_bytes = b'\x89PNG\r\n\x1a\n...'  # Binary data

with open("downloaded.png", "wb") as file:
    file.write(image_bytes)
```

Example: Save data in binary format

```
# Save a list of numbers as binary data
import struct

numbers = [10, 20, 30, 40, 50]

with open("data.bin", "wb") as file:
    for num in numbers:
        # Pack as 4-byte integer
        file.write(struct.pack('i', num))
```

`"ab"` - Append Binary

When to use: When you want to add binary data to the end of a file.

```
# Add more binary data to existing file
with open("data.bin", "ab") as file:
    file.write(b'\x00\x01\x02\x03')
```

Important Binary Mode Rules:

1. ✅ Always use `"b"` when working with images, PDFs, videos, etc.
2. ❌ Don't use `"b"` for text files (`.txt` , `.csv`) unless you have a specific reason
3. ⚠️ Binary data is `bytes` type, not `str`
4. ⚠️ No automatic encoding/decoding happens in binary mode

Common Mistake: Opening an image in text mode

```
# ❌ WRONG - Corrupts the image!
with open("photo.jpg", "r") as file:
    data = file.read()

# ✅ CORRECT
with open("photo.jpg", "rb") as file:
    data = file.read()
```

Part 4: Using `with` Blocks (The Modern Way)

You've seen `with` blocks in all the examples above. Let's understand why they're essential.

The Old Way (Not Recommended):

```
# Manually opening and closing
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()  # Easy to forget!
```

Problems with the old way:

- ❌ You might forget to call `.close()`
- ❌ If an error occurs, the file never gets closed
- ❌ File resources stay locked until closed

The New Way (Always Use This!):

```
# Automatically closes the file
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# File is automatically closed here, even if errors occur!
```

Why `with` blocks are better:

✅ **Automatic cleanup** — File closes even if an exception occurs
✅ **Cleaner code** — No need for manual `.close()` calls
✅ **Industry standard** — Professional developers always use this
✅ **Prevents resource leaks** — No forgotten open files

Think of `with` blocks like this:
Imagine borrowing a book from the library. A `with` block is like having a librarian who automatically returns the book for you when you're done—you can't forget!

Example: Even with errors, files get closed

```
# File closes even when this crashes
try:
    with open("numbers.txt", "r") as file:
        number = int(file.read())  # Might raise ValueError
        print(number * 2)
except ValueError:
    print("Not a valid number!")
# File is still properly closed here!
```

Part 5: Reading Files - Different Methods

Once a file is open in read mode, you have several ways to get its contents.

Method 1: `read()` - Get Everything

```
with open("story.txt", "r") as file:
    entire_content = file.read()
    print(entire_content)
```

When to use:

- ✅ Small files (under 1 MB)
- ✅ When you need the whole file as one string
- ❌ Large files (wastes memory)

Method 2: `readline()` - Get One Line

```
with open("data.txt", "r") as file:
    first_line = file.readline()
    second_line = file.readline()
    print(first_line)
    print(second_line)
```

When to use:

- ✅ When you need to process lines one at a time
- ✅ When you only need the first few lines

Method 3: `readlines()` - Get List of Lines

```
with open("tasks.txt", "r") as file:
    all_lines = file.readlines()
```

```
    # Returns: ['Line 1\n', 'Line 2\n', 'Line 3\n']

for line in all_lines:
    print(line.strip())
```

When to use:

- ✅ When you need a list of all lines
- ✅ When you need to access lines by index
- ❌ Very large files (loads entire file into memory)

Method 4: Loop Directly (Best for Large Files)

```
with open("huge_log.txt", "r") as file:
    for line in file:
        print(line.strip())
```

When to use:

- ✅ **Large files** — Only one line in memory at a time
- ✅ When processing line by line
- ✅ Most memory-efficient method

Comparison Table:

Method	Returns	Memory Usage	Best For
`read()`	One string	High	Small files
`readline()`	One line at a time	Low	Sequential reading
`readlines()`	List of strings	High	Need all lines as list
Loop (`for line in file`)	One line per iteration	Low	Large files

Part 6: Writing to Files - Practical Examples

Writing Multiple Lines

```
# Method 1: Multiple write() calls
with open("output.txt", "w") as file:
    file.write("First line\n")
    file.write("Second line\n")
    file.write("Third line\n")

# Method 2: Write a list of lines
lines = ["First line\n", "Second line\n", "Third line\n"]
with open("output.txt", "w") as file:
    file.writelines(lines)
```

Creating a Simple Log System

```python
from datetime import datetime

def log_message(message, level="INFO"):
    """Add a timestamped log entry"""
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"[{timestamp}] [{level}] {message}\n"

    with open("app.log", "a") as log_file:
        log_file.write(log_entry)

# Usage
log_message("Application started", "INFO")
log_message("User logged in", "INFO")
log_message("Failed to connect to database", "ERROR")
log_message("Application shutting down", "INFO")
```

This creates `app.log`:

```
[2024-02-13 14:30:15] [INFO] Application started
[2024-02-13 14:30:20] [INFO] User logged in
[2024-02-13 14:30:45] [ERROR] Failed to connect to database
[2024-02-13 14:31:00] [INFO] Application shutting down
```

Saving User Data

```python
# Collect user information and save
def save_user_info():
    name = input("Enter your name: ")
    email = input("Enter your email: ")
    age = input("Enter your age: ")

    with open("users.txt", "a") as file:
        file.write(f"{name},{email},{age}\n")

    print("User information saved!")

save_user_info()
```

Part 7: Common Mistakes and How to Fix Them

Mistake 1: Not Using `.strip()` When Reading

```python
# Problem: Extra newlines in output
with open("names.txt", "r") as file:
    for name in file:
        print(name)  # Double spacing!

# Solution: Use .strip()
with open("names.txt", "r") as file:
```

```
    for name in file:
        print(name.strip())  # Clean output
```

Mistake 2: Using `"w"` Instead of `"a"` for Logs

```
# ❌ WRONG - Erases yesterday's logs!
with open("daily.log", "w") as log:
    log.write("Today's entry\n")

# ✅ CORRECT - Keeps all logs
with open("daily.log", "a") as log:
    log.write("Today's entry\n")
```

Mistake 3: Forgetting Newline Characters

```
# Problem: Everything on one line
with open("list.txt", "w") as file:
    file.write("Item 1")
    file.write("Item 2")
    file.write("Item 3")
# Result: "Item 1Item 2Item 3"

# Solution: Add \n
with open("list.txt", "w") as file:
    file.write("Item 1\n")
    file.write("Item 2\n")
    file.write("Item 3\n")
```

Mistake 4: Trying to Read After Writing Without Seeking

```
# Problem
with open("data.txt", "w+") as file:
    file.write("Hello World")
    content = file.read()  # Returns empty!

# Solution
with open("data.txt", "w+") as file:
    file.write("Hello World")
    file.seek(0)  # Go back to start
    content = file.read()  # Now it works!
```

Mistake 5: Opening Binary Files in Text Mode

```
# ❌ WRONG - Corrupts the file!
with open("image.jpg", "r") as file:
    data = file.read()

# ✅ CORRECT
```

```python
with open("image.jpg", "rb") as file:
    data = file.read()
```

Part 8: Complete Mode Comparison Cheat Sheet

```
📖 READ ONLY
"r"   → Read text file (must exist)
"rb"  → Read binary file (must exist)

✏️ WRITE ONLY (OVERWRITES!)
"w"   → Write text file (creates/erases)
"wb"  → Write binary file (creates/erases)

➕ APPEND ONLY (SAFE)
"a"   → Append to text file (creates/preserves)
"ab"  → Append to binary file (creates/preserves)

🔄 READ + WRITE
"r+"  → Read & write text (must exist, doesn't erase)
"rb+" → Read & write binary (must exist, doesn't erase)
"w+"  → Write & read text (creates/erases)
"wb+" → Write & read binary (creates/erases)
"a+"  → Append & read text (creates/preserves)
"ab+" → Append & read binary (creates/preserves)

🔒 EXCLUSIVE CREATE
"x"   → Create text file only (fails if exists)
"xb"  → Create binary file only (fails if exists)
```

Part 9: Real-World Scenarios

Scenario 1: Building a Contact Book

```python
def add_contact():
    """Add a new contact to the book"""
    name = input("Name: ")
    phone = input("Phone: ")
    email = input("Email: ")

    with open("contacts.txt", "a") as file:
        file.write(f"{name}|{phone}|{email}\n")
    print("Contact added!")

def view_contacts():
    """Display all contacts"""
    try:
        with open("contacts.txt", "r") as file:
            print("\n=== CONTACTS ===")
            for line in file:
                name, phone, email = line.strip().split("|")
                print(f"Name: {name}")
```

```
                print(f"Phone: {phone}")
                print(f"Email: {email}")
                print("-" * 30)
    except FileNotFoundError:
        print("No contacts yet!")

# Usage
add_contact()
view_contacts()
```

Scenario 2: Counting Word Frequency

```
def count_words(filename):
    """Count how many times each word appears"""
    word_count = {}

    with open(filename, "r") as file:
        for line in file:
            words = line.lower().strip().split()
            for word in words:
                word_count[word] = word_count.get(word, 0) + 1

    # Display top 10 words
    sorted_words = sorted(word_count.items(), key=lambda x: x[1], reverse=True)
    print("Top 10 words:")
    for word, count in sorted_words[:10]:
        print(f"{word}: {count}")

count_words("article.txt")
```

Scenario 3: Backup System

```
def backup_file(original_filename):
    """Create a backup copy of a file"""
    import shutil
    from datetime import datetime

    # Create backup filename with timestamp
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_name = f"{original_filename}.backup_{timestamp}"

    # Copy file
    with open(original_filename, "rb") as source:
        data = source.read()

    with open(backup_name, "wb") as backup:
        backup.write(data)

    print(f"Backup created: {backup_name}")
```

```
backup_file("important_data.txt")
```

Common Confusions Clarified

"When should I use `"r+"` vs `"a+"`?"

- Use `"r+"` when you need to **modify** specific parts of a file (like updating a counter)
- Use `"a+"` when you only need to **add** to the end (like a log file)

"What's the difference between `"w+"` and `"r+"`?"

- `"w+"` **erases** the file first (fresh start)
- `"r+"` **preserves** the file (modify existing content)

"Should I always use binary mode for images?"

Yes! Always use `"rb"`, `"wb"`, or `"ab"` for:

- Images (`.jpg`, `.png`, `.gif`)
- Videos (`.mp4`, `.avi`)
- PDFs (`.pdf`)
- Excel files (`.xlsx`)
- Any non-text file

"Do I need to add `\n` when writing?"

Yes, for text files! Unlike `print()`, `write()` doesn't automatically add newlines. You must add `\n` yourself.

"What happens if I open a file twice?"

```
# Both work fine - they're independent
with open("data.txt", "r") as file1:
    with open("data.txt", "r") as file2:
        # Can read from both simultaneously
        line1 = file1.readline()
        line2 = file2.readline()
```

However, opening for writing twice simultaneously can cause issues depending on your operating system.

Key Takeaways

Here's what you should remember from this lesson:

✅ **File modes control what you can do:**

- `"r"` = Read only (safest)
- `"w"` = Write (dangerous! erases content)
- `"a"` = Append (safe addition)
- `"r+"`, `"w+"`, `"a+"` = Read and write combinations
- Add `"b"` for binary files (images, PDFs, etc.)
- `"x"` = Create only (prevents overwriting)

✅ **Always use `with` blocks** — They automatically close files, even when errors occur

✅ **Reading methods:**

- `read()` for entire file
- `readline()` for one line
- `readlines()` for list of lines
- Loop for memory-efficient line-by-line processing

✅ **Remember:**

- Use `.strip()` when reading to remove `\n`
- Add `\n` when writing (it's not automatic)
- `"w"` erases, `"a"` preserves
- Use binary mode (`"rb"` , `"wb"`) for images and non-text files

✅ **Common patterns:**

- Logs → Use `"a"` mode
- Reports → Use `"w"` mode
- Reading → Use `"r"` mode
- Updating → Use `"r+"` mode

Mental Model: Think of file modes like this:

> `"r"` *= Borrowing a book (read-only, can't write notes)*
> `"w"` *= Getting a fresh notebook (erases old one)*
> `"a"` *= Adding pages to your diary (keeps old entries)*
> `"r+"` *= Editing a document (read and modify)*
> *Binary modes = Working with photos/videos (special handling)*

Looking Ahead: Now that you understand all file modes, you're ready to learn about:

- Working with CSV files (structured data)
- JSON file handling (modern data format)
- Exception handling for files (dealing with errors)
- File paths and directory operations
- Context managers (advanced `with` usage)

These skills build directly on what you learned today!

Practice Challenges

Try these exercises to solidify your understanding:

Challenge 1: Create a simple diary app that:

- Asks users for a daily entry
- Saves it with today's date to `diary.txt` using append mode
- Shows all previous entries when requested

Challenge 2: Write a program that:

- Reads a text file
- Counts total lines, words, and characters
- Displays the statistics (Hint: Use `"r"` mode and loop through lines)

Challenge 3: Build a file backup tool that:

- Reads a file in binary mode
- Creates a copy with `.backup` extension
- Verifies both files are identical (Hint: Use `"rb"` and `"wb"` modes)

Challenge 4: Create a visitor counter that:

- Reads current count from `counter.txt`
- Increments it by 1
- Writes new count back to the same file (Hint: Use `"r+"` mode with `seek(0)` and `truncate()`)

Try these yourself before looking up solutions—you'll learn more by experimenting!

Session 12: NumPy: Numerical Foundations

In this lesson, you'll learn to:

- **Explain** why NumPy is essential for data work and how it differs from regular Python lists
- **Create** NumPy arrays from scratch and convert existing data into arrays
- **Perform** basic mathematical operations on arrays efficiently
- **Reshape** arrays to organize data in different dimensions for analysis

---

Detailed Explanation

a. Intro: What Is NumPy?

Think of organizing a grocery store. You could use a notebook (like Python lists) to track inventory—writing down each item one by one. But what if you had thousands of products? You'd want a computerized system that can instantly calculate totals, update all prices at once, and organize items by category.

NumPy (Numerical Python) is that computerized system for numbers. It's a Python library that lets you work with large collections of numbers incredibly fast.

Here's the key difference: Regular Python lists are like text documents—flexible but slow when doing math. NumPy arrays are like spreadsheets built specifically for calculations—structured and blazingly fast.

If you've worked with Python lists before (the prerequisite), you know they can hold numbers like `[1, 2, 3, 4]`. NumPy builds on this concept but adds superpowers for number crunching.

---

b. Why It Matters

Problem: Imagine you have sales data for 10,000 products and need to:

- Apply a 10% discount to all prices
- Calculate total revenue
- Find average sales per category

With regular Python lists, you'd write loops for each operation. With 10,000 items, that's slow and tedious.

Solution: NumPy lets you do all this in one line—no loops needed—and runs 50-100 times faster.

You'll need this when:

- Analyzing datasets (even small ones with hundreds of rows)
- Building machine learning models
- Processing images (images are just arrays of pixel values)
- Working with scientific or financial data
- Doing any math on collections of numbers

Real-world use case: Every data science library (pandas, scikit-learn, TensorFlow) is built on top of NumPy. Learning it is like learning the foundation before building a house.

---

c. Detailed Walkthrough

Why NumPy? The Performance Story

Let's see the difference with a real example.

Problem: Add 1 to every number in a list of 1 million numbers.

Python List Approach (the slow way):

```
# Regular Python list
numbers = [1, 2, 3, 4, 5]  # imagine this has 1 million items
result = []

for num in numbers:
    result.append(num + 1)
# Result: [2, 3, 4, 5, 6]
# Time: ~100 milliseconds for 1 million items
```

NumPy Approach (the fast way):

```
import numpy as np

numbers = np.array([1, 2, 3, 4, 5])  # imagine this has 1 million items
result = numbers + 1
# Result: [2 3 4 5 6]
# Time: ~2 milliseconds for 1 million items
```

See the difference? No loop needed! NumPy adds 1 to every element automatically. This is called **vectorization**—applying operations to entire arrays at once.

Why is it faster?

- Python lists store data scattered across memory (like files on a messy desk)
- NumPy arrays store data in one continuous block (like a filing cabinet)
- NumPy uses optimized C code under the hood instead of slower Python loops

---

Creating Arrays: Your Building Blocks

NumPy arrays are the foundation. Here's how to create them:

Method 1: From a Python List

```
import numpy as np

# Create from list
prices = np.array([10, 20, 30, 40])
print(prices)
# Output: [10 20 30 40]
```

Think of this as pouring loose change into a coin sorter—you're organizing existing data.

Method 2: Using Built-in Functions

```
# Create array of zeros (useful for initializing data)
zeros = np.zeros(5)
print(zeros)
# Output: [0. 0. 0. 0. 0.]

# Create array of ones
ones = np.ones(4)
print(ones)
# Output: [1. 1. 1. 1.]

# Create range of numbers (like range() but returns array)
sequence = np.arange(0, 10, 2)  # start, stop, step
print(sequence)
# Output: [0 2 4 6 8]

# Create evenly spaced numbers
spaced = np.linspace(0, 1, 5)  # start, stop, how many numbers
print(spaced)
# Output: [0.   0.25 0.5  0.75 1.  ]
```

When to use each:

- `zeros()` → When you need to fill data later (like creating empty boxes)
- `ones()` → When you need a starting value of 1 (multiplying or counting)
- `arange()` → When you need a sequence with specific steps (like page numbers)
- `linspace()` → When you need exactly N evenly spaced values (graphing, time series)

Common Mistake: Forgetting `np.` before functions

```
# Wrong
prices = array([10, 20, 30])  # NameError!

# Right
prices = np.array([10, 20, 30])  # Always use np.
```

Basic Array Math: Operations Made Simple

This is where NumPy shines. Math on entire arrays happens in one line.

Element-wise Operations (applies to each element)

```
prices = np.array([10, 20, 30, 40])

# Add 5 to every price
new_prices = prices + 5
print(new_prices)
# Output: [15 25 35 45]

# Apply 10% discount (multiply by 0.9)
discounted = prices * 0.9
print(discounted)
# Output: [ 9. 18. 27. 36.]

# Square each price
squared = prices ** 2
print(squared)
# Output: [ 100  400  900 1600]
```

Think of it like a magic wand—you wave it once, and the operation happens to everything.

Array + Array Operations

```
monday_sales = np.array([100, 150, 200])
tuesday_sales = np.array([120, 130, 180])

# Total sales per product
total = monday_sales + tuesday_sales
print(total)
# Output: [220 280 380]

# Sales growth (Tuesday - Monday)
growth = tuesday_sales - monday_sales
print(growth)
# Output: [ 20 -20 -20]
```

Aggregation Functions (summarize data)

```
sales = np.array([100, 200, 150, 300, 250])

# Total sales
total = sales.sum()
print(total)  # 1000

# Average sales
average = sales.mean()
print(average)  # 200.0

# Highest sale
max_sale = sales.max()
print(max_sale)  # 300
```

```
# Lowest sale
min_sale = sales.min()
print(min_sale)  # 100
```

Common Mistake: Mixing array sizes incorrectly

```
prices = np.array([10, 20, 30])
discounts = np.array([1, 2])  # Different size!

result = prices + discounts  # Error or unexpected behavior!
# Arrays must be same size OR use broadcasting (advanced topic)
```

Fix: Make sure arrays match in size for element-wise operations.

Shape and Reshape: Organizing Your Data

Arrays aren't always 1D lines—they can be 2D tables or 3D cubes. **Shape** tells you the dimensions.

Understanding Shape

Think of shape like describing a building:

- 1D array: A single row (like a hallway) → shape: `(5,)`
- 2D array: Rows and columns (like a spreadsheet) → shape: `(3, 4)` means 3 rows, 4 columns
- 3D array: Layers of tables (like stacked spreadsheets) → shape: `(2, 3, 4)`

```
# 1D array
row = np.array([1, 2, 3, 4, 5])
print(row.shape)
# Output: (5,) → 5 elements in one dimension

# 2D array (like a table)
table = np.array([[1, 2, 3],
                  [4, 5, 6]])
print(table.shape)
# Output: (2, 3) → 2 rows, 3 columns

# 3D array (think: multiple tables stacked)
cube = np.array([[[1, 2], [3, 4]],
                 [[5, 6], [7, 8]]])
print(cube.shape)
# Output: (2, 2, 2) → 2 layers, 2 rows each, 2 columns each
```

Reshaping: Reorganizing Without Changing Data

Imagine you have 12 eggs. You can arrange them:

- In a line: 12 × 1
- In a rectangle: 3 rows × 4 columns
- In a cube: 2 layers × 2 rows × 3 columns

The eggs don't change—just how they're organized.

```
# Start with 1D array
data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
print(data.shape)
# Output: (12,)

# Reshape to 2D (3 rows, 4 columns)
table = data.reshape(3, 4)
print(table)
# Output:
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
print(table.shape)
# Output: (3, 4)

# Reshape to 3D (2 layers, 2 rows, 3 columns)
cube = data.reshape(2, 2, 3)
print(cube.shape)
# Output: (2, 2, 3)
```

Rule: The total number of elements must match!

- 12 elements can become (3, 4) ✓ → 3 × 4 = 12
- 12 elements CANNOT become (3, 5) ✗ → 3 × 5 = 15 (doesn't match!)

Real Use Case: You might have 10,000 pixel values in a line and need to reshape them into a 100×100 image (2D grid).

Common Mistake: Trying to reshape with wrong dimensions

```
data = np.array([1, 2, 3, 4, 5])

# Wrong
table = data.reshape(2, 3)  # Error! 5 elements can't fit in 2×3=6 slots

# Right
table = data.reshape(5, 1)  # 5×1=5 ✓
```

Pro Tip: Use `-1` to let NumPy calculate one dimension

```
data = np.array([1, 2, 3, 4, 5, 6])

# NumPy figures out: 6 elements ÷ 2 rows = 3 columns
table = data.reshape(2, -1)
print(table.shape)
# Output: (2, 3)
```

Performance vs Python Lists: The Speed Test

Let's prove NumPy is faster with a real benchmark.

Scenario: Multiply 1 million numbers by 2

```
import numpy as np
import time

# Create 1 million numbers
size = 1_000_000

# Python List approach
python_list = list(range(size))
start = time.time()
result = [x * 2 for x in python_list]
python_time = time.time() - start
print(f"Python list: {python_time:.4f} seconds")

# NumPy approach
numpy_array = np.arange(size)
start = time.time()
result = numpy_array * 2
numpy_time = time.time() - start
print(f"NumPy array: {numpy_time:.4f} seconds")

# Compare
speedup = python_time / numpy_time
print(f"NumPy is {speedup:.1f}x faster!")
```

Typical Output:

```
Python list: 0.1523 seconds
NumPy array: 0.0031 seconds
NumPy is 49.1x faster!
```

Why This Matters:

- Small data (100 items)? You won't notice the difference.
- Big data (100,000+ items)? NumPy is essential.
- Machine learning models? NumPy saves hours of processing time.

Memory Efficiency Too:

```
import sys

# Python list
py_list = [1] * 1000
print(f"Python list size: {sys.getsizeof(py_list)} bytes")

# NumPy array
np_array = np.ones(1000)
print(f"NumPy array size: {np_array.nbytes} bytes")
```

NumPy uses less memory because it stores data more efficiently.

---

d. Common Confusions

1. "Do I always need NumPy?"

- **No!** For small tasks (10-100 numbers), Python lists are fine.
- Use NumPy when: data is large, you're doing math, or using data science libraries.

2. "Why does my array look different?"

```
# Regular Python list
py_list = [1, 2, 3]
print(py_list)
# Output: [1, 2, 3]  (with commas)

# NumPy array
np_array = np.array([1, 2, 3])
print(np_array)
# Output: [1 2 3]  (no commas)
```

NumPy uses a different display format—this is normal!

3. "What's the difference between shape (5,) and (5, 1)?"

- `(5,)` → 1D array (a row): `[1, 2, 3, 4, 5]`
- `(5, 1)` → 2D array (a column):

```
[[1]
 [2]
 [3]
 [4]
 [5]]
```

Both have 5 elements, but different dimensions.

Key Takeaways

What to Remember:

- **NumPy is built for speed:** It performs math on arrays 50-100x faster than Python lists by using optimized C code and storing data efficiently.
- **Arrays are the foundation:** Create them with `np.array()`, `np.zeros()`, `np.ones()`, `np.arange()`, or `np.linspace()` depending on your needs.
- **Vectorization eliminates loops:** Instead of writing loops, apply operations directly to arrays: `prices * 0.9` applies to every element automatically.
- **Shape defines structure:** Check with `.shape`, reorganize with `.reshape()`, but remember: total elements must match (12 elements can become (3, 4) but not (3, 5)).
- **Use it when it matters:** Small datasets? Lists are fine. Large datasets or math-heavy work? NumPy is essential.

Mental Model: Think of NumPy as upgrading from a calculator (Python lists) to a spreadsheet (NumPy arrays)—both can do math, but spreadsheets are built for it and handle large amounts of data effortlessly.

What's Next: Now that you understand NumPy basics, you're ready to learn **indexing and slicing** (accessing specific parts of arrays) and **broadcasting** (how NumPy handles arrays of different sizes). These skills will let you manipulate data like a pro.

Quick Reference Code:

```
import numpy as np

# Create arrays
arr = np.array([1, 2, 3])
zeros = np.zeros(5)
sequence = np.arange(0, 10, 2)

# Math operations
result = arr + 5        # Add to all
result = arr * 2        # Multiply all
total = arr.sum()       # Sum
average = arr.mean()    # Average

# Shape and reshape
print(arr.shape)        # Check dimensions
table = arr.reshape(3, 1)  # Reorganize
```

This note covered everything from "what is NumPy" to "why use it" to "how to use it." Practice these concepts with your own data, and you'll quickly see why NumPy is the backbone of data science in Python!

Session 13: Data Structures for AI

Multidimensional Arrays, Slicing, Broadcasting & Random Data

What You'll Learn

In these notes, you'll learn to...

- **Explain** what multidimensional arrays are and how to read their shape, dimensions, and size
- **Build** 1D, 2D, and 3D NumPy arrays and reshape them without losing data
- **Slice** arrays using index notation to extract rows, columns, and sub-regions — the way real AI pipelines do it
- **Predict** the output shape when broadcasting combines arrays of different sizes
- **Generate** random datasets using NumPy and explain how randomness connects to probability distributions

---

Section 1 — Why Data Shape Matters in AI

Before a model can learn anything, data has to be formatted in a very specific way. A neural network doesn't read a spreadsheet — it reads a **grid of numbers**, often with many dimensions stacked on top of each other.

For example:

- A **grayscale image** is a 2D array: rows × columns of pixel brightness values
- A **colour image** is a 3D array: rows × columns × 3 colour channels (RGB)
- A **batch of colour images** is a 4D array: number of images × rows × columns × channels

This is the language AI speaks. NumPy is the tool that lets you build, reshape, and manipulate these structures before feeding them into a model.

> 💡 ***Mental Model:*** *Think of arrays like a set of nested boxes. A 1D array is one row of boxes. A 2D array is a grid of boxes. A 3D array is a stack of grids. Each level of nesting is a new dimension.*

Section 2 — Multidimensional Arrays

Setting Up

```
import numpy as np
```

Always import NumPy as `np` — this is the universal convention.

1D Array — A Single Row

```
a = np.array([10, 20, 30, 40, 50])
print(a.shape)   # (5,)  → 5 elements in one dimension
print(a.ndim)    # 1
```

2D Array — A Grid (Rows × Columns)

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])

print(b.shape)   # (2, 3)  → 2 rows, 3 columns
print(b.ndim)    # 2
```

Think of this like a spreadsheet — rows going down, columns going across.

3D Array — A Stack of Grids

```
c = np.array([[[1, 2], [3, 4]],
              [[5, 6], [7, 8]]])

print(c.shape)   # (2, 2, 2)  → 2 grids, each with 2 rows and 2 columns
print(c.ndim)    # 3
```

> ***How to read the dimension:*** *Count the number of opening brackets `[` before the first number. That's the number of dimensions.*

Reading Array Properties

Every NumPy array carries three key properties you should always know:

Property	What It Tells You	Example
`.shape`	Size along each dimension	`(3, 4)` → 3 rows, 4 cols
`.ndim`	Number of dimensions	`2`
`.size`	Total number of elements	`12`
`.dtype`	Data type of elements	`int64`, `float64`

```
x = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0],
              [7.0, 8.0, 9.0]])

print(x.shape)   # (3, 3)
print(x.ndim)    # 2
print(x.size)    # 9
print(x.dtype)   # float64
```

Reshaping Arrays

You can reorganise the same data into a different shape — as long as the **total number of elements stays the same**.

```
a = np.array([1, 2, 3, 4, 5, 6])   # shape: (6,) → 6 elements

b = a.reshape(2, 3)   # shape: (2, 3) → still 6 elements
print(b)
# [[1 2 3]
#  [4 5 6]]

c = a.reshape(3, 2)   # shape: (3, 2) → still 6 elements
print(c)
# [[1 2]
#  [3 4]
#  [5 6]]
```

> ⚠️ ***Watch Out!*** *You can't reshape* `(6,)` *into* `(4, 2)` *— that would need 8 elements, not 6. NumPy will throw a* `ValueError` *. Always verify:* `rows × cols = total elements` *.*

Useful Array Creation Functions

Instead of typing numbers manually, NumPy gives you shortcuts:

`np.full(shape, value)` — Fill an array with one repeated value

```
np.full((3, 4), 7)
# [[7 7 7 7]
```

```
#  [7 7 7 7]
#  [7 7 7 7]]
```

`np.linspace(start, stop, num)` — Evenly spaced values between two numbers

```
np.linspace(0, 1, 5)
# [0.   0.25  0.5  0.75  1.0]
```

This is used heavily in AI to create evenly distributed test values or learning rate schedules.

`np.tile(array, reps)` — Repeat an array a set number of times

```
np.tile([1, 2, 3], 3)
# [1 2 3 1 2 3 1 2 3]

np.tile([[1, 2], [3, 4]], (2, 3))
# [[1 2 1 2 1 2]
#  [3 4 3 4 3 4]
#  [1 2 1 2 1 2]
#  [3 4 3 4 3 4]]
```

Section 3 — Slicing NumPy Arrays

Slicing means extracting a specific part of an array. In AI, you'll slice arrays constantly — to grab a batch of training samples, isolate one feature column, or extract a region from an image.

Slicing a 1D Array

```
a = np.array([10, 20, 30, 40, 50])

a[0]      # 10          → first element
a[-1]     # 50          → last element
a[1:4]    # [20 30 40]  → index 1 up to (not including) 4
a[::2]    # [10 30 50]  → every other element
a[::-1]   # [50 40 30 20 10]  → reversed
```

The slicing formula: `array[start : stop : step]`

- Omit `start` → begins from index 0
- Omit `stop` → goes to the end
- Omit `step` → defaults to 1

Slicing a 2D Array

For 2D arrays, you provide two indexes separated by a comma: `array[rows, columns]`

```
b = np.array([[1,  2,  3,  4],
              [5,  6,  7,  8],
              [9, 10, 11, 12]])
```

```
b[0, 0]       # 1          → row 0, column 0
b[1, 2]       # 7          → row 1, column 2
b[0, :]       # [1 2 3 4]  → entire first row
b[:, 1]       # [2 6 10]   → entire second column
b[0:2, 1:3]   # [[2 3]     → rows 0–1, columns 1–2
              #  [6 7]]
```

> ***The `:` means "all of this dimension".*** *So `b[:, 0]` means "all rows, column 0" — i.e., the first column.*

Real AI Use Case: Selecting Features

In machine learning, your dataset is often a 2D array where each row is a sample and each column is a feature. Slicing lets you isolate exactly what you need:

```
# dataset shape: (1000, 5) — 1000 samples, 5 features
dataset = np.random.rand(1000, 5)

# Extract the first 100 training samples
train = dataset[:100, :]

# Extract only feature columns 0 and 2
features = dataset[:, [0, 2]]

# Extract the last column as the label/target
labels = dataset[:, -1]
```

> ***Common Mistake:*** *Beginners often forget that `b[1]` and `b[1, :]` both give the second row — but in 3D arrays, `b[1]` gives you the second 2D slice, not a row. Always be explicit with your indexes in higher dimensions.*

Section 4 — Broadcasting

Broadcasting is NumPy's way of performing operations on arrays that don't have exactly the same shape — without you having to manually copy or resize anything.

The Simple Case

```
a = np.array([1, 2, 3, 4])
a + 10
# [11 12 13 14]
```

You added a single number (`10`) to a 4-element array. NumPy **stretched** the `10` across all 4 positions automatically. That's broadcasting at its simplest.

Broadcasting Rules

When you operate on two arrays with different shapes, NumPy follows these rules:

1. Compare shapes **from the right** dimension
2. Dimensions are compatible if they are **equal** or one of them is **1**

3. A dimension of size `1` gets **stretched** to match the other
4. If shapes can't be made compatible — you get a `ValueError`

Worked Example

```
A = np.array([[1],   # shape: (3, 1)
              [2],
              [3]])

B = np.array([10, 20, 30, 40])   # shape: (1, 4)

A + B
# [[11 21 31 41]
#  [12 22 32 42]
#  [13 23 33 43]]
# Output shape: (3, 4)
```

What happened: Column `A` was stretched across 4 columns. Row `B` was stretched down 3 rows. They met in the middle as a (3, 4) grid.

```
Shape A:  (3, 1)
Shape B:  (1, 4)
              ↕ compare right to left
              4 vs 1 → stretch A's columns to 4
              3 vs 1 → stretch B's rows to 3
Result:   (3, 4)  ✅
```

When Broadcasting Fails

```
A = np.array([[1, 2, 3]])    # shape: (1, 3)
B = np.array([[1, 2], [3, 4]])  # shape: (2, 2)

A + B   # ❌ ValueError — 3 and 2 are incompatible, neither is 1
```

> ***Quick Check:*** *Broadcasting works when, for every dimension, the sizes are either equal or at least one of them is* `1` *. If you see any mismatched sizes where neither is* `1` *, broadcasting will fail.*

Broadcasting in AI

This feature is everywhere in AI code. When you subtract the mean from a dataset, you're broadcasting a 1D mean array across a 2D data matrix. When you apply a bias term in a neural network layer, that's broadcasting too.

```
data = np.array([[2.0, 4.0, 6.0],
                 [1.0, 3.0, 5.0],
                 [3.0, 5.0, 7.0]])   # shape: (3, 3)

col_means = data.mean(axis=0)        # shape: (3,)  → mean of each column
centred = data - col_means           # Broadcasting subtracts means column-wise
```

Section 5 — Generating Random Data

In AI, random numbers are used everywhere — shuffling training data, initialising model weights, running simulations, and creating synthetic datasets for testing.

Generating Decimals Between 0 and 1

```
np.random.rand(3, 4)   # 3 rows, 4 columns of random floats in [0, 1)
# [[0.52 0.14 0.89 0.33]
#  [0.67 0.04 0.71 0.22]
#  [0.45 0.93 0.08 0.59]]
```

Generating Random Integers

```
np.random.randint(low=0, high=10, size=(3, 3))
# [[7 2 5]
#  [0 9 3]
#  [4 1 8]]
```

`high` is exclusive — so `randint(0, 10)` gives you 0 through 9, never 10.

Normal Distribution

Most real-world data (heights, test scores, measurement errors) follows a **normal distribution** — a bell curve centred around the mean.

```
np.random.randn(4, 4)   # Random values from a normal distribution
                        # mean = 0, standard deviation = 1
```

Function	Output Range	Distribution
`np.random.rand()`	0 to 1	Uniform (equal chance for any value)
`np.random.randint()`	Integer range you set	Uniform
`np.random.randn()`	Roughly -3 to 3	Normal (bell curve)

> 💡 ***Why normal distribution matters:*** *When you initialise the weights of a neural network, you typically draw from a normal distribution. Values near 0 are more common; extreme values are rare. This helps the model start learning from a balanced state.*

Section 6 — Data Standardization

The Problem

Imagine your dataset has two features: **age** (ranging from 20 to 60) and **salary** (ranging from 30,000 to 200,000). These features are on completely different scales. If you feed this directly into a model, salary will dominate simply because its numbers are bigger — not because it's more important.

Standardization fixes this by putting every feature on the same scale.

The Z-Score Formula

$$z = \frac{x - \mu}{\sigma}$$

- **x** = original value
- **μ** (mu) = mean of the dataset
- **σ** (sigma) = standard deviation of the dataset
- **z** = the standardized value

After standardization, every feature will have a **mean of 0** and a **standard deviation of 1**.

Step-by-Step in NumPy

```
data = np.array([20.0, 35.0, 50.0, 65.0, 80.0])

mean = data.mean()          # μ = 50.0
std  = data.std()           # σ = 21.21 (approx)

z = (data - mean) / std     # Standardize

print(z)
# [-1.41  -0.71   0.    0.71   1.41]
print(z.mean())   # ≈ 0.0
print(z.std())    # ≈ 1.0
```

The original values ranged from 20 to 80. After standardization, they range from about -1.41 to 1.41. The **pattern** in the data is preserved, but the **scale** is normalized.

Standardizing a 2D Dataset (Column-Wise)

In real AI work, your data is 2D — multiple features across many samples. You standardize each feature (column) independently:

```
dataset = np.array([[25,  50000],
                    [30,  80000],
                    [45, 120000],
                    [22,  35000]])

mean = dataset.mean(axis=0)   # Mean of each column  → [30.5, 71250]
std  = dataset.std(axis=0)    # Std of each column   → [8.81, 31...]

standardized = (dataset - mean) / std
```

`axis=0` means "compute across rows" — giving you one mean per column.

Min-Max Scaling (Alternative Method)

Instead of centering around 0, min-max scaling squeezes all values into the range **0 to 1**:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```
x_min = data.min()
x_max = data.max()
scaled = (data - x_min) / (x_max - x_min)
```

Method	Output Range	Best For
Z-score standardization	Roughly -3 to 3	Most ML models, neural networks
Min-max scaling	0 to 1	When you need a fixed bounded range

> ***Watch Out!*** *Always standardize using the* ***training data's mean and std****. When you process new/test data later, use the same training mean and std — not the test data's own statistics. Otherwise your model sees a different scale at test time than it trained on.*

🏁 Key Takeaways

- **NumPy arrays have shape.** Always check `.shape`, `.ndim`, and `.size` before doing anything. Shape mismatches are the #1 source of errors in AI code.
- **Slicing uses `[rows, cols]` syntax.** The `:` means "everything". `b[:, 0]` = first column of every row.
- **Reshaping changes structure, not content.** You can reshape as long as total elements stay the same: `rows × cols = .size`.
- **Broadcasting stretches arrays to match shapes.** It works when dimensions are equal or one of them is 1. When neither is 1 and they differ — it fails.
- **Random generation has three main tools:** `rand()` for uniform floats, `randint()` for integers, `randn()` for a normal distribution.
- **Standardization puts all features on the same scale.** The z-score formula gives mean = 0 and std = 1. Without this, larger-valued features unfairly dominate your model.

> *Every AI model you'll ever build expects data in a specific numerical format. Mastering how to build, slice, and transform arrays is what separates someone who can write code from someone who can actually feed a model.*
>
> ***Up next:*** *Pandas — reading real datasets, handling missing values, and preparing tabular data for AI.*

Practice Exercises

1. **Array Basics** — Create a 1D array of 10 values from 0 to 9. Print its shape, ndim, and dtype.
2. **Reshape** — Create a 1D array with 12 elements and reshape it into: (a) a 3×4 array, (b) a 2×2×3 array. Print both.
3. **Slicing** — Create a 4×5 array using `np.arange(20).reshape(4,5)`. Then extract: the second row, the third column, and the bottom-right 2×2 subarray.
4. **Broadcasting** — Create a (4,1) column array and a (1,5) row array. Add them together and print the result shape. Explain what happened.
5. **Random Data** — Generate a 5×3 array of random integers between 1 and 100. Then generate a 5×3 array from a normal distribution. Compare the value ranges.
6. **Challenge — Standardize** — Create a 2D array with two columns representing age (20–60) and salary (30000–150000) for 6 people. Standardize each column using the z-score formula. Print the mean and std of each column after standardization and verify they are ≈ 0 and ≈ 1.

Session 14: Pandas: The Data Table

What You'll Learn

In these notes, you'll learn to:

- **Explain** what Pandas is and why it's better than NumPy for table-like data
- **Compare** the two core Pandas structures — Series and DataFrame — with real examples
- **Load** a CSV file into Python using Pandas and read it correctly
- **Apply** basic inspection commands like `.head()`, `.tail()`, `.info()`, and `.describe()` to explore any dataset

---

Detailed Explanation

Intro: What Is Pandas?

Imagine you open an Excel file. It has rows, columns, and headers — maybe names, ages, and salaries all in one sheet. Some columns have numbers, some have text. Everything is neatly organized.

Now imagine trying to work with that in Python. Before Pandas, that was a real headache.

Pandas is a Python library that lets you load, organize, and work with that kind of table-based data — directly in Python. Think of it as *Excel, but programmable*.

It's built on top of NumPy (which you've already seen), but it goes much further. It handles **mixed data types**, supports **column names**, and makes data manipulation feel natural.

You'll use Pandas in almost every data science or machine learning project you ever work on — it's that fundamental.

Why It Matters: The Problem with NumPy

Before understanding Pandas, it helps to understand *why we need it*.

NumPy is great for pure numbers. But real-world data is messy. A typical dataset might look like this:

Name	Age	Salary	City
Alice	28	55000	Mumbai
Bob	35	72000	Delhi

Notice the problem? **Name** and **City** are text. **Age** and **Salary** are numbers. If you tried to put this into a NumPy array, NumPy would convert *everything* to strings — and suddenly you can't do math on Age or Salary anymore.

NumPy also has **no concept of column names**. It's just raw numbers in a grid.

Pandas solves both of these problems:

- It handles **mixed data types** across columns
- It gives your data **labels** — column names and row indexes — just like a spreadsheet

---

Pandas Data Structures

Pandas has two core building blocks. Understanding both is essential.

🔷 Series — The Single Column

A **Series** is like a single column in a spreadsheet. It's a one-dimensional list of values, but with labels attached.

```
import pandas as pd

ages = pd.Series([28, 35, 22], index=["Alice", "Bob", "Charlie"])
print(ages)
```

Output:

```
Alice      28
Bob        35
Charlie    22
dtype: int64
```

See how each value has a **label** (Alice, Bob, Charlie)? That's the key difference from a plain Python list or NumPy array. You can access values by name, not just by position.

You can also run statistics on a Series instantly:

```
print(ages.mean())   # 28.33
print(ages.max())    # 35
```

Think of a Series as: a labeled list — like a column in Excel with a name for each row.

🔷 DataFrame — The Full Table

A **DataFrame** is the big one. It's a full two-dimensional table — rows and columns — like an entire Excel sheet.

Every **column** in a DataFrame is actually a Series. So a DataFrame is just a collection of Series, side by side.

Here's how you create one from a dictionary:

```
data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [28, 35, 22],
    "Salary": [55000, 72000, 48000]
}

df = pd.DataFrame(data)
print(df)
```

Output:

```
      Name  Age  Salary
0    Alice   28   55000
1      Bob   35   72000
2  Charlie   22   48000
```

Notice:

- Each **column** has a name (`Name` , `Age` , `Salary`)
- Each **row** has an index (0, 1, 2 by default)
- Columns can hold **different data types** — strings and numbers coexist happily

---

🔶 **Common Confusion: Series vs DataFrame**

Feature	Series	DataFrame
Dimensions	1D (one column)	2D (rows + columns)
Analogy	A single Excel column	A full Excel sheet
Creation	`pd.Series(list)`	`pd.DataFrame(dictionary)`

Loading CSV Files with Pandas

You'll rarely create DataFrames by hand. In real life, you'll **load data from files** — most commonly CSV files.

A **CSV** (Comma-Separated Values) file is just a plain text file where each row is a line, and each value is separated by a comma. It's one of the most common data formats in data science.

Here's how you load one:

```
df = pd.read_csv("employees.csv")
```

That's it. One line. Pandas reads the file, detects the headers, and creates a DataFrame automatically.

Some useful options:

```
# If your file uses semicolons instead of commas:
df = pd.read_csv("data.csv", delimiter=";")

# If you want a specific column to be the row index:
df = pd.read_csv("data.csv", index_col="Name")

# If the file has no header row:
df = pd.read_csv("data.csv", header=None)
```

> ⚠️ ***Common Mistake:*** *Not knowing what delimiter your file uses. Always open the file once in a text editor to check — some CSVs use semicolons or tabs, not commas.*

Basic Inspection — Getting to Know Your Data

Once you've loaded a file, you don't just dive into analysis. First, you **explore it**. Pandas gives you a clean set of commands to do this quickly.

`df.shape` — How Big Is It?

```
print(df.shape)
# Output: (1000, 5) → 1000 rows, 5 columns
```

This tells you the size of your dataset at a glance. Use this first, always.

`df.columns` — What Are the Column Names?

```
print(df.columns)
# Output: Index(['Name', 'Age', 'Salary', 'City', 'Department'], dtype='object')
```

Helps you know exactly what fields you're working with — and how to spell them correctly when you reference them later.

`df.head()` — Peek at the Top

```
print(df.head())       # First 5 rows (default)
print(df.head(10))     # First 10 rows
```

This is your "first look" — use it right after loading to verify the data loaded correctly.

`df.tail()` — Peek at the Bottom

```
print(df.tail())       # Last 5 rows
```

Useful for checking that data didn't get cut off during loading, or that there are no stray rows at the end.

`df.info()` — Column-by-Column Summary

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Name        1000 non-null   object
 1   Age         998 non-null    int64
 2   Salary      1000 non-null   float64
 3   City        995 non-null    object
 4   Department  1000 non-null   object
```

This is one of the most powerful inspection tools. It tells you:

- How many **non-null** (non-missing) values each column has
- The **data type** of each column (`int64` , `float64` , `object`)

> 💡 ***Note:*** *In Pandas, text/string columns show up as* `object` *— that's normal. It doesn't mean anything is wrong.*

`df.describe()` — Quick Statistics

```
print(df.describe())
```

Output:

```
              Age         Salary
count   998.000000    1000.000000
mean     31.450000   62300.000000
std       7.230000   15400.000000
min      18.000000   30000.000000
25%      26.000000   51000.000000
50%      30.000000   60000.000000
75%      36.000000   72000.000000
max      65.000000  120000.000000
```

Gives you instant statistics — count, mean, min, max, and quartiles — for every **numeric column**. You get a feel for the data's range and distribution in seconds.

> ⚠️ ***Common Mistake:*** *Expecting* `describe()` *to show statistics for text columns — it only works on numeric ones by default.*

Accessing Data in a DataFrame

Once you know your data, you'll want to pull specific pieces of it.

Access a single column:

```
df["Age"]            # Returns a Series
df[["Name", "Age"]]  # Returns a DataFrame with two columns
```

Access rows by position with `.iloc[]` :

```
df.iloc[0]     # First row
df.iloc[0:3]   # First 3 rows
```

Access rows by label with `.loc[]` :

```
df.loc[0]              # Row with index label 0
df.loc[0, "Name"]      # Specific cell: row 0, column "Name"
```

The difference in plain English:

- `.iloc[]` — "Give me row number X" (like a position in a list)

- `.loc[]` — "Give me the row *labeled* X" (like looking up by name)

Key Takeaways

- **Pandas** is Python's go-to library for table-based data — it handles mixed data types and labeled columns that NumPy can't.
- A **Series** is a labeled, one-dimensional array — like a single spreadsheet column. A **DataFrame** is a full 2D table made of multiple Series.
- Use `pd.read_csv("file.csv")` to load data files. Check the delimiter if things look odd.
- Your **first five commands** after loading any dataset: `.shape` → `.columns` → `.head()` → `.info()` → `.describe()`
- `.iloc[]` accesses rows by **position number**; `.loc[]` accesses rows by **label name**.

Mental model: Think of a DataFrame as an Excel sheet that Python can read, filter, and transform with just a few lines of code.

Coming up next: Now that you can load and inspect data, you'll learn how to **clean it** — handling missing values, fixing data types, and filtering rows. That's where the real data science begins.

Keep going — you're building the foundation that every data scientist works from daily. The commands you learned today will be in every project you ever build.

Session 15: Introduction to JSON

What You'll Learn

In this lesson, you'll learn to:

- **Explain** what JSON is and why it's the universal language of APIs and AI agents
- **Convert** Python dictionaries into JSON and back, without losing any data
- **Parse** a real API response and pull out exactly the data you need
- **Navigate** nested JSON structures to access deeply buried values

---

Detailed Explanation

a. Intro: What Is JSON?

Imagine you're sending a care package to a friend in another country. You don't just throw everything into a box — you label each item clearly so your friend knows what's what. "Shirt: Blue, Size M." "Book: Harry Potter, Volume 1."

JSON does exactly that for data.

JSON stands for **JavaScript Object Notation**. It's a simple, text-based format for organizing and sending data between programs — kind of like a universal packing list that any system can read, whether it's Python, JavaScript, a mobile app, or an AI agent.

Here's the most important thing to know right now:

> ***JSON is just structured text. Nothing more.***

You've already worked with Python dictionaries — those `{key: value}` structures. JSON looks almost identical. So you're already halfway there.

---

b. Why It Matters

Every time your code talks to the outside world — fetching weather data, calling ChatGPT, logging into Google — the response almost always comes back as JSON.

Without knowing JSON, you'd receive a response from an API and have no idea how to read it. It would look like a wall of confusing text.

With JSON skills, you can:

- Read the response from any API (weather, payments, AI, maps)
- Pull out exactly the piece of data you need (just the temperature, just the price)
- Build AI agents that send and receive structured information reliably

You'll need this the moment you start building anything that connects to the internet or uses a third-party service.

---

c. Detailed Walkthrough

Part 1: JSON Structure — The Building Blocks

JSON has just a handful of building blocks. Once you know these, you can read almost any JSON in the wild.

The basic unit is a key-value pair:

```
{
  "name": "Alice",
  "age": 28,
  "is_active": true
}
```

Let's break this down:

- **Keys** are always strings in double quotes: `"name"`
- **Values** can be: a string `"Alice"`, a number `28`, a boolean `true` or `false`, a list, another object, or `null`
- Pairs are separated by commas
- The whole thing is wrapped in curly braces `{ }`

Think of it like a form someone filled out. The labels on the form are keys. The answers are values.

> ⚠️ ***Common Mistake:*** *In Python you can use single quotes `'name'`. In JSON, you must use double quotes `"name"`. Mixing them up is one of the most common beginner errors.*

Part 2: Dictionaries to JSON — Going Both Ways

Here's the good news: Python has a built-in tool called the **`json` module** that handles all of this for you.

Converting a Python dictionary → JSON string:

```python
import json

user = {
    "name": "Alice",
    "age": 28,
    "is_active": True
}

json_string = json.dumps(user)
print(json_string)
# Output: {"name": "Alice", "age": 28, "is_active": true}
```

`json.dumps()` — think of the **"s"** as "string." It *dumps* your dictionary into a JSON-formatted string.

Notice Python's `True` became JSON's `true`. The `json` module handles that translation automatically.

Converting JSON string → Python dictionary:

```python
json_string = '{"name": "Alice", "age": 28, "is_active": true}'

user = json.loads(json_string)
print(user["name"])  # Output: Alice
```

`json.loads()` — the **"s"** again means "string." It *loads* a JSON string back into a Python dictionary you can actually work with.

The mental model here:

Direction	Function	Memory Trick
Python dict → JSON text	`json.dumps()`	**Dump** it into text
JSON text → Python dict	`json.loads()`	**Load** it into Python

Part 3: Parsing API Responses

Now let's make this real. When you call an API — say, a weather API or the OpenAI API — the response arrives as JSON. Your job is to unpack it and grab what you need.

Here's what a typical API response might look like:

```python
import json

api_response = '''
{
  "status": "success",
  "temperature": 24,
  "city": "Bengaluru",
  "unit": "Celsius"
}
'''
```

```
data = json.loads(api_response)

print(data["city"])          # Bengaluru
print(data["temperature"])   # 24
```

You treat it exactly like a dictionary after you parse it with `json.loads()`. Access any value using its key in square brackets.

> ⚠️ ***Common Mistake:*** *Trying to use `data["temperature"]` before calling `json.loads()`. If you skip the parsing step, `data` is still just a string — and strings don't have keys.*

Part 4: Nested JSON — JSON Inside JSON

Real-world API responses are rarely flat. They're often **nested** — meaning a value inside the JSON is itself another JSON object, or a list of objects.

Think of it like a folder inside a folder on your computer. You have to open the outer folder first, then the inner one.

Example: An AI agent's API response

```
response = {
  "agent": "WeatherBot",
  "result": {
    "city": "Bengaluru",
    "forecast": {
      "today": "Sunny",
      "tomorrow": "Rainy"
    }
  }
}
```

To get tomorrow's forecast, you chain your keys:

```
tomorrow = response["result"]["forecast"]["tomorrow"]
print(tomorrow)  # Rainy
```

Read it left to right: *"Go into `result`, then into `forecast`, then grab `tomorrow`."*

What about lists inside JSON?

JSON often contains **arrays** (which become Python lists). For example:

```
response = {
  "agent": "ShoppingBot",
  "items": ["apples", "milk", "bread"]
}

print(response["items"][0])  # apples
print(response["items"][1])  # milk
```

First you access the key `"items"` to get the list, then you use an index `[0]`, `[1]` to get individual elements — just like a regular Python list.

A more realistic nested structure:

```
response = {
  "users": [
    {"id": 1, "name": "Alice"},
    {"id": 2, "name": "Bob"}
  ]
}

# Get Bob's name
print(response["users"][1]["name"])  # Bob
```

Step by step:

1. `response["users"]` → gives you the list
2. `[1]` → gives you the second item (Bob's dictionary)
3. `["name"]` → gives you the value `"Bob"`

---

d. Other Add-ons

Common Confusions

People often mix up **JSON** and **Python dictionaries**. Here's the key difference: a Python dictionary is a *live data structure* your program can work with. JSON is *text* — it's just for storage and transport. The `json` module is your bridge between the two.

Tip: Pretty-printing JSON

When debugging, raw JSON can be hard to read. Use `indent` to make it beautiful:

```
print(json.dumps(data, indent=4))
```

This adds indentation and line breaks so you can actually see the structure at a glance. Super useful when you're dealing with large, deeply nested responses.

Caution: Key errors in nested JSON

If you try to access a key that doesn't exist, Python throws a `KeyError`. Use `.get()` as a safe alternative:

```
city = data.get("city", "Unknown")
```

If `"city"` exists, you get its value. If not, you get `"Unknown"` instead of a crash. Always a good habit with API responses, since APIs can sometimes return incomplete data.

Key Takeaways

- **JSON is structured text** using key-value pairs, wrapped in `{ }`. It's the standard format for sending data between systems, especially APIs.
- `json.dumps()` converts a Python dictionary → JSON string. `json.loads()` converts a JSON string → Python dictionary.
- **Always parse first.** Call `json.loads()` on an API response before trying to access any values.
- **Nested JSON** is just JSON inside JSON. Chain your keys (`data["a"]["b"]["c"]`) to dig into it — and use indexes (`[0]` , `[1]`) to pull from lists.
- **Use** `.get()` instead of `[]` when you're not sure a key will exist — it prevents crashes.

Mental Model: Think of JSON as a *shipping label* for data. It wraps your information in a universally readable format for the journey. Once it arrives, `json.loads()` unwraps it so Python can use it.

Coming Up Next: Now that you can read and write JSON confidently, you're ready to make real API calls — sending requests and handling the JSON responses that come back. Everything you learned here becomes the foundation for working with any API or AI agent in the real world.

Session 16: Pandas: Selection & Filtering

What You'll Learn

In this session, you'll learn to:

• Select single and multiple columns from DataFrames to focus on exactly the data you need • Use `.loc` and `.iloc` to access specific rows and cells by labels or positions • Filter datasets using Boolean conditions to answer questions like "Show me all female passengers" or "Find fares under $10" • Sort DataFrames by one or multiple columns to organize your data meaningfully

---

Detailed Explanation

a. Intro: What Is Selection & Filtering?

Imagine you're working at a library with 10,000 books. Someone asks: *"Show me all mystery novels published after 2010, sorted by rating."* You wouldn't scan every shelf manually—you'd use the library's search system to **filter** by genre and year, then **sort** the results.

Selection and filtering in Pandas work exactly the same way. Instead of manually browsing through thousands of rows in a spreadsheet, you write simple code to extract exactly what you need.

You already know how to load data with `pd.read_csv()` and preview it with `head()`. Now you'll learn to **ask questions** of your data: "Which passengers survived?" "Who paid the highest fare?" "Show me children under 10 years old." This is where data analysis truly begins.

---

b. Why It Matters

Real-world datasets are messy and huge. You rarely need *all* the data—you need **specific slices** that answer your questions.

You'll use selection and filtering when:

- Analyzing customer data: "Find all purchases over $500 in California"
- Cleaning datasets: "Remove rows with missing email addresses"

- Building reports: "Show top 10 products by revenue"
- Exploring trends: "Compare male vs female survival rates on the Titanic"

Real-world use case: A marketing team has 50,000 customer records. Instead of scrolling through Excel, you filter for "customers who haven't purchased in 6 months" in 2 seconds, then export that list for a re-engagement campaign.

---

c. Detailed Walkthrough

For this lesson, we'll use the **Titanic dataset**—a real dataset of passengers from the famous 1912 shipwreck. It includes columns like `survived`, `pclass` (ticket class), `sex`, `age`, `fare`, and more.

Loading the dataset:

```
import pandas as pd
import seaborn as sns

# Load Titanic dataset (comes built-in with Seaborn)
df = sns.load_dataset('titanic')

print(df.head())
```

Output (first 5 rows):

```
   survived  pclass     sex   age  fare
0         0       3    male  22.0   7.25
1         1       1  female  38.0  71.28
2         1       3  female  26.0   7.92
3         1       1  female  35.0  53.10
4         0       3    male  35.0   8.05
```

Let's explore this data step by step.

Step 1: Exploring Your Data First

Before filtering or selecting, always get a feel for your data.

Use `.info()` for a quick summary:

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   survived    891 non-null    int64
 1   pclass      891 non-null    int64
 2   sex         891 non-null    object
 3   age         714 non-null    float64
```

```
 4   fare        891 non-null    float64
...
```

What this tells you:

- 891 passengers (rows)
- 15 columns
- Some columns have missing data (e.g., `age` has only 714 values, so 177 are missing)
- Data types: `int64` (numbers), `float64` (decimals), `object` (text/strings)

Use `.describe()` for numeric statistics:

```
df.describe()
```

Output:

```
       survived      pclass         age        fare
count    891.00      891.00      714.00      891.00
mean       0.38        2.31       29.70       32.20
std        0.49        0.84       14.53       49.69
min        0.00        1.00        0.42        0.00
max        1.00        3.00       80.00      512.33
```

Quick insights:

- About 38% survived (`mean` of `survived` column ≈ 0.38)
- Average age ~30 years
- Fares ranged from $0 to over $500!

Tip: Always run `.info()` and `.describe()` when you first load a dataset. It saves you from surprises later.

Step 2: Selecting Columns

Problem: Your DataFrame has 15 columns, but you only care about `sex`, `age`, and `fare`.

Solution: Select specific columns

Selecting ONE column (returns a Series):

```
ages = df['age']
print(ages.head())
```

Output:

```
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
Name: age, dtype: float64
```

Notice this is a **Series** (just one column).

Selecting MULTIPLE columns (returns a DataFrame):

```
subset = df[['sex', 'age', 'fare']]
print(subset.head())
```

Output:

```
      sex   age   fare
0    male  22.0   7.25
1  female  38.0  71.28
2  female  26.0   7.92
3  female  35.0  53.10
4    male  35.0   8.05
```

Common mistake: Forgetting the **double brackets** for multiple columns.

```
# WRONG - This will cause an error
df['sex', 'age']

# CORRECT - Use double brackets
df[['sex', 'age']]
```

Why double brackets? You're passing a *list* of column names. `[['sex', 'age']]` means "here's a list of columns I want."

Selecting only numeric columns:

```
numeric_data = df.select_dtypes(include=['number'])
print(numeric_data.head())
```

This grabs all columns with numbers (like `age`, `fare`, `pclass`), ignoring text columns like `sex`.

Step 3: Row Indexing with `.loc` and `.iloc`

Selecting columns is easy. But what if you want **specific rows**? That's where `.loc` and `.iloc` come in.

Think of it like this:

- `.iloc` = "Select by **position**" (like saying "Give me row 5")
- `.loc` = "Select by **label/index**" (like saying "Give me the row labeled 'passenger_123'")

Using `.iloc` (integer position):

```
# Get the first row (position 0)
first_passenger = df.iloc[0]
print(first_passenger)
```

Output:

```
survived      0
pclass        3
```

```
sex        male
age        22.0
fare       7.25
Name: 0, dtype: object
```

Get first 3 rows:

```
first_three = df.iloc[0:3]
print(first_three)
```

Output:

```
   survived  pclass     sex   age  fare
0         0       3    male  22.0  7.25
1         1       1  female  38.0 71.28
2         1       3  female  26.0  7.92
```

Get a specific cell (row 2, column 3):

```
# Row 2, 4th column (age)
df.iloc[2, 3]  # Output: 26.0
```

Using `.loc` (label-based):

By default, Pandas uses row numbers (0, 1, 2...) as the index. So `.loc[0]` would also get the first row. But `.loc` really shines when you have **custom indexes**.

```
# Same as iloc[0] when index is numeric
df.loc[0]
```

Setting a custom index:

```
# Use 'name' column as the index
df_indexed = df.set_index('name')
print(df_indexed.loc['Braund, Mr. Owen Harris'])
```

Now you can look up passengers by name instead of position!

Key difference:

```
# iloc: Slicing EXCLUDES the end
df.iloc[0:3]  # Rows 0, 1, 2

# loc: Slicing INCLUDES the end
df.loc[0:3]   # Rows 0, 1, 2, 3
```

When to use which?

- Use `.iloc` when you're thinking "I want the first 10 rows" or "Give me row number 42"

- Use `.loc` when you're thinking "I want the row labeled 'customer_ABC'" or filtering by conditions (next section!)

Step 4: Boolean Filtering – Asking Questions

This is where the magic happens. **Boolean filtering** lets you ask questions like:

- "Show me all female passengers"
- "Find passengers younger than 18"
- "Who paid more than $100 for a ticket?"

How it works: Create a True/False mask

```
# Which passengers are female?
mask = df['sex'] == 'female'
print(mask.head())
```

Output:

```
0    False
1     True
2     True
3     True
4    False
Name: sex, dtype: bool
```

This creates a **Boolean Series**—True for females, False for males.

Use the mask to filter:

```
females = df[df['sex'] == 'female']
print(females.head())
```

Now `females` contains only rows where `sex` is `'female'`.

More examples:

```
# Passengers older than 60
seniors = df[df['age'] > 60]

# Passengers who paid less than $10
cheap_tickets = df[df['fare'] < 10]

# First-class passengers
first_class = df[df['pclass'] == 1]
```

Combining conditions with `&` (AND), `|` (OR), `~` (NOT):

Problem: Find female passengers in first class.

```
# Use & for AND
first_class_females = df[(df['sex'] == 'female') & (df['pclass'] == 1)]
```

```
print(first_class_females.head())
```

Important: Each condition needs **parentheses** around it!

```
# WRONG - Will cause an error
df[df['sex'] == 'female' & df['pclass'] == 1]

# CORRECT
df[(df['sex'] == 'female') & (df['pclass'] == 1)]
```

Using `|` for OR:

```
# Passengers in first OR second class
upper_class = df[(df['pclass'] == 1) | (df['pclass'] == 2)]
```

Using `~` for NOT:

```
# Everyone EXCEPT males
not_male = df[~(df['sex'] == 'male')]
```

Using `.isin()` for multiple values:

Instead of writing `(df['pclass'] == 1) | (df['pclass'] == 2)`, use:

```
upper_class = df[df['pclass'].isin([1, 2])]
```

Much cleaner!

Filtering missing data:

```
# Find rows where age is missing
missing_age = df[df['age'].isna()]

# Find rows where age is NOT missing
has_age = df[df['age'].notna()]
```

Filtering ranges with `.between()`:

```
# Ages between 20 and 30 (inclusive)
twenties = df[df['age'].between(20, 30)]
```

Real-world example:

You're analyzing the Titanic data. Your boss asks: *"How many children under 12 survived?"*

```
children_survived = df[(df['age'] < 12) & (df['survived'] == 1)]
print(len(children_survived))  # Number of rows
```

Step 5: Sorting DataFrames

Problem: Your data is in random order. You want to see the youngest passengers first, or sort by fare from highest to lowest.

Solution: Use `.sort_values()`

Sort by one column:

```
# Sort by age (youngest first)
sorted_by_age = df.sort_values('age')
print(sorted_by_age.head())
```

Output:

```
     survived  pclass     sex   age   fare
803         1       3    male  0.42  8.52
755         1       2    male  0.67 14.50
644         1       3  female  0.75 19.26
...
```

Sort descending (oldest first):

```
sorted_desc = df.sort_values('age', ascending=False)
```

Sort by multiple columns:

```
# Sort by class first, then by age within each class
sorted_multi = df.sort_values(['pclass', 'age'])
```

Specify different orders for each column:

```
# Class ascending, age descending
df.sort_values(['pclass', 'age'], ascending=[True, False])
```

Common mistake: Sorting doesn't change the original DataFrame unless you use `inplace=True`.

```
# This creates a new sorted DataFrame
sorted_df = df.sort_values('age')

# This modifies the original
df.sort_values('age', inplace=True)
```

Caution: Using `inplace=True` permanently changes your data. Usually better to create a new variable so you can go back if needed.

Step 6: Chaining Operations (Putting It All Together)

Pandas lets you **chain** multiple operations in one line. This is powerful for quick analysis.

Example: Find first-class female survivors, sort by fare, show only name, age, and fare

```
result = (df[(df['pclass'] == 1) &
             (df['sex'] == 'female') &
             (df['survived'] == 1)]
          [['name', 'age', 'fare']]
          .sort_values('fare', ascending=False))

print(result.head())
```

What's happening:

1. Filter for first-class females who survived
2. Select only `name`, `age`, `fare` columns
3. Sort by fare (highest first)

This reads almost like English: "Give me first-class female survivors, show their names/ages/fares, sorted by fare."

Tip: Use parentheses and line breaks for readability. Code is for humans too!

---

d. Other Add-ons

Common Confusions:

1. **`.loc` vs `.iloc`:**
   - `.iloc` = position (0, 1, 2...)
   - `.loc` = label/index value
   - After filtering, row positions change but labels stay the same, so `.iloc` might give unexpected results!
2. **Index gets messy after filtering:**

   ```
   # After filtering, index might be: [0, 5, 12, 34...]
   filtered = df[df['age'] > 50]

   # Reset to sequential: [0, 1, 2, 3...]
   filtered = filtered.reset_index(drop=True)
   ```

3. **Parentheses in Boolean conditions:** ALWAYS use them!

   ```
   # WRONG
   df[df['age'] > 30 & df['sex'] == 'male']

   # CORRECT
   df[(df['age'] > 30) & (df['sex'] == 'male')]
   ```

Tips:

- **Test small before scaling big:** Try your filter on `.head(50)` first to make sure it works
- **Save filtered results:** `filtered_df = df[condition]` so you don't lose your work

- **Check your output:** After filtering, always print `.shape` to see how many rows you got
- **Read the docs:** Pandas documentation is excellent. Google "pandas filter dataframe" for examples

Cautions:

- **Don't trust your eyes alone:** A DataFrame might look right but have hidden issues. Use `.info()` to verify.
- **Missing data breaks things:** `df['age'] > 30` won't include rows where age is `NaN`. Handle missing data first!
- **Chaining is powerful but can be unreadable:** If your chain is 5+ operations, break it into steps for clarity.

Key Takeaways

• **Select columns** with `df['column']` (one column, returns Series) or `df[['col1', 'col2']]` (multiple columns, returns DataFrame)

• `.iloc[row, col]` selects by position (0-indexed); `.loc[label, col]` selects by index label—use `.iloc` for "give me row 5" and `.loc` for filtering and custom indexes

• **Boolean filtering** answers questions: `df[df['age'] > 30]` finds rows where age > 30; combine conditions with `&` (AND), `|` (OR), `~` (NOT)—always use parentheses!

• **Sort** with `.sort_values('column')` for ascending or `.sort_values('column', ascending=False)` for descending; sort by multiple columns with a list

• **Chain operations** for powerful one-liners: filter → select columns → sort, all in one expression

Mental model: Think of filtering like using a search bar in an app—you type conditions, and Pandas instantly shows matching results. Sorting is like clicking column headers in Excel to reorganize.

What's next? Now that you can slice and filter data, you'll learn to **transform** it—adding new columns, grouping data to find patterns (like average fare by class), and handling missing values properly. Your data analysis superpowers are just getting started!

Session 17: Pandas: Cleaning & Grouping

What You'll Learn

In this session, you've learn to:

- Identify missing data in your dataset and apply appropriate cleaning strategies (dropping vs. filling)
- Fill missing values using different techniques like mean, median, mode, forward fill, and backward fill
- Group data by one or multiple columns to calculate summary statistics and uncover patterns
- Apply custom aggregation functions to answer specific business questions from your data

---

Detailed Explanation

Part 1: Handling Missing Values

What Is Missing Data?

Think of a spreadsheet where some cells are blank. Maybe a customer didn't provide their phone number, or a sensor failed to record temperature at certain times.

Missing data refers to absent or undefined values in your dataset. In Pandas, these appear as `NaN` (Not a Number) or `None`.

This builds directly on what you already know about DataFrames. Now we're learning how to handle the reality that real-world data is messy and incomplete.

Why It Matters

Missing data can break your analysis or machine learning models. You can't calculate an average age if some ages are missing. You can't predict customer behavior if key information is absent.

You'll need this when:

- Cleaning survey data where respondents skipped questions
- Working with sensor data that has recording gaps
- Preparing datasets for machine learning (most algorithms can't handle missing values)

Real-world example: In the Titanic dataset, the "Age" column has missing values because not all passenger ages were recorded. Before analyzing survival rates by age group, you must decide how to handle these gaps.

Detailed Walkthrough

Step 1: Identify Missing Data

Before fixing anything, find out what's missing.

```
# Check which columns have missing data
df.info()

# Count missing values per column
df.isna().sum()
```

The `info()` method shows you each column's data type and non-null count. The `isna().sum()` gives exact counts of missing values.

Step 2: Decide Your Strategy

You have two main options: **drop** or **fill**.

Option A: Dropping Missing Data

Use `dropna()` to remove rows with missing values.

```
# Remove ALL rows with ANY missing value
df_clean = df.dropna()

# Remove rows missing values in specific columns only
df_clean = df.dropna(subset=['Age', 'Fare'])
```

When to drop:

- When missing data is less than 5% of your dataset
- When the missing values are random and don't represent a pattern
- When you have enough remaining data after dropping

Warning: Dropping is destructive. You lose information permanently. If 30% of ages are missing, you'd lose 30% of your data. That's usually too much.

Option B: Filling Missing Data

Use `fillna()` to replace missing values with something meaningful.

```
# Fill all missing values with zero
df_filled = df.fillna(0)

# Fill missing ages with the average age
df['Age'] = df['Age'].fillna(df['Age'].mean())

# Fill missing values differently per column
df.fillna({
    'Age': df['Age'].median(),
    'Fare': df['Fare'].mean(),
    'Embarked': df['Embarked'].mode()[0]
})
```

Common fill strategies:

- **Numerical columns:** Use mean or median
  - Mean works when data is normally distributed
  - Median is better when you have outliers (like income data)
- **Categorical columns:** Use mode (most frequent value)
  - If most passengers embarked from 'S', fill missing embarkation points with 'S'

Step 3: Advanced Filling Techniques

Sometimes the order of your data matters. Think of stock prices or temperature readings over time.

Forward Fill (`ffill`): Copy the last known value forward

```
# If Monday's temperature was 25°C and Tuesday is missing,
# assume Tuesday was also 25°C
df['Temperature'] = df['Temperature'].fillna(method='ffill')
```

Backward Fill (`bfill`): Copy the next known value backward

```
# Use the next available value to fill gaps
df['Temperature'] = df['Temperature'].fillna(method='bfill')
```

When to use forward/backward fill:

- Time-series data where values change gradually
- Sequential data where missing values likely match nearby values

- Sensor readings where gaps represent temporary failures

Common Mistakes and Fixes:

Mistake 1: Using mean to fill categorical data

```
# WRONG - you can't average text values
df['Embarked'].fillna(df['Embarked'].mean())

# RIGHT - use mode for categories
df['Embarked'].fillna(df['Embarked'].mode()[0])
```

Mistake 2: Forgetting to assign the result back

```
# WRONG - this doesn't save the changes
df['Age'].fillna(df['Age'].mean())

# RIGHT - reassign to save
df['Age'] = df['Age'].fillna(df['Age'].mean())
```

Mistake 3: Blindly filling without understanding why data is missing

Always ask: Why is this data missing? Is there a pattern? For example, if all missing ages belong to crew members, filling with passenger age average would be wrong.

Choosing the Right Strategy

There's no universal solution. Your choice depends on:

1. **How much data is missing** - Less than 5%? Drop it. More? Fill it.
2. **Why it's missing** - Random gaps vs. systematic patterns
3. **What you're analyzing** - Survival analysis needs accurate ages; general trends might tolerate approximations
4. **Business context** - Consult domain experts when possible

Process:

1. Explore the data first (`df.info()` , `df.describe()`)
2. Understand the domain and context
3. Try different strategies and compare results
4. Document your decision for future reference

Part 2: Group By Operations

What Is Group By?

Imagine you have a stack of customer receipts. You want to know total sales per region. You'd physically sort receipts into piles by region, calculate each pile's total, then write down the results.

Group By does exactly this with data. It splits your DataFrame into groups based on column values, applies a calculation to each group, and combines the results.

This follows a **split-apply-combine** strategy: split data into groups, apply a function to each group, combine results into a new DataFrame.

Why It Matters

Group By transforms raw data into insights. Instead of looking at individual transactions, you see patterns across categories.

You'll need this when:

- Calculating total sales per product category
- Finding average salary by department
- Counting customers per city
- Comparing survival rates across passenger classes (Titanic dataset)

Real-world example: A retail manager wants to know which store generates the most revenue. Group By 'Store' and sum 'Sales' to get the answer instantly.

Detailed Walkthrough

Step 1: Basic Grouping

Group your data by one column and apply an aggregation function.

```
# Calculate average fare by passenger class
avg_fare = df.groupby('Pclass')['Fare'].mean()
```

What's happening:

1. `groupby('Pclass')` splits passengers into groups: First Class, Second Class, Third Class
2. `['Fare']` selects the Fare column from each group
3. `.mean()` calculates the average fare for each group

Common aggregation functions:

- `.sum()` - Add all values
- `.mean()` - Calculate average
- `.count()` - Count number of items
- `.min()` / `.max()` - Find smallest/largest value
- `.median()` - Find middle value

```
# Count passengers per class
df.groupby('Pclass').size()

# Find maximum age per class
df.groupby('Pclass')['Age'].max()
```

Step 2: Grouping by Multiple Columns

You can group by combinations of columns to get more detailed insights.

```
# Average fare by class AND embarkation point
df.groupby(['Pclass', 'Embarked'])['Fare'].mean()
```

This creates groups for each unique combination: (First Class, Southampton), (First Class, Cherbourg), (Second Class, Southampton), etc.

The result has a multi-level index (also called hierarchical index). It looks nested but can be hard to read.

```
# Make it easier to read - flatten the index
result = df.groupby(['Pclass', 'Embarked'])['Fare'].mean()
result = result.reset_index()
```

`reset_index()` converts the index back into regular columns, giving you a clean table.

Step 3: Aggregating Multiple Columns

Apply different calculations to different columns simultaneously.

```
# Calculate multiple statistics at once
df.groupby('Pclass').agg({
    'Fare': 'mean',
    'Age': ['mean', 'min', 'max'],
    'Survived': 'sum'
})
```

What's happening:

- For Fare: calculate mean
- For Age: calculate mean, minimum, AND maximum
- For Survived: count total survivors (sum of 1s)

This gives you a rich summary table in one operation.

Step 4: Custom Aggregation Functions

Sometimes built-in functions aren't enough. You can write your own.

Example: Calculate the range of fares (max - min)

```
# Define custom function
def fare_range(x):
    return x.max() - x.min()

# Use it in groupby
df.groupby('Pclass')['Fare'].agg(fare_range)
```

Your custom function receives each group's data as `x` and returns a single calculated value.

Another example: Calculate survival rate as a percentage

```
def survival_rate(x):
    return (x.sum() / x.count()) * 100

df.groupby('Pclass')['Survived'].agg(survival_rate)
```

Real Examples with Titanic Dataset

Question 1: What was the survival rate by passenger class and gender?

```
survival_by_class_sex = df.groupby(['Pclass', 'Sex'])['Survived'].mean()
survival_by_class_sex = survival_by_class_sex.reset_index()
```

Result shows that First Class females had the highest survival rate, while Third Class males had the lowest.

Question 2: How many passengers were in each class?

```
df.groupby('Pclass').size()
```

or

```
df['Pclass'].value_counts()
```

Question 3: What was the average fare by embarkation point for each class?

```
df.groupby(['Pclass', 'Embarked'])['Fare'].mean().reset_index()
```

This reveals pricing patterns: Southampton might be cheaper than Cherbourg for the same class.

Common Mistakes and Fixes

Mistake 1: Forgetting to specify which column to aggregate

```
# WRONG - too vague
df.groupby('Pclass').mean()

# RIGHT - specify the column
df.groupby('Pclass')['Fare'].mean()
```

Mistake 2: Not resetting index after grouping by multiple columns

```
# Hard to read multi-level index
result = df.groupby(['Pclass', 'Sex'])['Survived'].mean()

# Clean, flat DataFrame
result = df.groupby(['Pclass', 'Sex'])['Survived'].mean().reset_index()
```

Mistake 3: Using the wrong aggregation for your question

- Want to count items? Use `.count()` or `.size()`
- Want totals? Use `.sum()`
- Want averages? Use `.mean()`

Tips:

- Always explore your grouped data with `.head()` after grouping
- Use `reset_index()` to make results easier to work with
- Combine Group By with other Pandas operations for powerful analysis
- Print intermediate results to understand what each step does

---

Key Takeaways

Core Concepts to Remember:

- **Missing data requires investigation before action.** Drop when data is minimal (under 5%), fill when it's substantial. Choose mean/median for numbers, mode for categories.
- **Group By follows split-apply-combine logic.** Split data into groups by column values, apply aggregation functions, combine results into insights.
- **Different questions need different aggregations.** Use sum for totals, mean for averages, count for frequencies, and custom functions for unique calculations.
- **Always reset the index after grouping by multiple columns** to keep your DataFrame clean and readable.
- **Context matters more than code.** Understand your data and business questions before choosing cleaning or grouping strategies.

Mental Model:

Think of missing data as holes in a puzzle. You either remove pieces with holes (drop) or fill the holes with your best guess (fill).

Think of Group By as organizing a messy desk into labeled drawers, then counting or measuring what's in each drawer.

What's Next:

These cleaning and grouping skills are foundational for everything ahead. You'll use them when:

- Preparing data for machine learning models
- Creating visualizations that show patterns
- Writing SQL queries that summarize database tables
- Building dashboards that answer business questions

Master these techniques through practice. Load real datasets, get messy with missing values, and ask questions that require grouping. The more you practice, the more intuitive these operations become.

Session 18: SQL: Thinking in Tables

What You'll Learn

In this session, you've learned to:

- **Explain** why databases exist and what problems they solve over CSV/Excel files
- **Compare** relational and non-relational databases and when to use each
- **Describe** key database concepts — primary keys, foreign keys, schema, and data types
- **Build** a working SQLite3 database in Python and move data between SQL and Pandas

---

1. Why Do We Even Need Databases?

Imagine you're running an online store. You have customers, products, orders, and reviews. You start storing everything in Excel files. At first, it works fine.

Then you get 10,000 customers. Then 500,000 orders. Suddenly:

- The file takes forever to open
- Two people can't edit it at the same time
- Data gets duplicated everywhere
- There's no way to link customers to their orders reliably

This is the exact problem **databases** solve.

A database is an organized, structured collection of data — stored in tables with rows and columns — that allows permanent storage, relationships between data, and access by multiple users simultaneously.

Problem with CSV/Excel	How a Database Fixes It
No relationships between files	Tables link through keys
Data duplication	Store once, reference everywhere
No concurrent access	Multiple users can read/write safely
Poor data integrity	Enforced rules and data types
No security controls	Role-based access and permissions

---

2. Pandas vs. Databases — What's the Difference?

You already know Pandas. So where does a database fit in?

Think of Pandas as your **workbench** — great for chopping, mixing, and analyzing data. A database is your **storage room** — built to hold massive amounts of data permanently and serve many people at once.

Feature	Pandas DataFrame	Database (SQL)
Where data lives	RAM (memory)	Disk (permanent)
Data size	Up to ~5GB comfortably	Terabytes, no problem
Multiple users	No	Yes
Best for	Analysis, cleaning, visualization	Production storage, large scale
Goes away when?	When Python session ends	Never — persists forever

> 💡 *In real-world data workflows, you use **both** — query the database to get what you need, then load it into Pandas for analysis.*

---

3. What Is SQL?

SQL (Structured Query Language) is the language you use to talk to a database. It lets you:

- **Create** tables and define their structure
- **Insert** new records

- **Query** (read) data
- **Update** existing records
- **Delete** records

These four operations together are called **CRUD** — Create, Read, Update, Delete. Almost everything you do in a database is one of these four things.

---

4. Relational vs. Non-Relational Databases

Not all databases work the same way. There are two main families.

Relational Databases (SQL)

Data lives in **tables** — rows and columns, just like a spreadsheet. Tables are connected to each other through **keys**. This is the most common type for structured data.

Examples: **SQLite, PostgreSQL, MySQL, Microsoft SQL Server, Oracle**

Best for: Customer records, orders, financial data — anything with a clear, consistent structure.

Non-Relational Databases (NoSQL)

Data is stored in flexible formats — documents (like JSON), key-value pairs, graphs, or wide-column stores. There's no fixed structure, which makes them great for messy or complex data.

Examples:

- **MongoDB** — stores data as JSON-like documents
- **Redis** — ultra-fast key-value store
- **Cassandra** — handles massive distributed data
- **Neo4j** — stores data as graphs (great for social networks)

Best for: Images, logs, social media posts, nested or unstructured data.

> 🧠 ***Simple rule:*** *If your data fits neatly in rows and columns with relationships — use a relational database. If it's complex, nested, or unstructured — consider NoSQL.*

---

5. The Building Blocks of a Database

Before you create a database, you need to understand three key concepts.

Primary Key

A **primary key** is a column (or set of columns) that **uniquely identifies every row** in a table. No two rows can have the same primary key. It can never be NULL.

Think of it like an Aadhaar number — every person has one, it's unique, and it never changes.

Examples: `customer_id`, `product_id`, `order_id`

These are usually auto-incremented integers — the database assigns 1, 2, 3... automatically.

Foreign Key

A **foreign key** is a column in one table that points to the **primary key of another table**. This is how tables are linked.

Example: An `orders` table has a `customer_id` column. That `customer_id` is the foreign key — it references the `customer_id` primary key in the `customers` table. This tells you *which customer placed which order*.

> *Foreign keys can repeat (one customer can place many orders), but they must always point to a valid entry in the other table.*

Schema

A **schema** is the **blueprint of your database** — it defines what tables exist, what columns each table has, what data types those columns hold, and what the keys and constraints are. Before you store any data, you design the schema first.

Data Types

Every column must have a defined data type. Common ones:

Type	What it stores	Example
`INTEGER`	Whole numbers	Age, quantity
`REAL` / `FLOAT`	Decimal numbers	Price, rating
`TEXT`	Strings	Name, email
`DATE` / `DATETIME`	Date and time values	Order date
`BOOLEAN`	True/False	Is active?
`BLOB`	Binary data	Images, files

Defining data types protects your data — SQL won't let you accidentally store someone's name in an age column.

---

6. SQLite3 — Your First Database in Python

SQLite3 is a lightweight database engine that comes **built into Python** — no installation needed. It's perfect for learning, prototyping, and small to medium projects. The entire database is stored in a single `.db` file on your computer.

The Basic Workflow

Every time you work with SQLite3 in Python, you follow these steps:

```
import sqlite3

# Step 1: Connect to a database (creates the file if it doesn't exist)
conn = sqlite3.connect('mystore.db')

# Step 2: Create a cursor — your tool for sending SQL commands
cursor = conn.cursor()

# Step 3: Execute SQL commands
cursor.execute("YOUR SQL HERE")
```

```
# Step 4: Commit to save changes permanently
conn.commit()

# Step 5: Close the connection when done
conn.close()
```

> ⚠️ *Always `commit()` after inserting or changing data — otherwise your changes won't be saved. And always `close()` your connection to free up resources.*

Creating a Table

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS customers (
        customer_id  INTEGER PRIMARY KEY AUTOINCREMENT,
        name         TEXT NOT NULL,
        email        TEXT UNIQUE NOT NULL,
        age          INTEGER
    )
""")
conn.commit()
```

`CREATE TABLE IF NOT EXISTS` means: "Create this table, but don't crash if it already exists." Always use this — it's safer.

`AUTOINCREMENT` means the database automatically assigns IDs (1, 2, 3...) without you having to specify them.

Inserting Data

```
# Insert one record
cursor.execute("""
    INSERT INTO customers (name, email, age)
    VALUES (?, ?, ?)
""", ("Priya Sharma", "priya@email.com", 28))

# Insert many records at once
customers_data = [
    ("Rohan Verma", "rohan@email.com", 34),
    ("Aisha Khan", "aisha@email.com", 22),
]
cursor.executemany("""
    INSERT INTO customers (name, email, age) VALUES (?, ?, ?)
""", customers_data)

conn.commit()
```

Notice the `?` placeholders — always use these instead of putting values directly in the string. It's safer and prevents a common security issue called SQL injection.

Querying Data

```
cursor.execute("SELECT * FROM customers")

# Fetch all rows at once
all_rows = cursor.fetchall()

# Or fetch one row at a time
one_row = cursor.fetchone()
```

---

7. Moving Data Between SQL and Pandas

This is where the real power comes in. You can go back and forth between SQL and Pandas seamlessly.

SQL → Pandas (most common)

```
import pandas as pd
import sqlite3

conn = sqlite3.connect('mystore.db')

# Load SQL query results directly into a DataFrame
df = pd.read_sql("SELECT * FROM customers", conn)

conn.close()
```

Use this when: You want to filter/query data in the database first, then use Pandas for deeper analysis or visualization.

Pandas → SQL

```
# Save a DataFrame into the database as a table
df.to_sql('customers', conn, if_exists='replace', index=False)
```

`if_exists='replace'` means: if the table already exists, overwrite it. You can also use `'append'` to add rows without deleting existing ones.

Use this when: You've cleaned or transformed data in Pandas and want to save it back into the database permanently.

---

8. Putting It All Together — The Big Picture

Here's how everything connects in a real workflow:

```
Raw Data (CSV / API / JSON)
        ↓
  Load into Pandas (clean & transform)
```

```
      ↓
Save to SQLite Database (.db file)
      ↓
Query with SQL (filter, sort, join)
      ↓
Load results into Pandas (analyze & visualize)
```

This pattern — moving data between storage and analysis tools — is the foundation of what **data engineers** and **data analysts** do every day. It's called an **ETL pipeline** (Extract, Transform, Load).

Key Takeaways

- **Databases solve real problems** that CSV and Excel files can't — scale, relationships, concurrent access, and permanence.
- **Pandas lives in memory; databases live on disk.** Use SQL for storage and scale, Pandas for analysis and visualization.
- A **primary key** uniquely identifies each row. A **foreign key** links tables together. A **schema** is the blueprint of your entire database.
- **SQLite3 is built into Python** — it's the easiest way to start practicing with real databases, no setup required.
- `pd.read_sql()` and `df.to_sql()` are your bridges between SQL and Pandas — learn them well.

> ***Mental model:*** *Think of a database as a permanent, well-organized warehouse. SQL is the language you use to talk to the warehouse manager. Pandas is your personal workstation where you actually analyze what you've pulled out.*

Coming up next: You'll learn how to **JOIN tables together** — combining data from multiple tables into one result, which is where relational databases really shine.

Session 19: SQL: Core Querying

What You'll Learn

In this session, you've learned to:

- **Write** SQL queries to retrieve specific data from a database table
- **Filter and sort** results using WHERE, ORDER BY, and LIMIT
- **Apply** advanced filters using IN, BETWEEN, and NULL handling
- **Map** SQL commands to their Pandas equivalents so you can use both confidently

---

1. Your First Query — SELECT

Last session, you created a database and inserted data into it. Now it's time to actually *get* that data out.

The command for this is `SELECT`. It's the most used command in all of SQL.

```sql
-- Get everything from the Titanic table
SELECT * FROM Titanic
```

The `*` means "all columns." Think of it as saying: *"Show me everything."*

Want only specific columns? Just name them:

```sql
-- Get only name and age
SELECT name, age FROM Titanic
```

Plain English: "Go to the Titanic table. Bring me only the name and age columns."

> 🔑 *SQL keywords like `SELECT` and `FROM` are case-insensitive, but writing them in **UPPERCASE** is the convention. It makes queries easier to scan at a glance.*

Pandas equivalent:

```python
df[['name', 'age']]
```

---

2. Filtering Rows — WHERE

`SELECT *` gives you everything. But what if you only want female passengers? That's where `WHERE` comes in — it filters rows based on a condition.

```sql
SELECT * FROM Titanic WHERE sex = 'female'
```

> ⚠️ ***Common mistake:** In Python you write `==` for comparison. In SQL, it's just `=`. This trips up almost everyone at first.*

You can combine multiple conditions using `AND` and `OR`:

```sql
-- Female passengers in first class only
SELECT * FROM Titanic WHERE sex = 'female' AND Pclass = 1

-- Passengers in first class OR second class
SELECT * FROM Titanic WHERE Pclass = 1 OR Pclass = 2
```

All supported operators:

Operator	Meaning
`=`	Equal to
`<>`	Not equal to
`>` / `<`	Greater / Less than
`>=` / `<=`	Greater or equal / Less or equal
`AND` / `OR`	Combine multiple conditions

Pandas equivalent:

```python
df[df['sex'] == 'female']
```

```
# Multiple conditions
df[(df['sex'] == 'female') & (df['Pclass'] == 1)]
```

---

3. Sorting Results — ORDER BY

Once you've filtered your data, you'll often want it in a specific order. Use `ORDER BY` to sort.

```
-- Youngest passengers first
SELECT * FROM Titanic ORDER BY age ASC

-- Oldest passengers first
SELECT * FROM Titanic ORDER BY age DESC
```

`ASC` = ascending (low → high). `DESC` = descending (high → low). If you don't specify, SQL defaults to `ASC`.

You can sort by **multiple columns** too. SQL sorts by the first column, then uses the second to break ties:

```
-- Sort by class (1st, 2nd, 3rd), then by fare highest-to-lowest within each class
SELECT * FROM Titanic ORDER BY Pclass ASC, Fare DESC
```

Pandas equivalent:

```
df.sort_values('age', ascending=True)

# Multiple columns
df.sort_values(['Pclass', 'Fare'], ascending=[True, False])
```

---

4. Limiting Results — LIMIT

A real database might have millions of rows. You don't want to load all of them just to check what the data looks like. Use `LIMIT` to cap how many rows come back.

```
-- Show only the first 5 rows
SELECT * FROM Titanic LIMIT 5
```

You can also **skip rows** using `OFFSET` — useful when building pagination:

```
-- Skip the first 10 rows, then return the next 5
SELECT * FROM Titanic LIMIT 5 OFFSET 10
```

> 💡 *Always use `LIMIT` when exploring an unfamiliar table. Never just fire `SELECT *` on a production database — you might accidentally pull back 10 million rows.*

Pandas equivalent:

```
df.head(5)
```

---

5. Removing Duplicates — DISTINCT

Want to know what unique passenger classes exist in the data? Use `DISTINCT` — it strips out all repeated values and returns only unique ones.

```
SELECT DISTINCT Pclass FROM Titanic
```

Result: `1`, `2`, `3` — no repeats.

You can also use it across **multiple columns** to find unique combinations:

```
-- What unique combinations of class and gender exist?
SELECT DISTINCT Pclass, sex FROM Titanic
```

Pandas equivalent:

```
df['Pclass'].unique()

# Multiple columns
df[['Pclass', 'sex']].drop_duplicates()
```

---

6. Renaming Columns — AS (Alias)

Sometimes column names are unclear, or you're doing a calculation and want to label the result. Use `AS` to give a column a temporary, readable name.

```
-- Rename for clarity
SELECT sex AS gender, age AS passenger_age FROM Titanic
```

Aliases are especially powerful with calculations:

```
-- Add 20% tax to every fare and label the result clearly
SELECT fare, fare * 1.2 AS fare_with_tax FROM Titanic
```

The alias only lives in your query result — it doesn't change the actual table at all.

---

7. Advanced Filtering

The basic `WHERE` clause with `=` and `AND` / `OR` gets you far. But SQL has three more powerful tools for filtering that keep your queries clean and readable.

The IN Operator

Instead of chaining multiple `OR` conditions, use `IN` to match against a **list of values**:

```sql
-- Without IN (messy)
SELECT * FROM Titanic WHERE embarked = 'S' OR embarked = 'C'

-- With IN (clean)
SELECT * FROM Titanic WHERE embarked IN ('S', 'C')
```

Both do the same thing — but `IN` is much easier to read, especially with longer lists.

Pandas equivalent:

```python
df[df['embarked'].isin(['S', 'C'])]
```

The BETWEEN Operator

To filter rows within a **numeric or date range**, use `BETWEEN`:

```sql
-- Passengers aged 20 to 30
SELECT * FROM Titanic WHERE age BETWEEN 20 AND 30
```

> 🔑 *Both endpoints are **inclusive** — so passengers who are exactly 20 or exactly 30 are included in the results.*

Pandas equivalent:

```python
df[df['age'].between(20, 30)]
```

Handling NULL Values

In SQL, missing data isn't stored as `0` or an empty string — it's stored as `NULL`. And `NULL` plays by different rules.

```sql
-- Find passengers with no recorded age
SELECT * FROM Titanic WHERE age IS NULL

-- Find passengers who DO have an age
SELECT * FROM Titanic WHERE age IS NOT NULL
```

> ⚠️ ***Critical mistake to avoid:*** *You cannot write `WHERE age = NULL`. It will silently return zero rows — no error, just wrong results. Always use `IS NULL` or `IS NOT NULL`.*

Pandas equivalent:

```python
df[df['age'].isna()]       # IS NULL
df[df['age'].notna()]      # IS NOT NULL
```

---

8. Inspecting Your Database — PRAGMA (SQLite only)

When working with SQLite3, there's a special command called `PRAGMA` that helps you explore the database itself — not the data inside, but the *structure*.

```
-- List all databases connected
PRAGMA database_list

-- See the schema (columns, types, keys) of a table
PRAGMA table_info(Titanic)
```

Think of `PRAGMA` as your "peek behind the curtain" tool. It's specific to SQLite and won't work in PostgreSQL or MySQL, but it's very handy while learning.

---

9. SQL Queries Inside Python

You already know how to connect to SQLite3 from the last lecture. Here's how you run SQL queries and pull results into Pandas:

```
import sqlite3
import pandas as pd

conn = sqlite3.connect('titanic.db')

# Run a SQL query and load results directly into a DataFrame
df = pd.read_sql("SELECT * FROM Titanic WHERE sex = 'female'", conn)

print(df.head())

conn.close()  # Always close when done!
```

This is the core workflow you'll use constantly:

1. Filter in SQL — keep only the rows you need
2. Load into Pandas — analyze, visualize, transform

This way, you're never loading a 10-million-row table into memory when you only need 50,000 rows.

---

10. SQL vs. Pandas — Side-by-Side Reference

What you want to do	SQL	Pandas
Get all data	`SELECT * FROM table`	`df`
Select columns	`SELECT col1, col2`	`df[['col1', 'col2']]`
Filter rows	`WHERE col = value`	`df[df['col'] == value]`
Multiple filters	`AND` / `OR`	`&` / `|`
Sort	`ORDER BY col DESC`	`.sort_values('col', ascending=False)`
Limit rows	`LIMIT 10`	`.head(10)`

Unique values	`SELECT DISTINCT col`	`.unique()` / `.drop_duplicates()`
Range filter	`BETWEEN 20 AND 30`	`.between(20, 30)`
List filter	`IN ('S', 'C')`	`.isin(['S', 'C'])`
Missing data	`IS NULL`	`.isna()`

Key Takeaways

- **SELECT + FROM** starts every query. Add `WHERE` to filter, `ORDER BY` to sort, and `LIMIT` to cap results.
- **Use `IN` instead of chaining `OR`** — it's cleaner and easier to read when matching multiple values.
- **`BETWEEN` is inclusive** — both the start and end values are included in the results.
- **NULL is not zero** — always use `IS NULL` or `IS NOT NULL`. Using `= NULL` gives wrong results silently.
- **SQL and Pandas are a team** — filter data at the database level with SQL, then load only what you need into Pandas for analysis.

> ***Mental model:*** *Think of a SQL query like building an instruction: "FROM this table → WHERE these conditions are true → SELECT these columns → ORDER BY this → LIMIT to this many rows." Read it in that order mentally, even though you write SELECT first.*

Coming up next: You'll learn how to combine data from **multiple tables using JOINs** — one of SQL's most powerful features, and the reason relational databases are so valuable.

Session 20: SQL: Joining Data

What You'll Learn

In this session, you've learned to:

- **Explain** why data is split across tables and what normalization means
- **Compare** the four types of SQL joins and when to use each
- **Write** SQL join queries using aliases and multiple table joins
- **Replicate** SQL joins in Pandas using `pd.merge()`

---

1. Why Is Data Split Across Tables?

Imagine a shopping app where every order row also stores the customer's name, email, and phone number. Now the customer changes their email — you'd have to update it in **hundreds of rows**. Miss one? Your data is broken.

The fix is **normalization** — splitting data into separate, focused tables to avoid repetition.

Example:

orders table (bad — redundant)
order_id, customer_name, customer_email, product, amount

Split into two cleaner tables:

customers	orders
customer_id, name, email	order_id, customer_id, product, amount

Now the email lives in **one place only**. Update it once — done.

Two key terms you'll use constantly:

- **Primary Key** — a unique ID for each row (e.g., `customer_id` in the customers table)
- **Foreign Key** — a column that references another table's primary key (e.g., `customer_id` in the orders table)

> 💡 *This design is maintained by **data architects** — the people who plan how databases are structured before anyone writes a single query.*

---

2. What Is a JOIN?

Now that data lives in separate tables, you need a way to bring it back together. That's what a **JOIN** does.

A JOIN combines rows from two tables based on a **common column** — usually the primary key and foreign key.

Basic syntax:

```
SELECT columns
FROM table1
JOIN table2
ON table1.common_column = table2.common_column;
```

Using aliases keeps things clean and short:

```
SELECT s.name, g.grade
FROM students AS s
JOIN grades AS g
ON s.student_id = g.student_id;
```

> *If two tables have a column with the **same name**, you must use an alias to tell SQL which one you mean. Without it, SQL gets confused and throws an error.*

---

3. The Four Types of Joins

We'll use three sample tables throughout: **Students**, **Classes**, and **Grades**.

🔷 Inner Join

Returns only the rows that have a **match in both tables**.

```
SELECT s.name, c.class_name
FROM students AS s
```

```
JOIN classes AS c
ON s.class_id = c.class_id;
```

If a student has no matching class, they're **left out** of the result entirely.

Use this when: You only want complete, matched data from both sides.

🔷 Left Join

Returns **all rows from the left table**, plus matching rows from the right. If there's no match, the right table columns show `NULL`.

```
SELECT s.name, c.class_name
FROM students AS s
LEFT JOIN classes AS c
ON s.class_id = c.class_id;
```

Even students with no class assigned will appear — their `class_name` will just be `NULL`.

Use this when: You want to keep all records from the left table, even if some have no match.

🔷 Right Join

The mirror of a Left Join. Returns **all rows from the right table**, plus matching rows from the left. Unmatched left rows show `NULL`.

Important caveat: SQLite does **not support Right Joins natively**. You can work around this by swapping the table order and using a Left Join instead — or use Pandas.

Use this when: You want to keep all records from the right table, even without matches.

🔷 Full Outer Join

Returns **all rows from both tables**. Where there's no match on either side, you get `NULL` for the missing columns.

Note: Like Right Join, Full Outer Join also has **limited support in SQLite**. Pandas handles it easily.

Use this when: You want a complete picture of both tables, matched or not.

Quick Comparison

Join Type	Keeps Left	Keeps Right	NULLs appear
Inner Join	Only matched	Only matched	Never
Left Join	All rows	Only matched	Right side
Right Join	Only matched	All rows	Left side
Full Outer Join	All rows	All rows	Both sides

---

4. Joining More Than Two Tables

You can chain multiple joins together. Just add another `JOIN` and specify the common column for each pair.

```sql
SELECT s.name, c.class_name, g.grade
FROM students AS s
LEFT JOIN classes AS c ON s.class_id = c.class_id
LEFT JOIN grades AS g ON s.student_id = g.student_id;
```

Each join links to a new table using its own `ON` condition. This lets you consolidate information spread across many related tables in one query.

> ***Practical tip:*** *Avoid joining too many tables at once. The more tables you join, the slower and harder to read your query becomes. Keep it manageable.*

---

5. Doing the Same Thing in Pandas

SQL joins have a direct equivalent in Pandas — `pd.merge()`.

```python
pd.merge(left_df, right_df, on='common_column', how='inner')
```

Just swap `how=` to change the join type:

SQL	Pandas `how=`
INNER JOIN	`'inner'`
LEFT JOIN	`'left'`
RIGHT JOIN	`'right'`
FULL OUTER JOIN	`'outer'`

For multiple table joins, chain merges one step at a time:

```python
step1 = pd.merge(students, classes, on='class_id', how='left')
result = pd.merge(step1, grades, on='student_id', how='left')
```

> *Column names must match exactly across both DataFrames before merging. If they don't, rename them first using* `.rename()`.

Pandas is also your best workaround for Right Joins and Full Outer Joins when using SQLite.

Key Takeaways

- **Normalization** splits data into separate tables to remove redundancy — reducing storage costs and making updates easier. Tables are linked using **primary keys** and **foreign keys**.
- **JOINs** bring that split data back together using a shared common column.

- **Inner Join** = matched rows only. **Left Join** = all left rows + matches. **Right Join** = all right rows + matches. **Full Outer Join** = everything from both sides.
- **SQLite has limited support** for Right and Full Outer Joins — use Left Join with swapped tables, or use Pandas instead.
- `pd.merge()` in Pandas mirrors SQL joins exactly — just chain merges for multiple tables.
- **Think of a JOIN as a zipper** — it connects two separate tables using a shared column as the common tooth.

Next up: Subqueries — running a query inside another query for more advanced filtering and joining.

Session 21: Matplotlib: Basic Plotting

In this session, you've learned to:

- Explain why data visualization is essential for data analysis and machine learning workflows
- Create five fundamental plot types using Matplotlib: line plots, scatter plots, bar charts, histograms, and pie charts
- Customize plots with labels, titles, colors, and markers to communicate insights effectively
- Apply the appropriate plot type based on your data and the story you want to tell

---

Detailed Explanation

What Is Data Visualization?

Imagine you have a spreadsheet showing daily temperatures for an entire year. You could scan through 365 rows of numbers, or you could look at a single graph that instantly shows you the pattern: hot in summer, cold in winter, with smooth transitions between seasons.

Data visualization is the process of representing data graphically so your brain can process patterns, trends, and insights much faster than reading raw numbers in tables.

Matplotlib is Python's most popular library for creating these visual representations. Think of it as your toolkit for turning boring spreadsheets into clear, informative graphs. Just like you learned to use Pandas for data manipulation, Matplotlib is the natural next step for communicating what that data actually means.

Why Visualization Matters

Your brain processes images 60,000 times faster than text. This isn't just interesting trivia—it's why visualization is critical in data science.

You'll need visualization when:

- **Exploring new datasets**: Quickly spot outliers, missing values, or unusual patterns that would take hours to find in tables
- **Training machine learning models**: Watch your model improve by plotting accuracy curves and loss over epochs
- **Presenting findings**: Stakeholders understand "sales dropped 30% in Q3" much better when they see the downward trend in a graph
- **Debugging**: A scatter plot can reveal that your data has two distinct clusters, suggesting you need different models for each group

Consider a simple example: You have data showing study hours versus exam marks for 50 students. In a table, you'd see 50 rows of numbers. In a scatter plot, you see the relationship instantly—more study hours clearly correlate with higher marks. That insight drives better decisions.

Getting Started: The Matplotlib Workflow

Before creating any visualization, you need to import Matplotlib. The standard practice is:

```
import matplotlib.pyplot as plt
```

This imports the pyplot module and gives it the nickname "plt" so you don't have to type the full name repeatedly.

The basic workflow for any plot follows three steps:

1. **Prepare your data** (create x and y values)
2. **Create the plot** (tell Matplotlib what to draw)
3. **Show the plot** (display it on screen)

Here's the simplest possible example:

```
import matplotlib.pyplot as plt

# Step 1: Prepare data
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# Step 2: Create plot
plt.plot(x, y)

# Step 3: Show plot
plt.show()
```

This creates a line plot showing a perfect linear relationship. But there's a problem—what does this graph represent? Without labels, no one knows what they're looking at.

Always enhance your plots with:

- **X-axis label**: `plt.xlabel("Hours Studied")`
- **Y-axis label**: `plt.ylabel("Exam Marks")`
- **Title**: `plt.title("Study Hours vs Exam Performance")`

These three lines transform your plot from meaningless to meaningful. Suddenly, viewers understand exactly what they're seeing.

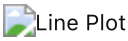
Line Plots: Showing Trends Over Time

What they are: Line plots connect individual data points with straight lines, creating a continuous visual flow. They're your go-to choice when you want to show how something changes over time or across a continuous range.

When to use them: Temperature changes throughout the day, stock prices over months, model accuracy improving across training epochs, population growth over decades.

Basic example with customization:

```
hours = [1, 2, 3, 4, 5, 6]
marks = [45, 55, 60, 70, 80, 90]

plt.plot(hours, marks, color='blue', linestyle='-', marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Marks Obtained")
plt.title("Impact of Study Hours on Performance")
plt.show()
```

 Line Plot

What's happening here?

- `color='blue'` makes the line blue (you can use 'red', 'green', or hex codes like '#FF5733')
- `linestyle='-'` creates a solid line (use '--' for dashed, ':' for dotted)
- `marker='o'` adds circular markers at each data point (use '+', 'x', or 's' for squares)

Common mistake: Forgetting `plt.show()`. Your plot won't appear without this command. Think of it as pressing "Enter" to submit your work.

Tip: If you're plotting multiple lines on the same graph (like comparing performance of different students), use different colors for each line so viewers can distinguish them easily.

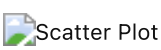
Scatter Plots: Revealing Relationships and Clusters

What they are: Scatter plots show individual data points as dots without connecting them. Each dot represents one observation, making them perfect for large datasets where you want to see the overall pattern without implying continuous connections.

When to use them: Examining relationships between two variables (height vs. weight), identifying clusters in customer data, detecting outliers in sensor readings, or visualizing classification results in machine learning.

Basic example:

```
hours = [1, 2, 2, 3, 3, 3, 4, 4, 5, 5, 6]
marks = [45, 50, 55, 58, 62, 65, 70, 72, 78, 82, 88]

plt.scatter(hours, marks, color='red', marker='x')
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Study Hours vs Marks (Individual Students)")
plt.show()
```

 Scatter Plot

The key difference from line plots: Notice we're using `plt.scatter()` instead of `plt.plot()`. Scatter doesn't draw lines between points, so you can see the natural spread and clustering of your data.

Real-world application: In machine learning clustering algorithms like K-means, scatter plots help you visualize how the algorithm groups similar data points together. You can color-code different clusters to see if the algorithm is working correctly.

Customization tip: Use different colors or marker shapes to represent different categories. For example, red dots for one student group and blue dots for another.

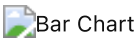
Bar Charts: Comparing Categories

What they are: Bar charts use rectangular bars to compare quantities across different categories. The height (or length) of each bar represents the value for that category.

When to use them: Comparing sales across different regions, showing accuracy of different machine learning models, displaying student counts in different grade categories, or ranking products by revenue.

Basic example:

```
students = ['Group A', 'Group B', 'Group C', 'Group D']
counts = [25, 40, 35, 30]

plt.bar(students, counts, color='green')
plt.xlabel("Student Groups")
plt.ylabel("Number of Students")
plt.title("Student Distribution Across Groups")
plt.show()
```

 Bar Chart

What's happening: `plt.bar()` creates vertical bars. Each bar's height represents the count for that group. At a glance, you can see Group B has the most students.

Horizontal bars: Sometimes horizontal bars are clearer, especially with long category names:

```
plt.barh(students, counts, color='orange')
plt.xlabel("Number of Students")
plt.ylabel("Student Groups")
plt.title("Student Distribution (Horizontal)")
plt.show()
```

Notice we switched to `plt.barh()` (bar horizontal) and swapped the x and y labels accordingly.

Machine learning application: After training multiple models, you can compare their accuracy with a bar chart. Instantly see which model performed best without scanning through numbers.

Histograms: Understanding Data Distribution

What they are: Histograms group numerical data into ranges (called "bins") and count how many values fall into each range. They show you the shape and spread of your data.

This is crucial: Unlike bar charts that compare categories, histograms show the frequency distribution of a single continuous variable.

When to use them: Understanding test score distributions, examining age ranges in a dataset, detecting if data is normally distributed, identifying skewness or outliers in house prices.

Basic example:

```
marks = [45, 50, 52, 55, 58, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90]

plt.hist(marks, bins=5, color='purple', edgecolor='black')
plt.xlabel("Marks Range")
plt.ylabel("Number of Students")
plt.title("Distribution of Student Marks")
plt.show()
```

 Histogram

What's happening with bins?

The `bins=5` parameter tells Matplotlib to group the data into 5 ranges. If your marks range from 45 to 90, it might create bins like: 45-54, 55-64, 65-74, 75-84, 85-90. Then it counts how many marks fall into each bin.

Choosing the right number of bins: Too few bins (like 2) hide detail. Too many bins (like 50 for only 18 data points) create noise. Start with 5-10 bins and adjust based on what reveals the pattern best.

Common confusion: "Isn't this just a bar chart?" No. Bar charts compare discrete categories (apples vs. oranges). Histograms show continuous data distribution (all the different temperatures recorded).

Tip: Adding `edgecolor='black'` creates borders around bars, making them easier to distinguish when they're close together.

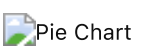
Pie Charts: Showing Parts of a Whole

What they are: Pie charts divide a circle into slices, where each slice represents a proportion of the total. The bigger the slice, the larger that category's share.

When to use them: Market share percentages, budget allocation, survey response breakdowns, or any situation where you want to show "what percentage of the total does each part represent?"

Basic example:

```
categories = ['Python', 'Java', 'JavaScript', 'C++']
usage = [40, 25, 20, 15]

plt.pie(usage, labels=categories, autopct='%1.1f%%')
plt.title("Programming Language Usage")
plt.show()
```

 Pie Chart

What's happening:

- `usage` contains the values (they don't need to add up to 100—Matplotlib calculates percentages automatically)
- `labels=categories` tells Matplotlib what to name each slice
- `autopct='%1.1f%%'` displays percentages on each slice (the '%1.1f' means show one decimal place)

Exploding slices for emphasis:

Sometimes you want to highlight a particular slice by pulling it away from the center:

```
explode = [0.1, 0, 0, 0]  # Pull out the first slice by 10%

plt.pie(usage, labels=categories, autopct='%1.1f%%', explode=explode)
plt.title("Programming Language Usage (Python Highlighted)")
plt.show()
```

The `explode` list has one value per slice. 0 means normal, 0.1 means pull out by 10%.

Caution: Pie charts work best with 3-6 categories. More than that becomes cluttered and hard to read. Also, humans are better at comparing bar heights than pie slice angles, so when precision matters, consider a bar chart instead.

Putting It All Together: Choosing the Right Plot

Each plot type tells a different story. Here's your decision guide:

Use line plots when: You have continuous data over time or a sequence (temperature throughout the day, model loss decreasing over epochs)

Use scatter plots when: You want to see relationships between two variables or identify clusters (height vs. weight, exam scores vs. study hours)

Use bar charts when: Comparing discrete categories side by side (revenue by region, accuracy of different models)

Use histograms when: You need to understand the distribution of a single variable (age distribution, price ranges, test score spread)

Use pie charts when: Showing composition—how parts make up a whole (market share, budget allocation)

Common beginner mistake: Using a line plot when you should use a scatter plot. If your data points aren't naturally connected in sequence, don't force a line through them. Let the scatter plot show the natural spread.

Connection to Machine Learning

In your machine learning journey, you'll constantly use these plots:

During training: Line plots showing loss decreasing over epochs tell you if your model is learning. If the line stays flat or goes up, something's wrong.

Model evaluation: Confusion matrices (visualized as heatmaps) show where your classifier makes mistakes. Bar charts compare precision, recall, and F1 scores across different models.

Data exploration: Before building any model, scatter plots reveal relationships between features. Histograms show if your data is skewed or has outliers that need handling.

Communicating results: Stakeholders don't want to see code or raw metrics. They want clear visualizations that tell the story of what your model discovered.

Key Takeaways

- **Data visualization transforms raw numbers into insights your brain can process instantly**—always visualize before drawing conclusions from data
- **The core workflow is simple**: import pyplot, prepare data, create plot with appropriate function, add labels/title, show plot
- **Choose your plot based on your story**: line plots for trends, scatter plots for relationships, bar charts for comparisons, histograms for distributions, pie charts for composition
- **Labels and titles are mandatory, not optional**—a graph without context is meaningless to viewers
- **Customization matters**: colors, markers, and line styles aren't decoration—they help viewers distinguish between different data series and understand patterns faster

Mental model: Think of Matplotlib as your visual translator. You feed it data (the language of numbers), and it produces graphs (the language of patterns). Your job is choosing which translation method (plot type) best communicates your message.

What's next: These five plot types form your foundation. Advanced topics include subplots (multiple graphs in one figure), 3D plotting, heatmaps for correlation matrices, and interactive visualizations. But master these basics first—they'll handle 90% of your visualization needs in data science and machine learning projects.

Session 22: Advanced Storytelling with Data

What You Learned

In this session, you learned to:

- **Explain** what EDA is and why every data project starts here
- **Choose** the right plot for the right question
- **Read** what a visualization is actually telling you — not just generate it
- **Identify** outliers using box plots and the IQR method

The Big Idea — Why Visualize at All?

You have data. You could just run numbers — mean, count, sum. So why bother plotting?

Here is why:

> *Both datasets below have the **same mean, same variance, same correlation.***

```
Dataset A               Dataset B
x     y                 x     y
10    8.04              10    9.14
8     6.95              8     8.14
13    7.58              13    8.74
...                     ...
```

But when plotted, they look **completely different.** One is a straight line. One is a curve. One has an outlier that breaks everything.

This is called Anscombe's Quartet — and it is the single best argument for why visualization is not optional. It is essential.

The Dataset — IRIS

The **IRIS dataset** was used throughout this session. Here is what it contains:

```
import seaborn as sns
iris = sns.load_dataset("iris")
print(iris.head())
```

```
   sepal_length  sepal_width  petal_length  petal_width    species
0           5.1          3.5           1.4          0.2     setosa
1           4.9          3.0           1.4          0.2     setosa
2           6.3          3.3           4.7          1.6  versicolor
3           7.2          3.0           5.8          1.8   virginica
```

150 flowers. 3 species. 4 measurements each.

The question EDA helped answer: Can we tell the three species apart just by looking at their measurements? If yes visually — a model can do it mathematically.

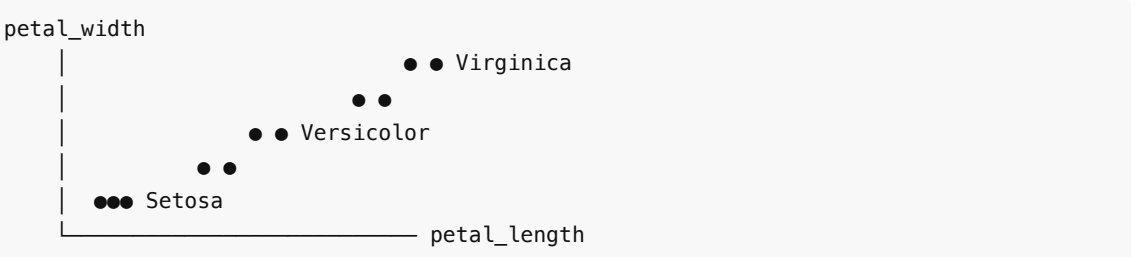
Visualization 1 — Scatter Plot

The question it answers: Is there a relationship between two variables?

Real scenario: You are analyzing student data — does study time relate to marks scored?

```
sns.scatterplot(data=iris,
                x="petal_length",
                y="petal_width",
                hue="species")
plt.title("Can we separate species by petal size?")
plt.show()
```

How to read it:



- **Clusters = separable.** Setosa sits completely alone. That is a strong signal.
- **Overlap = harder to separate.** Versicolor and Virginica overlap slightly.

- The imaginary line you draw between clusters? That is a **decision boundary** — you will use this concept in every classification model you ever build.

You will use scatter plots when:

- Checking if two variables move together
- Spotting clusters or groups in your data
- Understanding class separability before building a classifier

---

Visualization 2 — Pair Plot

The question it answers: Which pair of features best separates my data?

The problem with scatter plots: With 4 features, you would need to manually create 6 scatter plots. Tedious.

```
sns.pairplot(iris, hue="species")
plt.show()
```

One line. Every combination. Done.

How to read it:

```
              sepal_len   sepal_wid   petal_len   petal_wid
sepal_len   [ histogram ] [scatter ] [scatter ] [scatter ]
sepal_wid   [ scatter   ] [histogram] [scatter ] [scatter ]
petal_len   [ scatter   ] [scatter ] [histogram] [scatter ]
petal_wid   [ scatter   ] [scatter ] [scatter ] [histogram]
```

- **Diagonal** = distribution of each feature on its own
- **Off-diagonal** = scatter plot of every pair

What the session revealed: The petal_length vs petal_width cell showed the cleanest separation. That is a real insight — petal measurements are more useful than sepal measurements for distinguishing species.

You will use pair plots when: You just received a new dataset and need to quickly identify which features are worth investigating further.

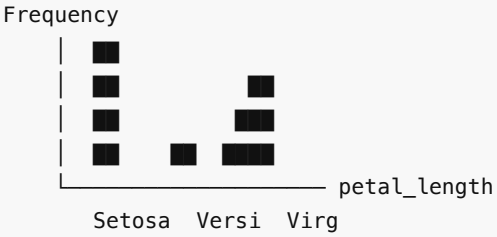
Visualization 3 — Histogram

The question it answers: How is a single variable distributed?

Real scenario: Your professor shares exam scores. You do not want to read 200 individual scores — you want to know: where did most students land?

```
sns.histplot(data=iris, x="petal_length",
             hue="species", bins=20, kde=True)
plt.title("Where does each species' petal length fall?")
plt.show()
```

How to read it:



- **Separate peaks** = species have different typical values — easier to distinguish
- **Overlapping peaks** = hard to tell apart using this feature alone
- The **KDE line** (the smooth curve) shows the overall shape without the noise of individual bins

Bin size matters:

```
# Too few bins — you lose detail
sns.histplot(data=iris, x="petal_length", bins=5)

# Too many bins — noisy and hard to read
sns.histplot(data=iris, x="petal_length", bins=100)

# Just right
sns.histplot(data=iris, x="petal_length", bins=20)
```

You will use histograms when: You want to understand the shape of a variable — is it skewed? Is it spread out? Does it have one peak or two?

Visualization 4 — Box Plot + Outlier Detection

The question it answers: What is the spread of my data — and are there any extreme values hiding in it?

Real scenario: A student scores 98 in a class where everyone else scored between 40–70. That 98 is an outlier — and a box plot would immediately flag it.

The Four Numbers You Need

Using exam scores: **12, 15, 18, 22, 25, 30, 35, 40, 80**

```
Q1  = 18     ← 25th percentile (lower quarter)
Q2  = 25     ← Median (middle value)
Q3  = 35     ← 75th percentile (upper quarter)
IQR = Q3 − Q1 = 35 − 18 = 17
```

The Outlier Rule

```
Lower Fence = Q1 − 1.5 × IQR = 18 − 25.5 = −7.5
Upper Fence = Q3 + 1.5 × IQR = 35 + 25.5 = 60.5
```

Score of **80** is above 60.5 → **Outlier.**

What a Box Plot Looks Like

```
      o   ← Outlier (80)
      |
     ─┴─  ← Upper fence (60.5)
   ┌─────┐
   │     │ ← Q3 (35)
   │  ───│ ← Median (25)
   │     │ ← Q1 (18)
   └─────┘
     ─┬─  ← Lower fence (-7.5)
```

```
sns.boxplot(data=iris, x="species", y="petal_length")
plt.title("Spread of Petal Length per Species")
plt.show()
```

You will use box plots when:

- Comparing distributions across multiple groups side by side
- Hunting for outliers before feeding data into a model
- Checking if one group is more spread out than another

Visualization 5 — Violin Plot

The question it answers: Same as a box plot — but what does the full shape look like?

A box plot tells you where the middle 50% lives. It does not tell you if data is piled up at the top of that range or the bottom.

A violin plot shows you exactly that.

```
sns.violinplot(data=iris, x="species", y="petal_length")
plt.title("Full Distribution Shape by Species")
plt.show()
```

Box plot vs Violin plot:

```
Box Plot          Violin Plot
   ─┬─               ╭─╮
  ┌─┴─┐             ╭╯ ╰╮
  │ ──│     vs      │   │
  └─┬─┘             ╰╮ ╭╯
    ┴                ╰─╯

 Tells you WHERE    Tells you WHERE
 the middle is       + the SHAPE
```

- **Wide section** = many data points at that value
- **Narrow section** = fewer data points
- **Two bumps** = two common value ranges (bimodal) — box plot would completely miss this

You will use violin plots when: The shape of distribution matters, not just the summary statistics.

Interactive Plots — Plotly

Static plots are for reports. Interactive plots are for exploration.

```
pip3 install plotly
```

```
import plotly.express as px

fig = px.scatter(iris,
                 x="petal_length",
                 y="petal_width",
                 color="species",
                 hover_data=["sepal_length", "sepal_width"],
                 title="Interactive IRIS Scatter")
fig.show()
```

With Plotly, you can:

- **Hover** over any dot to see exact values
- **Zoom** into dense clusters
- **Click** species in the legend to hide/show them

You will use Plotly when: You are presenting to someone who needs to explore the data themselves, or when exact values matter.

Picking the Right Library

Task	Use
Quick EDA, statistical plots	**Seaborn**
Full control, publication plots	**Matplotlib**
Interactive, web-ready charts	**Plotly**

Saving your plot:

```
plt.savefig("plot.png", dpi=150)   # For presentations
plt.savefig("plot.svg")            # For reports (scales without blur)
```

Key Takeaways

- **EDA is not optional.** Numbers alone lie. Anscombe's Quartet proved that two datasets can share the same statistics and look nothing alike. Always plot first.
- **Every plot answers one specific question.** Use scatter plots for relationships, histograms for distributions, box plots for spread and outliers, violin plots for shape, pair plots for all-at-once exploration.

- **Reading a plot is the real skill.** Generating one takes one line of code. Knowing what it is telling you — that is what makes a data scientist.
- **Mental model for box plots:** The box is where the middle 50% of your data lives. Anything beyond 1.5 × IQR from that box is unusual enough to investigate.

Session 23: Python Masterclass is not a part of Evaluation

Session 24: The EDA Checklist

Inspect Data Quickly

Core Concept: Initial Reconnaissance

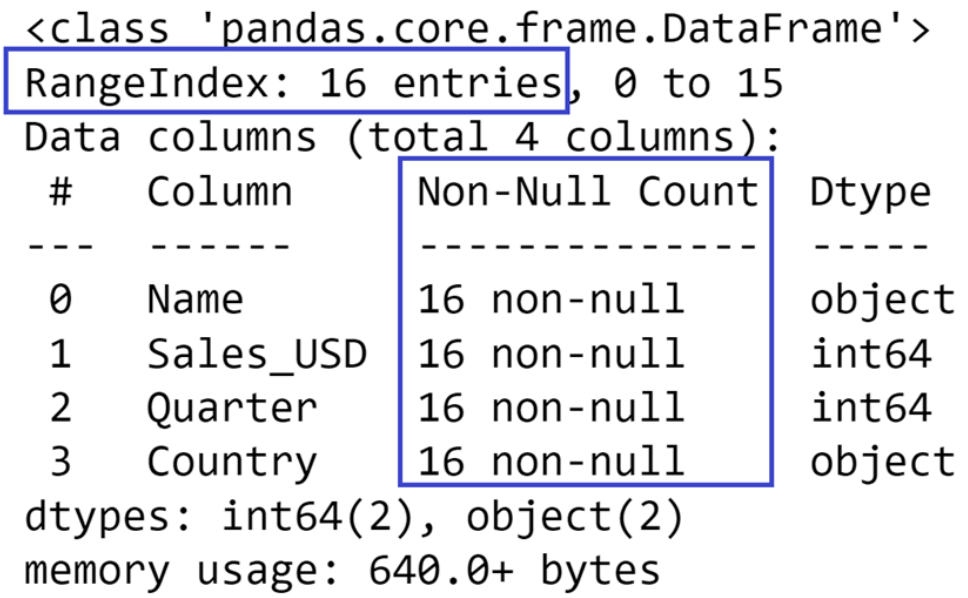
Data inspection is the mandatory first step of Exploratory Data Analysis (EDA). These 5 methods (`head` , `tail` , `sample` , `shape` , `info`) form the standard operating procedure for loading any new dataset.

---

1. Visual Inspection

Never assume the data loaded correctly just because Pandas didn't throw an error.

```
import pandas as pd
df = pd.read_csv('employees.csv')

# 1. Check the top 5 rows (default) to verify headers mapped correctly
df.head()

# 2. Check the last 10 rows to ensure file integrity at the EOF marker
df.tail(10)

# 3. Check 5 random rows to verify data isn't uniquely structured at the edges
df.sample(5, random_state=42) # random_state ensures reproducibility
```



2. Dimensionality with `.shape`

`.shape` is not a method; it is a DataFrame attribute derived from the underlying NumPy architecture. Therefore, it does not use parentheses `()`.

```
dimensions = df.shape
print(f"The dataset has {dimensions[0]} rows and {dimensions[1]} columns.")
```

---

3. Deep Dive with `.info()`

`df.info()` is the diagnostic x-ray of your DataFrame. You must learn to read its output fluently.

```
df.info()

# Expected output breakdown:
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 100 entries, 0 to 99                <- Validates exact row count
# Data columns (total 3 columns):                 <- Validates exact column count
#  #   Column   Non-Null Count  Dtype
# ---  ------   --------------  -----
#  0   ID       100 non-null    int64              <- Perfect column, no missing data
#  1   Name     98 non-null     object             <- String column, 2 missing
values!
#  2   Salary   100 non-null    float64            <- Decimal column
```

(Cruciual takeaway: Comparing the `Non-Null Count` against the `RangeIndex` immediately exposes columns with missing `NaN` data).

Pro-Tip: Memory Usage

By default, `df.info()` estimates memory. To get the exact RAM usage of the DataFrame (crucial for large files), pass `memory_usage='deep'`.

```
df.info(memory_usage='deep')
```

Group Data with groupby

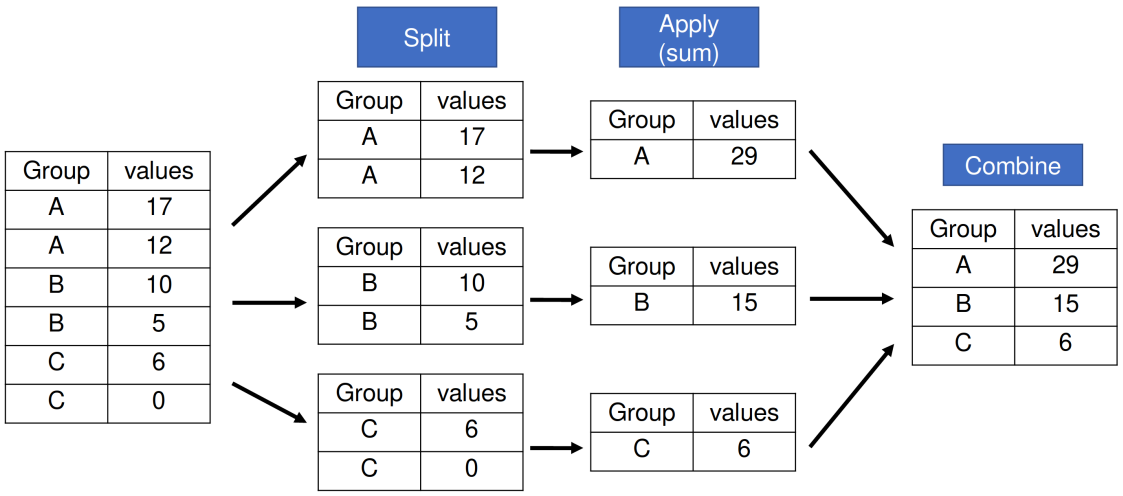
1. Why Group?

Data is only valuable if it tells a story. Raw data is often too noisy. By grouping data, we find patterns across categories. We move from looking at individual "data points" to looking at "segment behavior".

2. Analogy: The Laundry Service

The "Split-Apply-Combine" pattern is the industry standard for grouped operations:

- **Split**: Subdividing the data based on a key (e.g., Color).
- **Apply**: Calculating something for each group (e.g., counting items).
- **Combine**: Merging results back into a clean summary table.



3. Key Concepts

- **`df.groupby('Col')`** : Returns a `DataFrameGroupBy` object (The "Lazy" object).
- **Aggregation Functions**: Tools like `.sum()`, `.mean()`, `.count()`, and `.min()/max()`.
- **Selecting Columns**: Using `df.groupby('Col')['Value_Col']` to target specific data.
- **The `numeric_only` Rule**: In modern Pandas (2.0+), if you calculate a mean or sum on a group with text columns, you **must** specify `numeric_only=True` or Pandas will throw a `TypeError`.

4. Under the Hood: The "Lazy" GroupBy

What happens when you run `grouped = df.groupby('City')`?

1. **No Calculation (Yet)**: Pandas does **not** calculate anything initially. It simply scans the 'City' column and creates a "map" of which rows belong to which city.
2. **The Metadata Map**: It creates a hash table where the keys are the cities and the values are lists of row numbers (indices).
3. **Efficiency**: This "Lazy Evaluation" is incredibly fast because Pandas waits until you specify the math (like `.mean()`) before it actually looks at the numeric data.

5. Detailed Examples

1. Basic Summation

```
import pandas as pd
data = {'Region': ['North', 'South', 'North', 'South'], 'Sales': [1000, 2000, 1500, 3000]}
df = pd.DataFrame(data)

# Total sales per region
region_totals = df.groupby('Region')['Sales'].sum()
```

2. Multi-column Selection

```
# Multiple metrics for one group (e.g., Sales and Units)
stats = df.groupby('Region')[['Sales', 'Units']].mean()
```

6. Common Pitfalls

- **The "Lost" Columns**: When you group by 'Department', any column that isn't the group key OR a numeric value will be dropped by default aggregation (like `.mean()`) because you can't find the "average" of a String.
- **The Forgotten Aggregation**:

  ```
  res = df.groupby('City')
  print(res) # Output: <pandas.core.groupby.generic.DataFrameGroupBy object...>
  ```

  Fix: Always follow groupby with an action like `.sum()` or `.mean()`.
- **Numeric vs. Non-numeric**: Attempting to `.mean()` on a column of Strings will result in an error or an empty result in newer Pandas versions (Numeric-only rule).

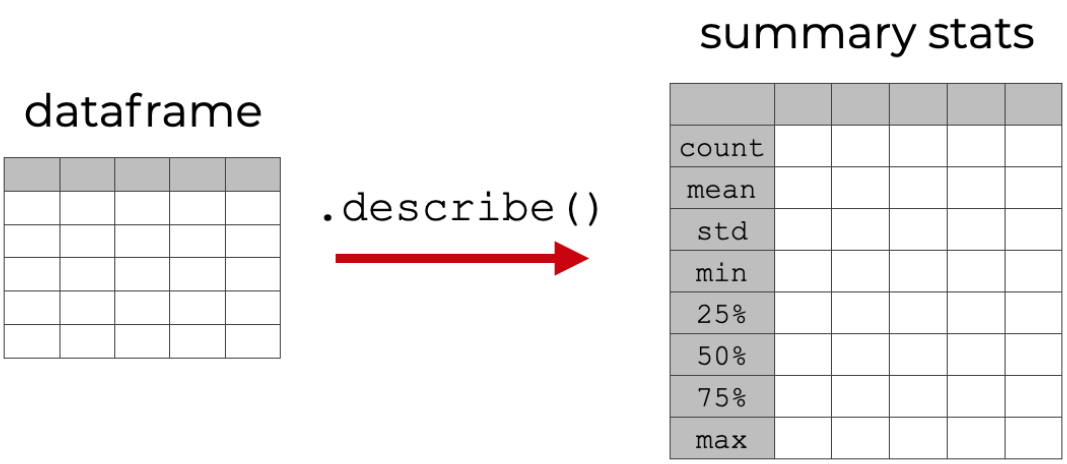
Get Summary Statistics with describe

1. Why describe()?

In exploratory data analysis (EDA), `describe()` is often the very first command an analyst runs after loading data. It provides a statistical snapshot that reveals data quality issues (like negative prices or extreme outliers) and structural trends instantly.

2. Analogy: The Executive Summary

- **The 30,000-ft View**: Seeing the whole forest, not the individual trees.
- **The Vital Signs**: Checking the "Pulse" of the dataset.



3. Key Concepts

- `.describe()` : Automatically identifies numerical columns and calculates a 5-number summary (Min, Q1, Median, Q3, Max) plus the Count, Mean, and Std.
- **Percentiles**: The 25%, 50%, and 75% marks.
  - *Example*: 75% means "75% of the values in this column are BELOW this number."
- **Non-Numerical Data**: By default, `describe()` ignores text columns. To see a summary of strings (Unique count, Top value, Frequency), use `df.describe(include='all')` .

4. Under the Hood: Handling Holes (NaN)

How does Pandas summarize data when some entries are missing?

1. **Silent Exclusion**: Unlike raw Python math, Pandas statistics methods **ignore** `NaN` by default.
2. **The Denominator logic**: When calculating the `mean` , Pandas divides the sum by the count of *valid* numbers, not the total number of rows.
3. **Memory Efficiency**: `describe()` is a "Reduction" operation. It processes the entire column and returns a very small Series or DataFrame, making it extremely memory-efficient for reporting.

5. Detailed Examples

1. Basic Numerical Summary

```
import pandas as pd
df = pd.DataFrame({'Age': [20, 30, 40, 50, 1000]}) # 1000 is an outlier!

print(df.describe())
```

```
# Look at the 'max'. If it's 1000 for 'Age', you instantly know there's a data entry
error.
```

2. Including Text/Category Data

For strings, it shows `unique` (how many categories), `top` (most common), and `freq` (how often 'top' appears).

```
df = pd.DataFrame({'Status': ['Active', 'Active', 'Pending']})
print(df.describe(include='object'))
```

3. Customizing Percentiles

```
# See more specific slices (10th and 90th percentile)
df.describe(percentiles=[0.1, 0.9])
```

6. Common Pitfalls

- **The "Missing" Columns**: Expecting to see 'Name' in the numerical summary. (Pandas splits them; you need `include='all'`).
- **Standard Deviation Confusion**: Forgetting that `std` measures how spread out the data is. A `std` of 0 means every single row has the exact same value.
- **Transposing**: For very wide DataFrames (many columns), the report is hard to read. Use `df.describe().T` to flip it and read columns as rows.

Handling outliers using Pandas logic

Core Concept: Extreme Value Management

Machine Learning models are sensitive to massive numbers. A single typo (e.g., `Salary = 5,000,000` instead of `50,000`) can pull the statistical averages of an entire dataset wildly off-target. Outliers must be algorithmically identified and managed.

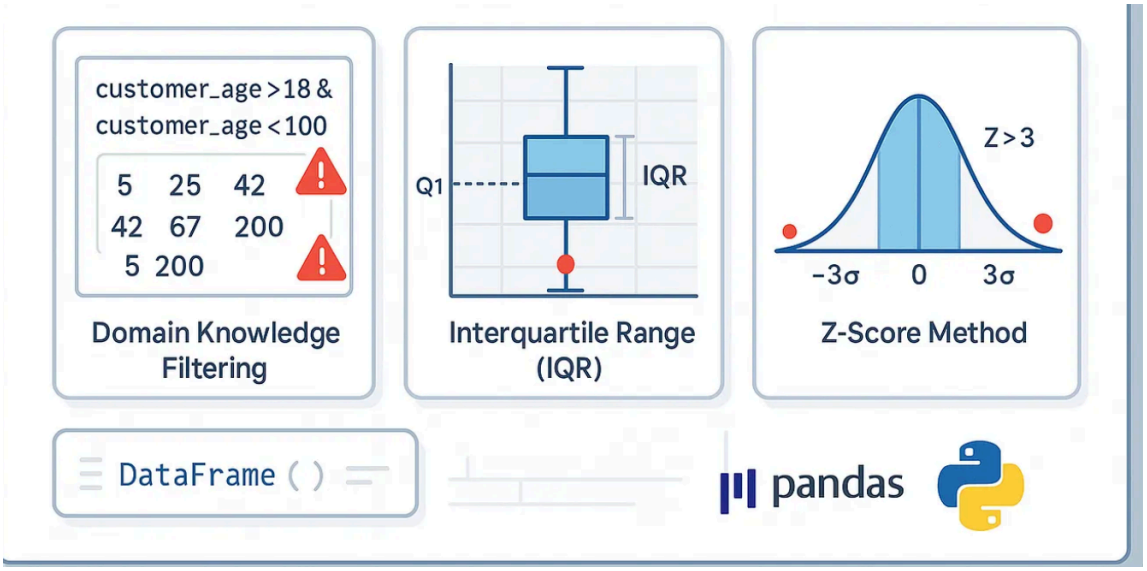
---

1. Algorithmic Detection: The IQR Method

The Interquartile Range (IQR) focuses on the "middle 50%" of your data, ignoring the extremes to find a stable baseline.

```
import pandas as pd

# Assume 'df' holds 'Income' data
Q1 = df['Income'].quantile(0.25)
Q3 = df['Income'].quantile(0.75)
IQR = Q3 - Q1

# The standard multiplier is 1.5. A multiplier of 3.0 represents extreme outliers.
```

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```



---

2. Hard Removal: Trimming

By leveraging standard boolean indexing, we can permanently drop rows containing outliers. This is aggressive and reduces dataset size.

```
# 1. Ask the question: Which rows are "normal"?
is_normal_mask = (df['Income'] >= lower_bound) & (df['Income'] <= upper_bound)

# 2. Extract only the normal rows
trimmed_df = df[is_normal_mask]
```

---

3. Soft Mitigation: Clipping (Winsorization)

Instead of dropping the entire row—which might contain perfectly valid data in other columns like `Age` or `Education` —we compress the extreme value down to the boundary limits.

```
# The built in .clip() method takes the lower and upper arguments
df['Income_Clipped'] = df['Income'].clip(lower=lower_bound, upper=upper_bound)

# Example: If the upper bound is $150,000
# A raw income of $500,000 becomes $150,000.
# A raw income of $40,000 remains $40,000.
```

---

4. Alternate Detection: The Z-Score Method

While IQR is robust (resistant to outliers), the Z-Score method assumes your data is normally distributed (a bell curve). It measures how many standard deviations a value is away from the mean. A Z-score > 3 or < -3 is generally considered an outlier.

```
mean_income = df['Income'].mean()
std_income = df['Income'].std()

# Calculate Z-scores manually
df['Z_Score'] = (df['Income'] - mean_income) / std_income

# Filter to keep only rows within 3 standard deviations
z_score_trimmed_df = df[(df['Z_Score'] > -3) & (df['Z_Score'] < 3)]
```

Use corr() and cov() for Correlation & Covariance Matrices

Core Concept: Multivariate Relationships

When building predictive models, identifying "features" (variables) that have strong mathematical relationships with your "target" (the thing you are trying to predict) is crucial. `df.corr()` automates this discovery process across millions of data points instantly.

---

1. Generating a Correlation Matrix

The `.corr()` method evaluates every numeric column against every other numeric column in the DataFrame. Non-numeric columns (like Strings/Objects) are automatically ignored.

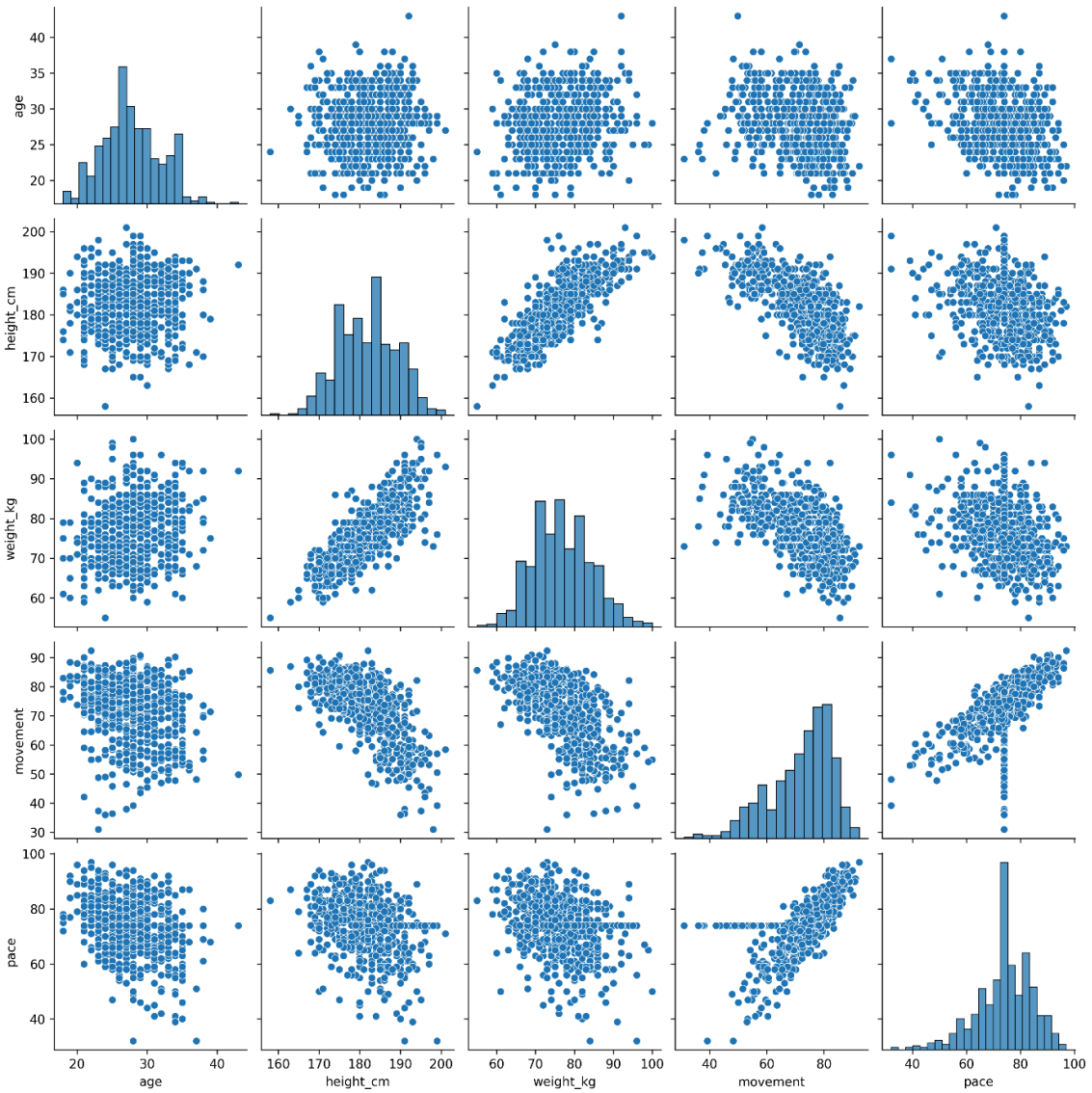
```
import pandas as pd

housing = pd.DataFrame({
    'Price':        [450000, 320000, 580000, 275000, 710000, 390000],
    'Square_Feet':  [1800, 1350, 2200, 1100, 2800, 1600],
    'Num_Bedrooms': [3, 2, 4, 2, 5, 3],
    'Age_of_Home':  [15, 32, 8, 45, 5, 22],
    'Address':      ['101 Maple St', '202 Oak Ave', '303 Pine Rd', '404 Elm Dr',
'505 Cedar Ln', '606 Birch Blvd']
})

# Assume a real estate DataFrame 'housing'
# Columns: ['Price', 'Square_Feet', 'Num_Bedrooms', 'Age_of_Home', 'Address']

# Calculate the Pearson correlation matrix
matrix = housing.corr(numeric_only=True)

# Look at how everything relates specifically to 'Price'
price_factors = matrix['Price']
```

```
# Output Example:
# Price           1.000   (Perfect correlation with itself)
# Square_Feet     0.895   (Strong positive relationship)
# Num_Bedrooms    0.650   (Moderate positive relationship)
# Age_of_Home    -0.420   (Moderate negative relationship - older is cheaper)
```

Pearson vs Spearman vs Kendall

- `method='pearson'` (Default): Measures linear relationships. Assumes data is normally distributed.
- `method='spearman'` : Measures monotonic relationships using rank-order. Better for data with extreme outliers or non-linear curves.
- `method='kendall'` : Another rank-based calculation, highly resilient to small sample sizes.

```
# Use Spearman if you have massive, unclipped outliers
robust_matrix = housing.corr(method='spearman', numeric_only=True)
```



2. Generating a Covariance Matrix

Covariance measures the joint variability of two variables.

```
# Calculate the Covariance matrix
cov_matrix = housing.cov(numeric_only=True)
```

Why don't we use Covariance visually? Because it is heavily influenced by scale. If you measure house prices in US Dollars and `Square_Feet` in inches, the covariance will be a massive, uninterpretable multi-million integer. If you measure house prices in Millions of Dollars and `Square_Feet` in Acres, the covariance will be a tiny fraction.

Correlation (`.corr()`) solves this by dividing the covariance by the standard deviations of both variables, mathematically trapping the result between `-1` and `1` regardless of the scale.

Rule of thumb: Read `.corr()` with your eyes. Feed `.cov()` to your algorithms.

---

1. What is a Heatmap?

A heatmap visualizes 2-dimensional data by replacing numbers with corresponding colors from a gradient scale. The darker or lighter the color, the higher or lower the underlying value. It is the perfect chart for analyzing patterns in matrix-like data (such as pivot tables).

2. Using sns.heatmap()

To create a heatmap, you need 2D data (like a Pandas DataFrame, a Numpy 2D array, or a list of lists). Seaborn's `sns.heatmap()` maps these numbers to a color palette automatically.

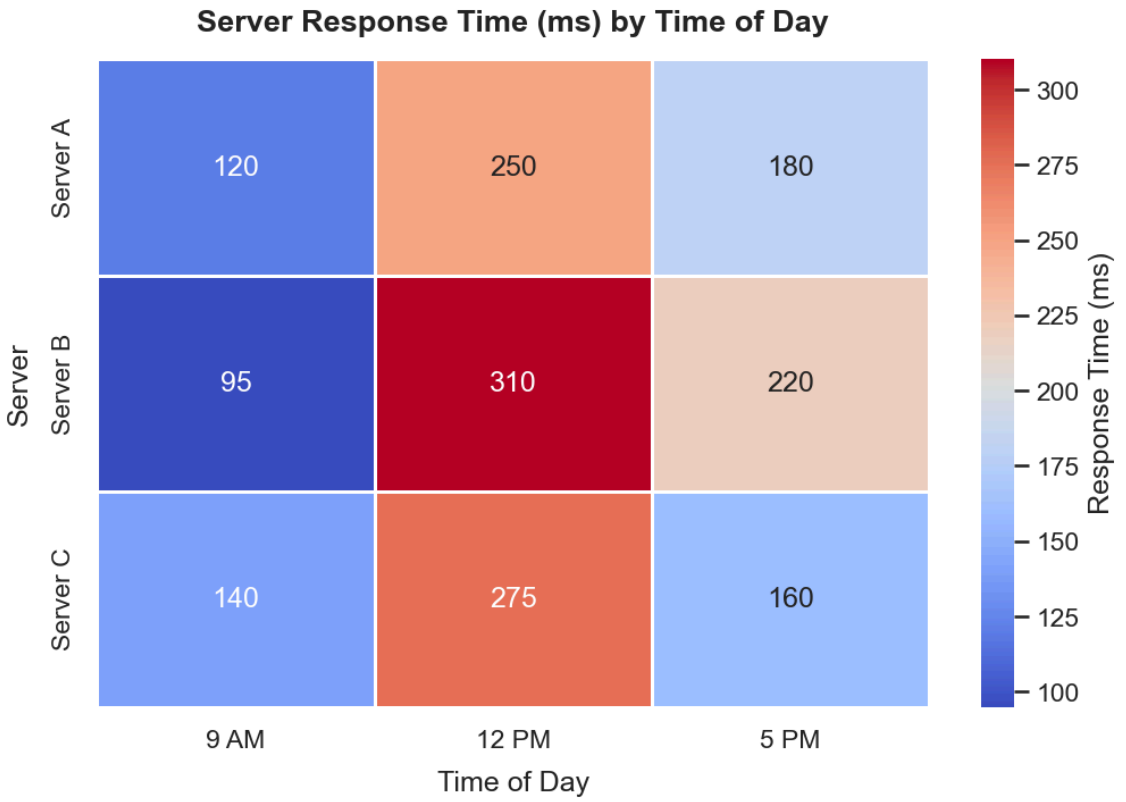
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Creating a 2D matrix of data
data = pd.DataFrame({
    'Jan': [100, 150, 200],
    'Feb': [110, 160, 210],
    'Mar': [120, 170, 220]
}, index=['2020', '2021', '2022'])

# Create a basic heatmap
sns.heatmap(data)
plt.title("Basic Heatmap")
plt.show()
```

3. Customizing the Heatmap

Heatmaps are highly customizable to improve readability.

- **annot=True**: Drops the actual numerical values on top of the colored blocks.
- **cmap**: Changes the color palette (e.g., `'coolwarm'` , `'Blues'` , `'viridis'`).

- **linewidths**: Adds thin lines between cells to separate them clearly.
- **fmt**: Formats the annotated text (e.g., `fmt='.1f'` for 1 decimal place, `fmt='d'` for integers).




```
# A customized, highly readable heatmap
sns.heatmap(data, annot=True, cmap='coolwarm', linewidths=1, fmt='d')
plt.title("Customized Monthly Data")
plt.show()
```

4. Key Considerations

When choosing a `cmap`, think about your data semantics:

- **Sequential**: `Blues` or `Greens` (good for data ranging from 0 to a high number).
- **Diverging**: `coolwarm` or `RdYlGn` (good for data that has a neutral midpoint, like temperatures or correlations ranging from -1 to 1).

---

1. Adding Basic Text

The `plt.text(x, y, s)` function places a string `s` directly onto the chart's coordinate grid at the designated `x` and `y` exact locations.

```
import matplotlib.pyplot as plt

plt.scatter([1, 2, 3], [10, 20, 30])
```

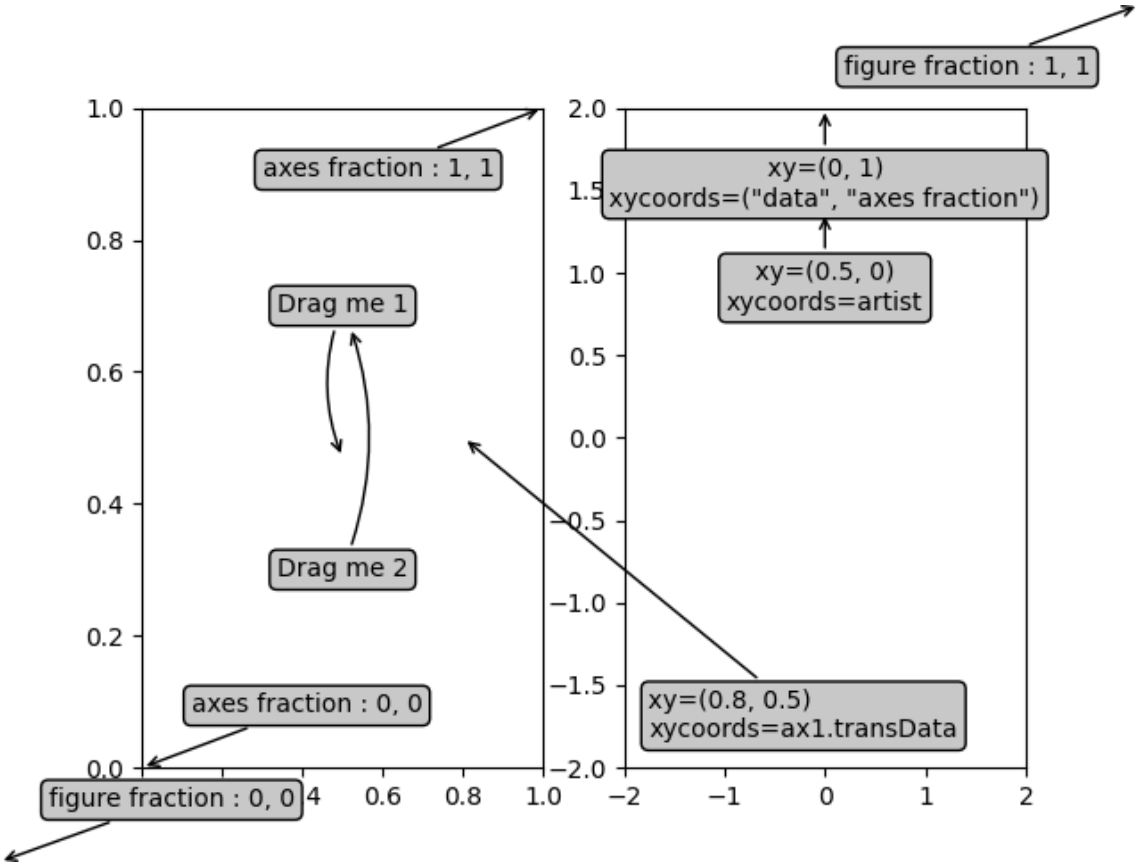
```
# Place text exactly at x=1.5, y=25
plt.text(1.5, 25, "Notice the linear upward trend!", fontsize=12, color='red')
plt.show()
```

2. Dynamic Annotations with Arrows

The `plt.annotate()` function is vastly superior for data storytelling. It places text floating away from the data, and draws an arrow from the text pointing perfectly at an anomaly.

It takes several key parameters:

- `text` : The string you want to display.
- `xy` : A tuple `(x, y)` representing the coordinate you want the tip of the arrow to touch (the data point).
- `xytext` : A tuple `(x, y)` representing the coordinate where the floating text block should be drawn.
- `arrowprops` : A dictionary containing styling instructions for the arrow.



3. Implementing plt.annotate()

```
import matplotlib.pyplot as plt
```

```
years = [2018, 2019, 2020, 2021, 2022]
sales = [100, 110, 45, 120, 130] # Massive drop in 2020

plt.plot(years, sales, marker='o')

# Annotate the 2020 drop
plt.annotate(
    text="Pandemic Drop",
    xy=(2020, 45),              # Arrow points EXACTLY at this data coordinate
    xytext=(2019, 90),          # Text sits high and to the left
    arrowprops=dict(facecolor='black', shrink=0.05) # Draw a black arrow
)

plt.title("Yearly Sales Revenue")
plt.show()
```

4. Arrowprops Dictionary

The `arrowprops` dictionary accepts many styling keys. The `shrink=0.05` argument is particularly useful—it slightly shortens the arrow so it doesn't impale the data point, leaving a clean sliver of empty space between the arrowhead and the marker.

---

1. Generating the Correlation Matrix

Before plotting, you must calculate the underlying math. Calling `.corr()` on a Pandas DataFrame calculates the Pearson correlation coefficient calculated between every single pair of numerical columns.

```
import pandas as pd
import seaborn as sns

# Load a dataset
penguins = sns.load_dataset('penguins')

# Calculate the correlation matrix (numeric_only is required to ignore text columns)
corr_matrix = penguins.corr(numeric_only=True)
print(corr_matrix)
```

2. Visualizing with sns.heatmap()

You pass the resulting `corr_matrix` directly into `sns.heatmap()`. Because correlations are bounded between -1.0 and 1.0, you must carefully configure the visual parameters to optimize readability.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

sns.heatmap(
    corr_matrix,
    annot=True,          # Show the actual correlation numbers
```

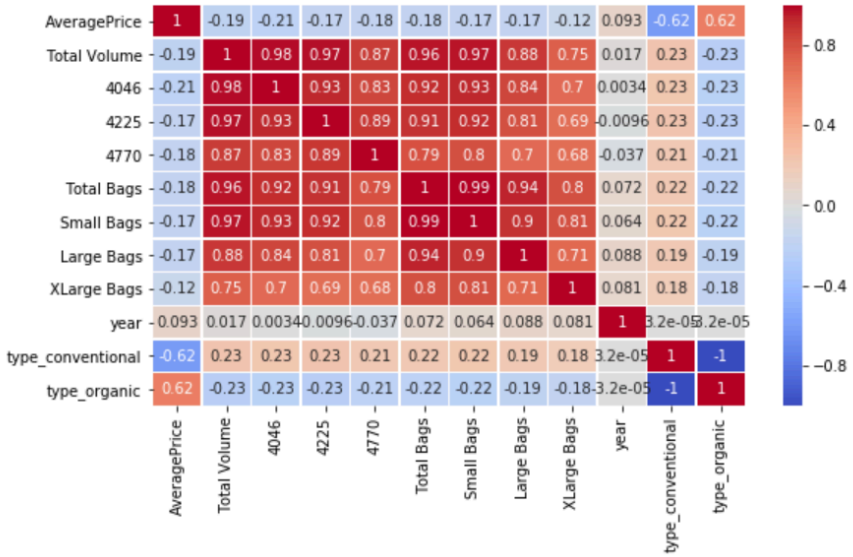
```
    cmap='coolwarm',      # Use a diverging color scheme (Red/Blue)
    vmin=-1,              # Anchor the color scale minimum at -1
    vmax=1,               # Anchor the color scale maximum at 1
    center=0,             # Ensure 0 (no correlation) is exactly in the middle color
    fmt=".2f"             # Round to 2 decimal places
)

plt.title("Penguin Feature Correlation Heatmap")
plt.show()
```

3. Why vmin and vmax matter

If you don't explicitly set `vmin=-1` and `vmax=1`, Seaborn will automatically set the color scale based on whatever the highest and lowest numbers happen to be in your specific dataset (e.g., -0.3 to 0.8). This makes comparing different datasets impossible and visually misrepresents the strength of the correlation (making a weak 0.4 correlation look dark red because it happened to be the highest value).

```
In [60]: plt.figure(figsize=(9,5))
         sns.heatmap(df.corr(),cmap='coolwarm',annot=True,linewidth =.5)
```

```
Out[60]: <matplotlib.axes._subplots.AxesSubplot at 0x1fa0f93eb00>
```



Session 25: APIs: The Agent's Hands

What You'll Learn

In this lesson, you'll learn to:

- **Explain** what an API is and how it acts as a bridge between two programs
- **Apply** Python's `requests` library to send API requests and handle responses
- **Interpret** API responses, status codes, and JSON data
- **Identify** real-world challenges like rate limiting and authentication, and describe how to handle them

---

Detailed Explanation

What Is an API?

Imagine you're at a restaurant. You don't walk into the kitchen and cook your own food. Instead, you tell the waiter what you want. The waiter goes to the kitchen, gets your food, and brings it back to you.

An **API (Application Programming Interface)** works exactly the same way.

- **You** = your Python program (the client)
- **The waiter** = the API
- **The kitchen** = the server (where data lives)

You send a **request** through the API, and the server sends back a **response**. You never touch the server directly.

One-line definition: An API is a messenger that lets two programs talk to each other.

Websites are built for humans — you click, scroll, and read. APIs are built for programs — your Python script sends a request and gets back data.

Why It Matters

You'll need APIs constantly as a data professional. Here's why:

- You want to pull live weather data into your project? Use a weather API.
- You want to use ChatGPT's language model in your own app? Use OpenAI's API.
- You want to fetch stock prices, tweets, or news? All APIs.

Without APIs, your program has no way to talk to the outside world. Every modern data pipeline, product, or application is built on top of APIs.

> ***You will use this when:*** *building data pipelines, automating data collection, working with AI models, or integrating third-party services into your projects.*

How Authentication Works: API Keys

Most APIs don't let just anyone in. They use an **API key** — a secret string of characters that proves you are who you say you are.

Think of it like a reservation at a private restaurant. Without a reservation (API key), you don't get in.

```
Client (Your Program)  --->  API Key Included in Request  --->  Server
                       <---  Response if Key is Valid    <---
```

Important rules about API keys:

- **Never share your API key.** Treat it like a password.
- **Never hardcode it** directly in your code. Use environment variables instead.
- Practice **key rotation** — periodically replace old keys with new ones. This limits damage if a key is ever leaked.

Making API Calls with Python

Python's `requests` library is your tool for communicating with APIs.

The two most common request types:

Method	Purpose
`GET`	Fetch/retrieve data from the server
`POST`	Send data to the server

Basic GET Request — Step by Step:

```
import requests

# Step 1: Define the API endpoint (the URL you're calling)
url = "https://api.example.com/data"

# Step 2: Add your API key in headers
headers = {
    "Authorization": "Bearer YOUR_API_KEY"
}

# Step 3: Send the request
response = requests.get(url, headers=headers)

# Step 4: Check if it worked
print(response.status_code)  # Should be 200 if successful

# Step 5: Parse the response
data = response.json()
print(data)
```

What's happening here?

- `requests.get()` sends a GET request to the URL
- `headers` carries your API key securely
- `.status_code` tells you if the request succeeded or failed
- `.json()` converts the raw response into a Python dictionary

Understanding Status Codes

Every API response comes with a **status code** — a number that tells you what happened.

Code Range	Meaning	Example
`200`	Success	Data returned successfully
`400` series	Client-side error	Wrong URL, missing parameter
`401`	Unauthorized	Bad or missing API key
`404`	Not Found	Endpoint doesn't exist

`500` series	Server-side error	Something broke on the server

Rule of thumb: 2xx = good. 4xx = your mistake. 5xx = their mistake.

Handling Errors Gracefully with Try-Except

APIs can fail — the internet goes down, the server is overloaded, or your key expires. Your code should handle this without crashing.

```
import requests

url = "https://api.example.com/data"
headers = {"Authorization": "Bearer YOUR_API_KEY"}

try:
    response = requests.get(url, headers=headers)
    response.raise_for_status()  # Raises an error for 4xx/5xx codes
    data = response.json()
    print(data)

except requests.exceptions.ConnectionError:
    print("Could not connect. Check your internet.")

except requests.exceptions.HTTPError as e:
    print(f"HTTP error occurred: {e}")
```

Why this matters: In real pipelines processing thousands of API calls, one failed request shouldn't crash your entire program.

Working with JSON

API responses almost always come back as **JSON (JavaScript Object Notation)** — a lightweight, human-readable format for structured data.

JSON looks like a Python dictionary:

```
{
  "user": "Ritz",
  "city": "Bengaluru",
  "score": 98
}
```

After calling `.json()` in Python, you can work with this just like any dictionary:

```
data = response.json()
print(data["user"])   # Output: Ritz
print(data["city"])   # Output: Bengaluru
```

When APIs return nested or complex JSON, you'll need to flatten it into a structured format — like a Pandas DataFrame — before analysis. This is a common step in data engineering pipelines.

Reading API Documentation

Every API comes with **documentation** — think of it as the API's instruction manual.

Good documentation tells you:

- What **endpoints** are available (the URLs you can call)
- What **parameters** to include in your request
- How to **authenticate** (where to put your API key)
- What the **response** looks like (the JSON structure)
- **Rate limits** — how many requests you're allowed

Practical tip: Most API docs include code examples in Python. Always start there. Read the docs before writing a single line of code.

Real-World Challenges

Rate Limiting

APIs don't let you send unlimited requests. A common limit might be **3 requests per second**. If you need to pull data for 10,000 records, that's over 55 minutes just waiting.

Solutions include:

- **Parallel processing** — sending multiple requests simultaneously
- **Multiple API keys** — distributing load across keys
- **Caching** — storing results so you don't re-fetch what you already have

Each solution adds complexity and cost. This is a real engineering tradeoff.

Secure Key Storage

In production environments (cloud systems, company servers):

- Never store API keys in your code files
- Use **environment variables** or a **secrets manager** (like AWS Secrets Manager)

```
import os

api_key = os.environ.get("MY_API_KEY")  # Reads key from environment
```

Large JSON Responses

Some APIs return deeply nested, complex JSON. You'll need to write transformation logic to flatten it into a usable table or DataFrame — a very common task in data engineering.

Frameworks That Simplify API Work

Langchain is a popular Python framework that wraps multiple LLM APIs (OpenAI, Gemini, Claude, etc.) into a single, consistent interface. Instead of learning the unique syntax for each API, you write one Langchain call and swap out the model easily.

When to use a framework vs. raw requests:

Situation	Use
Working with LLM APIs repeatedly	Langchain or vendor SDK
One-off API call or custom API	Raw `requests` library
API with no Python package available	Always use raw `requests`

Knowing the fundamentals of `requests` is non-negotiable. Frameworks come and go. The underlying HTTP knowledge stays with you.

Common Mistakes and How to Fix Them

Mistake	Fix
Hardcoding API keys in scripts	Use `os.environ` or a secrets file
Not checking the status code	Always check before parsing `.json()`
Assuming the API is always up	Wrap calls in `try-except`
Ignoring rate limits	Add `time.sleep()` between requests or implement retry logic
Not reading the docs	Read documentation before writing any code

---

Key Takeaways

- An **API** is a messenger between your program and a server — you send requests, it returns responses
- **API keys** authenticate your access; treat them like passwords and never hardcode them
- Python's **`requests` library** is your primary tool — `GET` fetches data, `POST` sends data
- **Status codes** tell you what happened: 200 = success, 4xx = your error, 5xx = server error
- **JSON** is the standard format for API data — parse it with `.json()` and work with it like a dictionary
- Real-world challenges like **rate limiting**, **key security**, and **large JSON handling** require thoughtful system design, not just code

Mental model: Think of an API as a drive-through window. You pull up (make a request), place your order (send parameters + your key), and receive your food (JSON response) — without ever entering the kitchen.

Coming up next: Now that you can fetch data from APIs, you'll learn how to clean, transform, and load that data into structured formats — the core of any data pipeline.

Session 26: API Security & Ethics

Why API Security Matters

Imagine you printed your house key and posted a photo of it on a public noticeboard. Anyone could copy it and walk into your home.

Leaving your API key exposed in your code does exactly that.

Here's what can go wrong:

- Someone uses your key to make thousands of API calls
- You receive a massive bill (some APIs charge per request)
- Your account gets suspended for violating usage limits
- Sensitive data gets accessed without authorisation

The scary part? **Bots constantly scan public repositories on GitHub** looking for exposed keys. This isn't a rare edge case — it happens within minutes of a key being pushed to a public repo.

You'll need this awareness every time you write code that connects to an external service — which is almost every real-world project.

Protecting Your API Keys: The Right Way

❌ What NOT to Do — Hardcoding Keys

This is the most common mistake beginners make:

```
# NEVER do this
api_key = "sk-abc123yourrealsecretkeyhere"

response = requests.get("https://api.example.com/data", headers={"Authorization":
api_key})
```

If this file gets uploaded to GitHub — even accidentally — your key is compromised.

✅ The Right Approach — Environment Variables

Instead of writing the key directly in your code, store it in a separate file called `.env`. Your Python script then loads the key from that file at runtime.

Step 1: Create a `.env` file in your project folder

```
API_KEY=sk-abc123yourrealsecretkeyhere
```

Step 2: Install the `python-dotenv` library

```
pip install python-dotenv
```

Step 3: Load the key in your Python script

```
from dotenv import load_dotenv
import os

load_dotenv()  # Reads the .env file

api_key = os.getenv("API_KEY")  # Fetches the key safely

print(api_key)  # Confirms it loaded — remove this in production!
```

Your key never appears in your actual code. Clean, safe, and professional.

✅ Keeping `.env` Out of GitHub — `.gitignore`

Creating a `.env` file is only half the job. You also need to make sure Git doesn't accidentally upload it.

Create a file called `.gitignore` in your project folder and add:

```
.env
```

That single line tells Git: *"Ignore this file. Never track it. Never push it."*

Think of `.gitignore` as a "do not photograph" sign — the file exists on your machine, but it never leaves it.

Key Rotation: Change Your Locks Regularly

Even if you're careful, there's still a small chance a key gets leaked through logs, shared screens, or old backups.

Key rotation means deleting your current API key and generating a new one — on a regular schedule, typically every 15–30 days. It's like changing your Wi-Fi password periodically. If someone quietly had your old password, they lose access the moment you rotate.

How to rotate a key:

1. Go to the API provider's dashboard
2. Generate a new key
3. Update your `.env` file with the new key
4. Delete the old key from the dashboard

This limits the damage if a key was ever silently compromised.

The Least Privilege Principle

When you create an API key, most platforms let you choose what that key is *allowed* to do. Always choose the minimum permissions necessary.

If your script only needs to **read** data, don't create a key that can also **write** or **delete**. This is called the **Least Privilege Principle** — give each key only the access it needs, nothing more.

It's like giving a delivery person a key to your mailbox, not to your front door.

Cloud Secret Managers (For When You Deploy)

When your code runs on a server or in the cloud, you can't rely on a local `.env` file. Cloud providers offer dedicated services for this:

- **AWS Secrets Manager**
- **Google Cloud Secret Manager**

These services store your keys encrypted and make them available securely to your deployed applications. You don't need to use these right now — but knowing they exist prepares you for real-world deployment.

Ethics in API Usage

APIs are **shared resources**. Think of them like a public library. Everyone has the right to use it — but if one person walks in and photocopies every single book without permission, they're ruining it for everyone else.

Using an API ethically means:

- **Respecting rate limits** — APIs restrict how many requests you can make per minute or per day. Exceeding these limits can get your access blocked.
- **Reading the terms and conditions** — Every API has usage rules. Violating them can result in legal consequences.
- **Not scraping personal data without consent** — Pulling someone's private information through an API without their knowledge is a privacy violation.
- **Not misusing AI APIs** — Tools like ChatGPT's API can be powerful. Using them for harmful, deceptive, or unethical outputs is your responsibility to avoid.

Ethical API usage isn't just about following rules — it's about being a responsible developer who others can trust.

Handling Rate Limiting in Code

Most APIs restrict how many requests you can make in a short period. If you exceed the limit, you'll get an error (usually a `429 Too Many Requests` response).

The simplest way to avoid this is to **add a delay between requests** using `time.sleep()`:

```
import requests
import time

urls = ["https://api.example.com/match/1",
        "https://api.example.com/match/2",
        "https://api.example.com/match/3"]

for url in urls:
    response = requests.get(url)
    print(response.json())
    time.sleep(2)  # Wait 2 seconds before the next request
```

This tells Python: *"After each request, pause for 2 seconds before continuing."*

For more robust applications, you'd also implement **retry logic with back-off** — if a request fails, wait a few seconds and try again, instead of immediately crashing.

Common Mistakes and How to Fix Them

Mistake	Why It's a Problem	Fix
Hardcoding API keys in code	Anyone who sees your code sees your key	Use `.env` and `os.getenv()`
Pushing `.env` to GitHub	Bots scrape repos for exposed keys	Add `.env` to `.gitignore`
Never rotating keys	A silently leaked key stays active indefinitely	Rotate every 15–30 days

Giving keys too many permissions	A stolen key can do more damage	Apply the Least Privilege Principle
Ignoring rate limits	Your access gets blocked, scripts break	Use `time.sleep()` between requests

---

Key Takeaways

- An **API** is a code-to-code bridge. Websites are for humans; APIs are for programs. Every request uses server resources — which is why keys and ethics both matter.
- **Never hardcode API keys** in your scripts. Store them in a `.env` file, load them using `python-dotenv`, and exclude the file from version control using `.gitignore`.
- **Key rotation** (every 15–30 days) and the **Least Privilege Principle** (minimum necessary permissions) are two habits that professional developers build early.
- **Ethical API usage** means respecting rate limits, reading terms of service, and never collecting or exposing personal data without consent. APIs are shared resources — treat them accordingly.
- Use `time.sleep()` to pace your requests and avoid hitting rate limits. In production environments, add retry logic for graceful failure handling.

Mental model: Think of your API key as a credit card. You wouldn't leave it on a public table, hand it to strangers, or let it never expire. Treat it with the same care.

Coming up next: Now that you know how to work with APIs securely and ethically, the next step is putting these skills together — making authenticated API calls, parsing responses, and handling real-world data at scale.

Session 27: EDA Workshop

The Analogy That Ties It All Together

> *Think of yourself as a **film critic who owns a restaurant.***

A movie studio (TMDB) sends you a **daily delivery of raw ingredients** — cast lists, budgets, genres, revenue numbers, ratings. But they arrive jumbled in boxes with no labels (JSON). Your job is to:

1. **Sign for the delivery** and check what arrived (API + authentication)
2. **Sort everything into labelled containers** — genres in one, cast in another, budgets in a third (database tables)
3. **Prep and combine the ingredients** into one clean dish (Pandas joins + cleaning)
4. **Plate it beautifully** so your guests immediately understand what they're eating (EDA + visualizations)
5. **Open the restaurant** so anyone can walk in and explore the menu (Streamlit)

Every step today maps to one stage in that kitchen. Let's cook.

Part 1 — Signing for the Delivery: The TMDB API

What is TMDB and why does it matter?

TMDB (The Movie Database) is a free alternative to IMDB that offers API access for educational purposes — no fees, just a free account. This makes it perfect for real-world practice.

An **API key** is like your delivery pass. Without it, the studio won't hand you anything.

```
# Store your key safely — in Google Colab, use Secrets
from google.colab import userdata
API_KEY = userdata.get('TMDB_API_KEY')
```

Never hardcode your API key directly in the script. Think of it like your restaurant's back-door code — you don't write it on a sticky note on the door.

Reading the menu: API documentation

Before you call any endpoint, **read the documentation**. This is non-negotiable. The docs tell you exactly what to ask for and what you'll get back.

The three endpoints used in this session:

Endpoint	What it returns
`/genre/movie/list`	Genre IDs and names (Action = 28, Comedy = 35...)
`/discover/movie`	A filtered list of movies — by rating, year, genre
`/movie/{id}`	Full details for one movie — cast, director, revenue, runtime

Making the first call

```
import requests

url = "https://api.themoviedb.org/3/genre/movie/list"
params = {"api_key": API_KEY, "language": "en-US"}

response = requests.get(url, params=params)

if response.status_code == 200:
    data = response.json()
    print(data)
else:
    print(f"Request failed: {response.status_code}")
```

What comes back? JSON — a nested dictionary. Not a table. Not a spreadsheet. Just a blob of structured text.

```
{
  "genres": [
    {"id": 28, "name": "Action"},
    {"id": 12, "name": "Adventure"},
    {"id": 35, "name": "Comedy"}
```

```
  ]
}
```

Common confusion — `response.text` vs `response.json()`

- `response.text` gives you a raw string — the whole response as plain text
- `response.json()` parses that string into a Python dictionary you can actually work with

Always use `.json()` when the API returns JSON data.

Here's how the full ingestion pipeline flows — from API call to DataFrame:---

Part 2 — Sorting the Containers: Database Design & Storage

Why not just dump everything into one table?

Imagine you received a box of ingredients and instead of sorting them — you just threw everything into one giant bowl. Onions mixed with chocolate, raw chicken on top of strawberries. That's what one giant table looks like. Relational databases keep things **separated, labelled, and linked**.

The session designed **five tables** for the TMDB data:

Table	What it stores
`genre`	`genre_id`, `genre_name`
`movies`	`movie_id`, `title`, `budget`, `revenue`, `rating`, `runtime`...
`movie_genre`	`movie_id`, `genre_id` (the link between movies and genres)
`cast`	`movie_id`, `actor_name`, `role_order`
`director`	`movie_id`, `director_name`

The `movie_id` is the **primary key** — it's the unique label on every container. Every other table references it as a **foreign key** — so you can always trace an actor or a genre back to the exact movie it belongs to.

Creating and inserting into SQLite

```
import sqlite3

conn = sqlite3.connect("tmdb_movies.db")
cursor = conn.cursor()

cursor.execute("""
```

```
    CREATE TABLE IF NOT EXISTS genre (
        genre_id INTEGER PRIMARY KEY,
        genre_name TEXT
    )
""")

# Insert genres fetched from the API
for genre in data['genres']:
    cursor.execute(
        "INSERT OR IGNORE INTO genre (genre_id, genre_name) VALUES (?, ?)",
        (genre['id'], genre['name'])
    )

conn.commit()
```

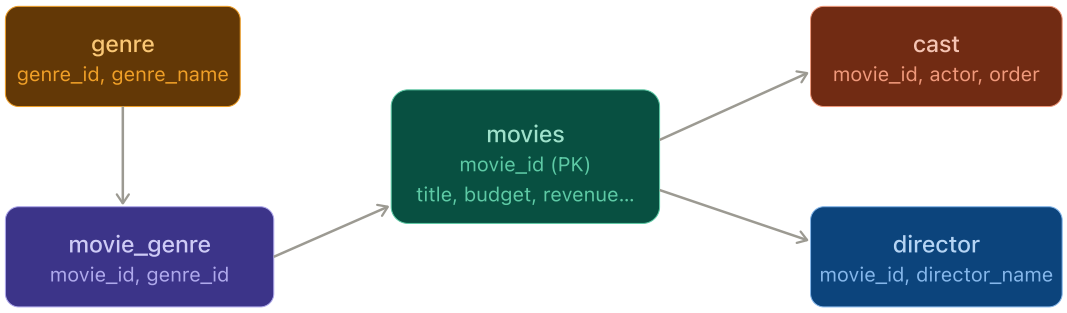
Handling rate limits — be a polite guest

TMDB doesn't want you hammering their servers with thousands of requests per second. If you do, they'll cut you off. The fix is simple:

```
import time

for movie_id in movie_ids:
    response = requests.get(f"https://api.themoviedb.org/3/movie/{movie_id}",
params={"api_key": API_KEY})
    data = response.json()
    # ...insert into DB...
    time.sleep(0.3)  # Wait 300ms between each call
```

`time.sleep()` is a small courtesy that keeps the API happy and your data flowing.

Here's how the five tables relate to each other:---



movie_id links all tables — the key ingredient label

Part 3 — Wash and Prep: Cleaning and Joining with Pandas

Loading from the database back into Pandas

The containers are sorted. Now you bring everything to the prep counter:

```
import pandas as pd
import sqlite3

conn = sqlite3.connect("tmdb_movies.db")

movies_df   = pd.read_sql("SELECT * FROM movies", conn)
genre_df    = pd.read_sql("SELECT * FROM genre", conn)
mapping_df  = pd.read_sql("SELECT * FROM movie_genre", conn)
cast_df     = pd.read_sql("SELECT * FROM cast", conn)
director_df = pd.read_sql("SELECT * FROM director", conn)
```

Joining the tables into a master dataset

A movie has multiple genres. You need to **aggregate** those genres per movie first, then join:

```
# Aggregate genres per movie into one comma-separated string
genre_agg = mapping_df.merge(genre_df, on="genre_id")
genre_agg = genre_agg.groupby("movie_id")['genre_name'].apply(lambda x: ',
'.join(x)).reset_index()
genre_agg.columns = ['movie_id', 'genres']

# Merge everything into one master DataFrame
master_df = movies_df.merge(genre_agg, on="movie_id", how="left")
master_df = master_df.merge(director_df, on="movie_id", how="left")
```

You can also do this directly in SQL using `JOIN` — same result, different tool. Both approaches are valid.

Cleaning the master dataset

```
# Check for missing values
print(master_df.isnull().sum())

# Drop rows where revenue or budget is 0 (unreliable data)
master_df = master_df[(master_df['budget'] > 0) & (master_df['revenue'] > 0)]

# Check and fix data types
print(master_df.dtypes)
master_df['release_date'] = pd.to_datetime(master_df['release_date'])

# Remove duplicates
master_df.drop_duplicates(subset='movie_id', inplace=True)
```

Important: Budget = 0 and Revenue = 0 in TMDB often means **data wasn't reported**, not that the movie was free. You drop these because they'd distort your analysis. This is a judgment call — and making good judgment calls is what separates a data analyst from someone who just runs code.

Part 4 — Plate the Dish: Visualizing Movie Insights

Why visuals, not just numbers?

A column of 5,000 movie budgets tells you nothing. A scatter plot of budget vs revenue immediately shows you which genres spend big and earn big — and which ones lose money.

The chef analogy: **You've prepped a beautiful meal. Now plate it so your guests can actually enjoy it.**

Genre frequency — what types of movies get made most?

```
import matplotlib.pyplot as plt

genre_counts = master_df['genres'].str.split(', ').explode().value_counts()

genre_counts.plot(kind='bar', figsize=(12, 5), color='steelblue')
plt.title("Most Common Movie Genres")
plt.xlabel("Genre")
plt.ylabel("Number of Movies")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Budget vs revenue — does spending more earn more?

```
import seaborn as sns

plt.figure(figsize=(8, 5))
sns.scatterplot(data=master_df, x='budget', y='revenue', alpha=0.5, color='coral')
plt.title("Budget vs Revenue")
plt.xlabel("Budget ($)")
plt.ylabel("Revenue ($)")
plt.show()
```

A scatter plot here shows you the **ROI story** of the movie industry at a glance. High-budget films cluster in the top right. Some low-budget films punch far above their weight.

Rating distribution — what ratings are most common?

```
sns.histplot(master_df['vote_average'], bins=20, kde=True, color='purple')
plt.title("Distribution of Movie Ratings")
plt.xlabel("Rating (out of 10)")
plt.show()
```

Release month analysis — when do studios release their best films?

```
master_df['release_month'] = master_df['release_date'].dt.month

monthly_avg = master_df.groupby('release_month')['revenue'].mean()
monthly_avg.plot(kind='bar', figsize=(10, 4), color='teal')
```

```
plt.title("Average Revenue by Release Month")
plt.xlabel("Month")
plt.ylabel("Avg Revenue ($)")
plt.show()
```

Common confusion — which chart to use when:

- **Bar chart** → comparing categories (genres, months, directors)
- **Scatter plot** → relationship between two numbers (budget vs revenue)
- **Histogram** → distribution of one number (ratings, runtime)
- **Line chart** → change over time (yearly trends)

The Full Kitchen, Start to Finish---

Quick Reference: Other APIs to Practice With

You don't have to stop at TMDB. The same skills you built today work on any API:

- **Pokémon API** (`pokeapi.co`) — great for simple practice, no key needed
- **OpenWeather API** — temperature, humidity, forecasts by city
- **GitHub API** — pull repo stats, commits, contributor data
- **Any API with docs** — if there's documentation and a GET endpoint, you can build a pipeline around it

The skill is not TMDB-specific. The skill is: **read docs → make a request → parse JSON → clean → analyse.**

Key Takeaways

- **The TMDB API** gives you free, rich movie data — genres, cast, revenue, ratings — all via simple HTTP GET requests. Always read the documentation before calling any endpoint.
- **Database design matters.** Splitting data into five related tables (movies, genre, movie_genre, cast, director) keeps it organised and queryable. The `movie_id` is the thread that connects everything.
- **Cleaning is a judgment call, not just a checklist.** Dropping rows where budget = 0 isn't just removing nulls — it's making a decision about data reliability. That thinking is what makes a good analyst.
- **Visualizations tell the story.** Bar charts for genre counts, scatter plots for budget vs revenue, histograms for rating distribution — each chart answers a specific question. Always pick the right chart for the right question.

- **The ETL pattern is everywhere.** Extract from an API, Transform in Pandas, Load into a database — this is the backbone of real-world data engineering. You just built a mini version of it.
- **Mental model:** Think of the whole pipeline as a kitchen. TMDB is the supplier, your database is the labelled pantry, Pandas is the prep counter, your charts are the plated dishes, and Streamlit is the restaurant where guests experience it all.

> ***This wraps up the module.*** *You started by learning what data is. Today you built a complete, end-to-end data pipeline — from a live API to a structured database to cleaned Pandas DataFrames to real visual insights. That is industry-level work. The only thing left is to keep building.* 🏗️